Mathematical Foundations of Deep Neural Networks

Lecture Notes (Ch. 1–10)

Contents

Chapter 1. Linear Regression and Gradient Descent 7

1 The Regression Problem 7

2 Matrix Calculus Toolkit 7

3 Normal Equations 8

4 Gradient Descent 9

4.1 The algorithm . . . 9

4.2 Convergence on least squares . . . 9

5 Polynomial Features and Generalization 10

5.1 Feature maps . . . 10

5.2 Training error versus generalization . . . 10

6 Proof Supplement: Uniqueness and the Pseudoinverse 10

7 Selected exercises 11

Chapter 2. Probability, Likelihood and Information Theory 13

1 Events and Conditional Probability 13

2 Random Variables and Distributions 13

3 Maximum Likelihood Estimation 14

4 Bayesian Inference and MAP 14

4.1 Beta–Bernoulli conjugacy . . . 15

5 MAP as Regularization 15

5.1 Gaussian mean with a Gaussian prior . . . 15

6 Information and Entropy 16

7 Kullback–Leibler Divergence **17**

8 Mutual Information **18**

9 Cross-Entropy **18**

Chapter 3. Probabilistic Regression, Regularization and Kernel Ridge **20**

1 Gaussian Likelihood Explains Squared Error **20**

2 Ridge Regression **20**

2.1 Definition and closed form 20

2.2 MAP interpretation 20

3 Other Penalties and the Lasso **21**

4 Kernel Ridge Regression **21**

4.1 Feature-space and sample-space forms 22

4.2 Kernels 22

4.3 Worked examples 22

5 Selected exercises **23**

Chapter 4. Classification: Logistic Regression, Softmax and SVM **26**

1 Odds, Logit and the Logistic Model **26**

2 Likelihood, Gradient, Hessian and Convexity **26**

3 Labels in $\{\pm 1\}$, Logistic Loss and BCE **27**

3.1 Surrogate risk bounds 27

4 Multiclass Classification and Softmax **28**

5 Support Vector Machines **28**

5.1 Geometry and the primal problem 28

5.2 Lagrangian, max–min inequality, weak duality 29

5.3 The dual problem 29

6 Selected exercises **30**

Chapter 5. Neural Networks and Backpropagation **33**

1 From Hand-Designed Features to Learned Functions **33**

2 Neurons, Layers and Shapes **33**

3 Why Nonlinearity and Depth **34**

4 Activation Functions and Outputs **34**

5 Computation Graphs and the Scalar Chain Rule **34**

6 Vector Calculus Rules for Backpropagation **35**

6.1 Backpropagation through an MLP 36

7 Worked Results **36**

7.1 Single neurons: MSE versus cross-entropy 36

7.2 Residual MLP 36

8 Selected exercises **37**

Chapter 6. Training and Optimization of Deep Networks **39**

1 Full, Stochastic and Mini-Batch Gradients **39**

2 Activation Functions Revisited **39**

3 Weight Initialization **40**

3.1 Why the scale matters 40

3.2 Zero and symmetric initialization 40

3.3 Xavier and He initialization 41

3.4 Variance of a sigmoid output 42

4 Learning-Rate Schedules **42**

5 Batch Normalization **42**

5.1 Training-time computation 42

5.2 Test time 43

6 Gradient Descent and Taylor's Theorem **43**

7 Smoothness and the Descent Lemma **44**

8 Stochastic Gradient Descent **44**

9 Momentum **45**

9.1 Heavy-ball momentum 45

9.2 Nesterov momentum 45

10 Adaptive Methods **46**

10.1 Exponential moving averages and bias correction 46

10.2 Special cases and invariances 47

Chapter 7. Convolutional Neural Networks and Architectures **48**

1 Why a New Layer **48**

2 Convolution as a Local Dot Product **48**

2.1 A fixed filter: moving average 48

2.2 Single and multiple channels 48

2.3 Parameter counts 49

3 Stride, Padding and Output Size **49**

4 Learning Filters **50**

5 Shift Equivariance **50**

6 1×1 Convolution, Pooling and Global Average Pooling **51**

7 Algebraic Facts about CNN Layers **51**

8 Vocabulary and Costs **52**

9 AlexNet **52**

10 ZFNet and VGG **53**

11 GoogLeNet and the Inception Module **53**

11.1 Naive module 53

11.2 Bottleneck module 53

11.3 Split–transform–merge 54

12 Why Depth Can Fail, and Residual Networks **54**

13 Saliency Maps **54**

Chapter 8. Universal Approximation, Barron's Theorem and Generalization **56**

1 Three Sources of Error **56**

2 Preliminaries from Analysis **56**

2.1 Measures . . . 56
2.2 Normed spaces and density . . . 56

3 Cybenko's Universal Approximation Theorem 57
3.1 The functional-analytic lemmas . . . 57
3.2 Density for discriminatory functions . . . 58
3.3 Continuous sigmoidals are discriminatory . . . 58

4 Barron's Theorem: a Dimension-Free Rate 59
4.1 Fourier preliminaries . . . 59

5 Proof of Barron's Theorem 59
5.1 Convex hulls and Maurey's lemma . . . 60
5.2 Barron functions lie in a convex hull of half-space indicators . . . 61
5.3 Completing the proof of Theorem 1 . . . 62

6 Generalization and Rademacher Complexity 63
6.1 Population and empirical risk . . . 63
6.2 Rademacher complexity . . . 63
6.3 Two-layer networks . . . 64

Chapter 9. Recurrent Neural Networks and LSTM 65

1 Why Recurrent Networks 65

2 The Recurrence 65

3 Backpropagation Through Time 66
3.1 The idea . . . 66
3.2 A worked two-step example . . . 66
3.3 General formulation . . . 67

4 Vanishing and Exploding Gradients 68
4.1 RNNs are very deep networks . . . 68
4.2 Exploding gradients and gradient clipping . . . 69
4.3 Vanishing gradients . . . 69

5 Long Short-Term Memory 69
5.1 The LSTM cell . . . 69
5.2 Gradient flow through the cell . . . 70

6 Applications 71

6.1 Language modeling 71
6.2 Image captioning 72

Chapter 10. Attention and Transformers 74

1 Attention-Based Image Captioning 74

2 Spatial and Channel Attention 75

3 Inductive Bias 75

4 The General Attention Layer 76

4.1 One query 76
4.2 Why divide by $\sqrt{D}$ 76
4.3 Many queries, keys and values 77

5 Permutation Symmetry 78

6 Positional Encoding 78

7 Masked Self-Attention 79

8 Multi-Head Attention 80

9 The Transformer Encoder 80

10 Layer Normalization 81

11 Why Transformers 82

Chapter 1. Linear Regression and Gradient Descent

Regression, normal equations, gradient descent, generalization

1 The Regression Problem

In regression we observe pairs $(\mathbf{x}_i, y_i)$, $i = 1, \dots, n$, with input $\mathbf{x}_i \in \mathbb{R}^k$ and real response y_i, and we want a function that predicts y from $\mathbf{x}$. The simplest choice is a linear function. We motivate the choice of "best line" by the sum of squared errors (SSE): a horizontal benchmark line gives one SSE, rotating it changes the SSE, and plotting SSE against the rotation angle gives a curve that first decreases and then increases. The best line is the minimizer of this curve.

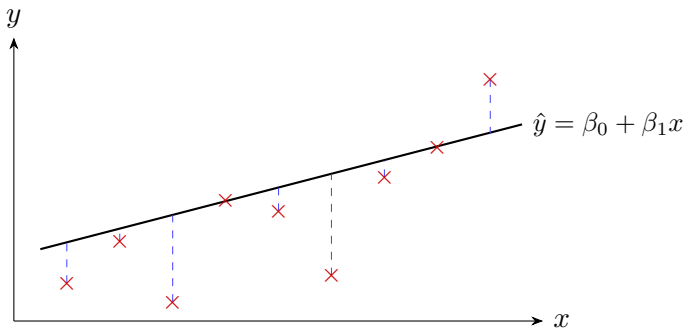
Definition 1.1: linear regression model and design matrix

$$y_i = \beta_0 + \sum_{j=1}^{k} \beta_j x_{ij} + \varepsilon_i, \qquad i = 1, \dots, n.$$

Absorb the intercept with a constant input 1: let $d = k + 1$, $\boldsymbol{\beta} = (\beta_0, \dots, \beta_k)^\top \in \mathbb{R}^d$, and let the **design matrix** $X \in \mathbb{R}^{n \times d}$ have ith row $(1, x_{i1}, \dots, x_{ik})$. Then

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad f(\boldsymbol{\beta}) = \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 = \|\mathbf{y} - X\boldsymbol{\beta}\|^2 = (\mathbf{y} - X\boldsymbol{\beta})^\top (\mathbf{y} - X\boldsymbol{\beta}).$$

Rows of X index samples; columns index features.



Residuals $y_i - \hat{y}_i$ (dashed) of a fitted line; least squares minimizes the sum of their squares.

2 Matrix Calculus Toolkit

Motivation. Every learning algorithm in the course minimizes a scalar loss over a vector or matrix of parameters. Rather than differentiate coordinate by coordinate, we collect a few identities for gradients of linear and quadratic forms that make such computations one-line.

We use the gradient convention: the gradient of a scalar with respect to a vector is a column vector of the same size.

Theorem 1.2: basic gradients

For $\mathbf{a} \in \mathbb{R}^d$ and $A \in \mathbb{R}^{d \times d}$,

$$\nabla_{\boldsymbol{\beta}}(\mathbf{a}^\top \boldsymbol{\beta}) = \mathbf{a}, \qquad \nabla_{\boldsymbol{\beta}}(\boldsymbol{\beta}^\top A \boldsymbol{\beta}) = (A + A^\top)\boldsymbol{\beta}, \qquad \nabla^2_{\boldsymbol{\beta}}(\boldsymbol{\beta}^\top A \boldsymbol{\beta}) = A + A^\top.$$

If A is symmetric the last two become $2A\boldsymbol{\beta}$ and $2A$.

Proof. $\mathbf{a}^\top\boldsymbol{\beta} = \sum_j a_j\beta_j$, so $\partial/\partial\beta_k = a_k$. For the quadratic form, $\boldsymbol{\beta}^\top A\boldsymbol{\beta} = \sum_{i,j} A_{ij}\beta_i\beta_j$. Differentiating with respect to β_k, the product rule picks up the terms with $i = k$ and those with $j = k$:

$$\frac{\partial}{\partial\beta_k}\sum_{i,j} A_{ij}\beta_i\beta_j = \sum_j A_{kj}\beta_j + \sum_i A_{ik}\beta_i = (A\boldsymbol{\beta})_k + (A^\top\boldsymbol{\beta})_k.$$

Differentiating once more gives the constant matrix $A + A^\top$. ■

3 Normal Equations

Motivation. Least squares is the one learning problem with a closed-form solution, and it is the benchmark for everything else. Setting the gradient to zero gives the normal equations; their geometry, projection of $\mathbf{y}$ onto the column space, explains what the fit is doing.

Theorem 1.3: least squares solution

$\nabla f(\boldsymbol{\beta}) = 2X^\top(X\boldsymbol{\beta} - \mathbf{y})$ and $\nabla^2 f = 2X^\top X \succeq 0$. Every minimizer solves the **normal equations**

$$X^\top X\boldsymbol{\beta} = X^\top\mathbf{y}.$$

If X has full column rank d, then $X^\top X \succ 0$ and the unique minimizer is $\hat{\boldsymbol{\beta}} = (X^\top X)^{-1}X^\top\mathbf{y}$.

Proof. Expand $f = \mathbf{y}^\top\mathbf{y} - 2\mathbf{y}^\top X\boldsymbol{\beta} + \boldsymbol{\beta}^\top X^\top X\boldsymbol{\beta}$ (the two cross terms are equal scalars). With the toolkit, $\nabla f = -2X^\top\mathbf{y} + 2X^\top X\boldsymbol{\beta}$ and $\nabla^2 f = 2X^\top X$. For every $\mathbf{v}$, $\mathbf{v}^\top X^\top X\mathbf{v} = \|X\mathbf{v}\|^2 \geq 0$, so f is convex and stationary points are global minimizers. If the columns of X are linearly independent, $X\mathbf{v} \neq \mathbf{0}$ whenever $\mathbf{v} \neq \mathbf{0}$; then $X^\top X \succ 0$, it is invertible, f is strictly convex and the minimizer is unique. ■

Remark 1.4: geometry and rank

The normal equations say $X^\top(\mathbf{y} - X\hat{\boldsymbol{\beta}}) = \mathbf{0}$: the residual is orthogonal to every column of X, so $X\hat{\boldsymbol{\beta}}$ is the orthogonal projection of $\mathbf{y}$ onto the column space. Since the first column is $\mathbf{1}_n$, the residuals sum to zero. Having $n > d$ does not by itself guarantee full rank; with dependent columns the minimizer is not unique and the pseudoinverse gives the minimum-norm solution. Numerically one solves the linear system by QR or SVD rather than forming an inverse.

Example 1.5: least squares on three points

Points $(0, 1), (1, 2), (2, 2)$ and model $y = \beta_0 + \beta_1 x$:

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad X^\top X = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad X^\top\mathbf{y} = \begin{pmatrix} 5 \\ 6 \end{pmatrix}, \quad (X^\top X)^{-1} = \frac{1}{6}\begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix}.$$

Hence $\hat{\boldsymbol{\beta}} = \frac{1}{6}(25 - 18, -15 + 18)^\top = (7/6,\ 1/2)^\top$. Predictions are $(7/6, 5/3, 13/6)$, residuals $(-1/6, 1/3, -1/6)$ (sum zero), and SSE $= 1/6$.

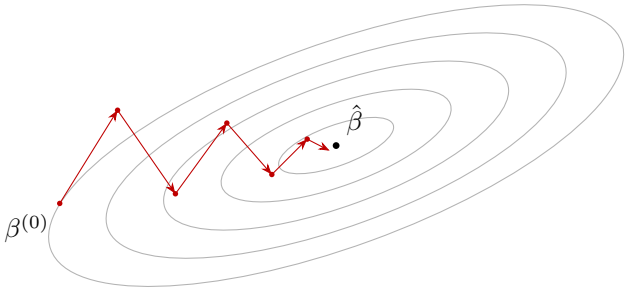
4 Gradient Descent

4.1 The algorithm

Instead of solving the normal equations directly we may iterate

$$\boldsymbol{\beta}_{t+1} = \boldsymbol{\beta}_t - \eta \nabla f(\boldsymbol{\beta}_t) = \boldsymbol{\beta}_t + 2\eta X^\top (\mathbf{y} - X\boldsymbol{\beta}_t),$$

i.e. componentwise $\beta_j \leftarrow \beta_j + 2\eta \sum_i (y_i - \hat{y}_i) x_{ij}$ with $x_{i0} = 1$. The factor 2 is often absorbed into the learning rate η. Each full-batch step uses all samples. The negative gradient is the direction of steepest descent: for a unit vector $\mathbf{v}$ the first-order change $\nabla f^\top \mathbf{v}$ is minimized, by Cauchy–Schwarz, at $\mathbf{v} = -\nabla f / \|\nabla f\|$.



Gradient descent on the level sets of a quadratic objective: each step moves against the gradient, so an elongated bowl produces a zigzag path.

4.2 Convergence on least squares

Write $J(\boldsymbol{\beta}) = \frac{1}{2}\|X\boldsymbol{\beta} - \mathbf{y}\|^2$ (so $\nabla J = X^\top(X\boldsymbol{\beta} - \mathbf{y})$) and $H = X^\top X$, assumed positive definite.

Theorem 1.6: exact convergence condition

Gradient descent $\boldsymbol{\beta}_{t+1} = \boldsymbol{\beta}_t - \eta \nabla J(\boldsymbol{\beta}_t)$ converges to $\hat{\boldsymbol{\beta}}$ for every initial point if and only if

$$0 < \eta < \frac{2}{\lambda_{\max}(X^\top X)}.$$

Proof. Error recursion. Since $\nabla J(\hat{\boldsymbol{\beta}}) = \mathbf{0}$, $X^\top \mathbf{y} = H\hat{\boldsymbol{\beta}}$, so $\nabla J(\boldsymbol{\beta}_t) = H(\boldsymbol{\beta}_t - \hat{\boldsymbol{\beta}})$. With $\mathbf{e}_t = \boldsymbol{\beta}_t - \hat{\boldsymbol{\beta}}$,

$$\mathbf{e}_{t+1} = \mathbf{e}_t - \eta H \mathbf{e}_t = (\mathbf{I} - \eta H)\mathbf{e}_t, \qquad \mathbf{e}_t = (\mathbf{I} - \eta H)^t \mathbf{e}_0.$$

Diagonalize. H is symmetric, so $H = Q\Lambda Q^\top$ with Q orthogonal and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$, $\lambda_i > 0$. Using $\mathbf{I} = QQ^\top$, $\mathbf{I} - \eta H = Q(\mathbf{I} - \eta\Lambda)Q^\top$. Let $\mathbf{c}_t = Q^\top \mathbf{e}_t$. Multiplying the recursion by $Q^\top$ and using $Q^\top Q = \mathbf{I}$ gives $\mathbf{c}_{t+1} = (\mathbf{I} - \eta\Lambda)\mathbf{c}_t$, a diagonal recursion:

$$c_{t,i} = (1 - \eta\lambda_i)^t \, c_{0,i}.$$

Because $\|\mathbf{c}_t\| = \|\mathbf{e}_t\|$, $\mathbf{e}_t \to \mathbf{0}$ iff every coordinate $c_{t,i} \to 0$.

Sufficiency. If $|1 - \eta\lambda_i| < 1$ for all i, every coordinate tends to zero for every $\mathbf{e}_0$.

Necessity. If $|1 - \eta\lambda_i| \geq 1$ for some i, start from $\mathbf{e}_0 = \mathbf{q}_i$ (the ith eigenvector). Then $\mathbf{e}_t = (1 - \eta\lambda_i)^t \mathbf{q}_i$ and $\|\mathbf{e}_t\| = |1 - \eta\lambda_i|^t \geq 1$, so this initial point does not converge.

Solve. $|1 - \eta\lambda_i| < 1 \iff 0 < \eta\lambda_i < 2 \iff 0 < \eta < 2/\lambda_i$. Requiring this for all i is equivalent to $0 < \eta < 2/\lambda_{\max}$. ∎

Remark 1.7: conditioning

The contraction factor of direction i is $|1-\eta\lambda_i|$. The rate $\max_i |1-\eta\lambda_i|$ is minimized by $\eta^\star = 2/(\lambda_{\max}+\lambda_{\min})$, giving rate $(\kappa-1)/(\kappa+1)$ with condition number $\kappa = \lambda_{\max}/\lambda_{\min}$. Elongated (ill-conditioned) level sets make gradient descent slow; this is revisited in Chapter 6. At $\eta = 2/\lambda_{\max}$ the steepest direction oscillates with factor -1 and never decays, so the inequality is strict. In Example 1.5, $\lambda(X^\top X) = 4 \pm \sqrt{10}$ and the condition is $0 < \eta < (4-\sqrt{10})/3 \approx 0.279$.

5 Polynomial Features and Generalization

5.1 Feature maps

For scalar input take $\boldsymbol{\phi}_M(x) = (1, x, x^2, \dots, x^M)^\top$ and $y(x) = \sum_{j=0}^{M} \beta_j x^j$. The model is nonlinear in x but linear in the coefficients, so all of the above applies with the (Vandermonde-type) design matrix whose rows are $\boldsymbol{\phi}_M(x_i)^\top$.

5.2 Training error versus generalization

Because a degree-M model contains every lower-degree model, the minimal training SSE cannot increase as M increases. Validation error can nevertheless increase: a high-degree fit follows noise and oscillates between data points (overfitting), while too low a degree cannot capture the trend (underfitting). Performance is measured by the root mean squared error

$$\mathrm{MSE} = \frac{1}{n}\mathrm{SSE}, \qquad E_{\mathrm{RMS}} = \sqrt{\mathrm{MSE}}.$$

Remark 1.8: roles of the data splits

Training data choose the coefficients; validation data choose the degree and the regularization strength; a held-out test set estimates final performance. In typical learning curves the gap between training and validation RMS grows with M for $M \geq 5$, and degree 9 with $n = 10$ overfits badly while $n = 100$ points support the same degree much better. "About ten points per parameter" is a heuristic, not a theorem. Regularization (Chapter 3) is the second remedy.

6 Proof Supplement: Uniqueness and the Pseudoinverse

Motivation. When $X^\top X$ is singular the normal equations have many solutions. The pseudoinverse picks the one of minimum norm, which is also the solution gradient descent converges to from zero, an early example of implicit regularization.

Theorem 1.9: set of minimizers

The minimizers of $\|\mathbf{y} - X\boldsymbol{\beta}\|^2$ are exactly the solutions of $X^\top X\boldsymbol{\beta} = X^\top \mathbf{y}$; this system is always consistent, and the solution set is $\hat{\boldsymbol{\beta}}_0 + \ker X$ for any particular solution $\hat{\boldsymbol{\beta}}_0$. Among them, $X^+\mathbf{y}$ has minimal Euclidean norm.

Proof. Consistency: $X^\top \mathbf{y}$ lies in the column space of $X^\top$, which equals the column space of $X^\top X$ (both have orthogonal complement $\ker X$, since $X^\top X\mathbf{v} = \mathbf{0} \Rightarrow \|X\mathbf{v}\|^2 = 0$). Two solutions differ by $\mathbf{v}$ with $X^\top X\mathbf{v} = \mathbf{0}$, i.e. $\mathbf{v} \in \ker X$, and give the same fitted values. Write $X = U\Sigma V^\top$ (thin

SVD with r positive singular values). Every solution is $V_r\Sigma_r^{-1}U_r^\top\mathbf{y}+\mathbf{v}$ with $\mathbf{v}\perp\operatorname{range}(V_r)$; by Pythagoras its norm is minimized at $\mathbf{v}=\mathbf{0}$, which is $X^+\mathbf{y}$. ■

7 Selected exercises

Motivation. Three derivations complete the chapter: the algebra of the hat matrix behind the SSE, the claim that gradient descent from zero finds the minimum-norm solution, and the optimal constant step size quoted in the conditioning remark.

Exercise 1.1 (Hat matrix and the residual sum of squares)

Let $X\in\mathbb{R}^{n\times d}$ have full column rank $d<n$ and put $H=X(X^\top X)^{-1}X^\top$.
(a) Show that H is symmetric and idempotent, $HX=X$ and $\operatorname{tr}H=d$; show that $\mathbf{I}-H$ is symmetric and idempotent with $\operatorname{tr}(\mathbf{I}-H)=n-d$.
(b) Show that the fitted values are $H\mathbf{y}$, the residual vector is $\mathbf{e}=(\mathbf{I}-H)\mathbf{y}$, $\mathrm{SSE}=\mathbf{y}^\top(\mathbf{I}-H)\mathbf{y}$, and $\|\mathbf{y}\|^2=\|H\mathbf{y}\|^2+\|\mathbf{e}\|^2$.
(c) Let $\mathbf{y}=X\boldsymbol{\beta}+\boldsymbol{\varepsilon}$ with $\mathbb{E}\boldsymbol{\varepsilon}=\mathbf{0}$ and $\operatorname{Cov}(\boldsymbol{\varepsilon})=\sigma^2\mathbf{I}$. Show that $\mathbb{E}[\mathrm{SSE}]=(n-d)\sigma^2$, so $\mathrm{SSE}/(n-d)$ is unbiased for σ^2.

Solution. [Supplement]

(a) $X^\top X$ is symmetric, hence so is its inverse, and $H^\top=X(X^\top X)^{-1}X^\top=H$. Next,

$$H^2=X(X^\top X)^{-1}(X^\top X)(X^\top X)^{-1}X^\top=H,\qquad HX=X(X^\top X)^{-1}X^\top X=X.$$

By cyclicity of the trace, $\operatorname{tr}H=\operatorname{tr}\left((X^\top X)^{-1}X^\top X\right)=\operatorname{tr}\mathbf{I}_d=d$. For $\mathbf{I}-H$: it is symmetric, $(\mathbf{I}-H)^2=\mathbf{I}-2H+H^2=\mathbf{I}-H$, and $\operatorname{tr}(\mathbf{I}-H)=n-d$.

(b) By the least squares theorem, $X\hat{\boldsymbol{\beta}}=X(X^\top X)^{-1}X^\top\mathbf{y}=H\mathbf{y}$, so $\mathbf{e}=\mathbf{y}-H\mathbf{y}=(\mathbf{I}-H)\mathbf{y}$. Using (a),

$$\mathrm{SSE}=\mathbf{e}^\top\mathbf{e}=\mathbf{y}^\top(\mathbf{I}-H)^\top(\mathbf{I}-H)\mathbf{y}=\mathbf{y}^\top(\mathbf{I}-H)\mathbf{y}.$$

Also $(H\mathbf{y})^\top\mathbf{e}=\mathbf{y}^\top H(\mathbf{I}-H)\mathbf{y}=\mathbf{y}^\top(H-H^2)\mathbf{y}=0$, so expanding $\|H\mathbf{y}+\mathbf{e}\|^2$ gives the Pythagorean identity.

(c) Since $(\mathbf{I}-H)X=X-HX=0$, we get $\mathbf{e}=(\mathbf{I}-H)(X\boldsymbol{\beta}+\boldsymbol{\varepsilon})=(\mathbf{I}-H)\boldsymbol{\varepsilon}$ and, as in (b), $\mathrm{SSE}=\boldsymbol{\varepsilon}^\top(\mathbf{I}-H)\boldsymbol{\varepsilon}$. For a fixed matrix A, a scalar equals its trace, so

$$\mathbb{E}[\boldsymbol{\varepsilon}^\top A\boldsymbol{\varepsilon}]=\mathbb{E}\operatorname{tr}(A\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top)=\operatorname{tr}\left(A\,\mathbb{E}[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top]\right)=\sigma^2\operatorname{tr}A.$$

With $A=\mathbf{I}-H$ this is $(n-d)\sigma^2$. The maximum likelihood estimate SSE/n of Chapter 3 therefore has mean $(n-d)\sigma^2/n<\sigma^2$: it is biased downward. ■

Exercise 1.2 (Gradient descent from zero finds $X^+\mathbf{y}$)

Let $X\neq 0$ have rank r and thin SVD $X=U_r\Sigma_rV_r^\top$ with singular values $s_1,\dots,s_r>0$; let the columns of V_0 complete those of V_r to an orthonormal basis of $\mathbb{R}^d$. Run gradient descent on $J(\boldsymbol{\beta})=\frac{1}{2}\|X\boldsymbol{\beta}-\mathbf{y}\|^2$ with $0<\eta<2/\lambda_{\max}(X^\top X)$; $X^\top X$ need not be invertible.
(a) Show that if $\boldsymbol{\beta}_0=\mathbf{0}$, every iterate lies in $\operatorname{range}(X^\top)=\operatorname{range}(V_r)$.
(b) Show that if $\boldsymbol{\beta}_0=\mathbf{0}$, then $\boldsymbol{\beta}_t\to X^+\mathbf{y}=V_r\Sigma_r^{-1}U_r^\top\mathbf{y}$.
(c) Show that for a general $\boldsymbol{\beta}_0$ the limit is $X^+\mathbf{y}+V_0V_0^\top\boldsymbol{\beta}_0$.

Solution. [Supplement]

(a) $\nabla J(\boldsymbol{\beta}) = X^\top(X\boldsymbol{\beta} - \mathbf{y}) \in \mathrm{range}(X^\top)$ for every $\boldsymbol{\beta}$. If $\boldsymbol{\beta}_t \in \mathrm{range}(X^\top)$, then $\boldsymbol{\beta}_{t+1} = \boldsymbol{\beta}_t - \eta\nabla J(\boldsymbol{\beta}_t)$ is a linear combination of two vectors of this subspace; induction from $\boldsymbol{\beta}_0 = \mathbf{0}$ proves the claim. Finally $X^\top = V_r\Sigma_r U_r^\top$ with $\Sigma_r U_r^\top$ of full row rank r, so $\mathrm{range}(X^\top) = \mathrm{range}(V_r)$.

(b) *Step 1 (coordinates).* By (a), $\boldsymbol{\beta}_t = V_r\mathbf{c}_t$ with $\mathbf{c}_t = V_r^\top\boldsymbol{\beta}_t \in \mathbb{R}^r$. We have $X^\top X = V_r\Sigma_r^2 V_r^\top$ and $X^\top\mathbf{y} = V_r\Sigma_r U_r^\top\mathbf{y}$. Multiplying the update by $V_r^\top$ and using $V_r^\top V_r = \mathbf{I}_r$,

$$\mathbf{c}_{t+1} = \mathbf{c}_t - \eta(\Sigma_r^2\mathbf{c}_t - \Sigma_r U_r^\top\mathbf{y}).$$

Step 2 (error recursion). Let $\mathbf{c}^\star = \Sigma_r^{-1}U_r^\top\mathbf{y}$, so $\Sigma_r^2\mathbf{c}^\star = \Sigma_r U_r^\top\mathbf{y}$. Subtracting, $\mathbf{c}_{t+1} - \mathbf{c}^\star = (\mathbf{I} - \eta\Sigma_r^2)(\mathbf{c}_t - \mathbf{c}^\star)$, a diagonal recursion with factors $1 - \eta s_i^2$.

Step 3 (contraction). The numbers s_i^2 are the nonzero eigenvalues of $X^\top X$, so $0 < \eta s_i^2 \le \eta\lambda_{\max} < 2$ and $|1 - \eta s_i^2| < 1$. Hence $\mathbf{c}_t \to \mathbf{c}^\star$ and $\boldsymbol{\beta}_t = V_r\mathbf{c}_t \to V_r\Sigma_r^{-1}U_r^\top\mathbf{y} = X^+\mathbf{y}$, the minimum-norm minimizer of the proof supplement.

(c) Write $\boldsymbol{\beta}_t = V_r\mathbf{c}_t + V_0\mathbf{d}_t$. Since $\nabla J(\boldsymbol{\beta}_t) \in \mathrm{range}(V_r)$ and $V_0^\top V_r = 0$, $\mathbf{d}_{t+1} = V_0^\top\boldsymbol{\beta}_{t+1} = V_0^\top\boldsymbol{\beta}_t = \mathbf{d}_t$, so $\mathbf{d}_t = V_0^\top\boldsymbol{\beta}_0$ for all t. Moreover $X^\top X V_0 = 0$, so $V_r^\top X^\top X\boldsymbol{\beta}_t = \Sigma_r^2\mathbf{c}_t$ and $\mathbf{c}_t$ obeys the recursion of Step 1 (from a different start). Steps 2–3 do not depend on the start, so $\mathbf{c}_t \to \mathbf{c}^\star$ and $\boldsymbol{\beta}_t \to X^+\mathbf{y} + V_0V_0^\top\boldsymbol{\beta}_0$. Gradient descent never changes the component in $\ker X$; started at zero, it selects the minimum-norm solution without any explicit penalty. ■

Exercise 1.3 (The optimal constant step size)

Let $H = X^\top X \succ 0$ have eigenvalues $0 < \lambda_{\min} = \lambda_d \le \cdots \le \lambda_1 = \lambda_{\max}$, $\kappa = \lambda_{\max}/\lambda_{\min}$, and $\rho(\eta) = \max_i |1 - \eta\lambda_i|$ for $\eta > 0$.
(a) Show that $\rho(\eta) = \max\{|1 - \eta\lambda_{\min}|, |1 - \eta\lambda_{\max}|\}$.
(b) Show that ρ is minimized at $\eta^\star = 2/(\lambda_{\max} + \lambda_{\min})$ and $\rho(\eta^\star) = (\kappa - 1)/(\kappa + 1)$.
(c) Deduce $\|\boldsymbol{\beta}_t - \hat{\boldsymbol{\beta}}\| \le \rho(\eta)^t\|\boldsymbol{\beta}_0 - \hat{\boldsymbol{\beta}}\|$, and compute $\eta^\star$ and $\rho(\eta^\star)$ for Example 1.5.

Solution. [Supplement]

(a) For fixed η, $\lambda \mapsto |1 - \eta\lambda|$ is the absolute value of an affine function, hence convex, so on $[\lambda_{\min}, \lambda_{\max}]$ its maximum is attained at an endpoint. All λ_i lie in this interval and both endpoints are eigenvalues, which gives the formula.

(b) *Step 1 (lower bound).* Since $|a| \ge a$ and $|a| \ge -a$, (a) gives $\rho(\eta) \ge m(\eta) := \max\{1 - \eta\lambda_{\min},\ \eta\lambda_{\max} - 1\}$. The first function decreases and the second increases in η; they are equal exactly at $\eta = \eta^\star$, so $m(\eta) \ge m(\eta^\star)$ for every $\eta > 0$, with

$$m(\eta^\star) = 1 - \frac{2\lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}.$$

Step 2 (attained). At $\eta^\star$, $1 - \eta^\star\lambda_{\min} = \frac{\kappa-1}{\kappa+1} \ge 0$ and $1 - \eta^\star\lambda_{\max} = -\frac{\kappa-1}{\kappa+1}$, so by (a) $\rho(\eta^\star) = \frac{\kappa-1}{\kappa+1} = m(\eta^\star) \le \rho(\eta)$ for all $\eta > 0$.

(c) In the proof of the convergence theorem, $\mathbf{c}_t = Q^\top\mathbf{e}_t$ satisfies $|c_{t,i}| = |1 - \eta\lambda_i|^t|c_{0,i}| \le \rho(\eta)^t|c_{0,i}|$. Squaring, summing over i and using $\|\mathbf{c}_t\| = \|\mathbf{e}_t\|$ gives the bound. In Example 1.5, $\lambda_{\max,\min} = 4 \pm \sqrt{10}$, so $\eta^\star = 2/8 = 1/4$ and

$$\rho(\eta^\star) = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{2\sqrt{10}}{8} = \frac{\sqrt{10}}{4} \approx 0.791.$$

The error shrinks by only about 21% per step, because $\kappa = (4 + \sqrt{10})/(4 - \sqrt{10}) \approx 8.6$. ■

Chapter 2. Probability, Likelihood and Information Theory

Random variables, maximum likelihood, MAP estimation, Bayesian models; Entropy, KL divergence, mutual information, cross-entropy

1 Events and Conditional Probability

A sample space Ω contains the possible outcomes and events are (measurable) subsets. The axioms are nonnegativity, $P(\Omega) = 1$ and countable additivity for disjoint events. They imply

$$P(\emptyset) = 0, \qquad P(E^c) = 1 - P(E), \qquad P(E \cup F) = P(E) + P(F) - P(E \cap F),$$

monotonicity $E \subseteq F \Rightarrow P(E) \leq P(F)$, and continuity along increasing or decreasing sequences of events.

Definition 2.1: conditional probability, Bayes' rule, independence

For $P(E) > 0$, $P(F \mid E) = P(F \cap E)/P(E)$. For a partition $E_1, E_2, \ldots$ of Ω,

$$P(F) = \sum_i P(F \mid E_i)P(E_i), \qquad P(E_i \mid F) = \frac{P(F \mid E_i)P(E_i)}{\sum_j P(F \mid E_j)P(E_j)}.$$

E, F are independent if $P(E \cap F) = P(E)P(F)$.

Example 2.2: why the prior matters

Let $P(D) = 0.01$, $P(+ \mid D) = 0.9$, $P(+ \mid D^c) = 0.1$. Then

$$P(D \mid +) = \frac{0.9(0.01)}{0.9(0.01) + 0.1(0.99)} \approx 0.0833.$$

A sensitive test does not make the posterior large when the prior is small. Two fair coin tosses: "first head" and "second head" each have probability 1/2 and their intersection 1/4, so they are independent.

2 Random Variables and Distributions

Motivation. Learning treats data as samples from an unknown distribution, and losses are expectations. We fix the notation for distributions, expectations and independence used in every later chapter.

A random variable is a measurable function $X : \Omega \to \mathbb{R}$ (e.g. the number of heads in three tosses, $X(HHT) = 2$). Its CDF is $F_X(t) = P(X \leq t)$.

- Discrete: probability mass $p(x) = P(X = x)$, $F_X(t) = \sum_{x \leq t} p(x)$.
- Continuous: density $f \geq 0$ with $\int f = 1$, $P(a < X \leq b) = \int_a^b f(u)\,du$. A density value is not a probability and may exceed one; $P(X = x) = 0$.

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \text{ or } \int g(x)f(x)\,dx, \qquad \mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2.$$

Theorem 2.3: tools used throughout the course

(i) Linearity: $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$ always.

(ii) Independence: $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$; hence $\operatorname{Var}(X+Y) = \operatorname{Var}X + \operatorname{Var}Y$. Uncorrelatedness ($\operatorname{Cov} = 0$) suffices for the variance identity.

(iii) Tower property: $\mathbb{E}[\mathbb{E}[Z \mid X]] = \mathbb{E}Z$.

(iv) $\mathbb{E}[(Z-c)^2] = \operatorname{Var}Z + (\mathbb{E}Z - c)^2 \geq \operatorname{Var}Z$ for every constant c.

Proof. (iv): write $Z - c = (Z - \mathbb{E}Z) + (\mathbb{E}Z - c)$, square, and use $\mathbb{E}[Z - \mathbb{E}Z] = 0$ to kill the cross term. The other items are standard. ■

3 Maximum Likelihood Estimation

Definition 2.4: likelihood and MLE

For iid data $x_1, \ldots, x_n$ from $p(x \mid \theta)$,

$$L(\theta) = \prod_{i=1}^{n} p(x_i \mid \theta), \qquad \hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} \sum_{i=1}^{n} \log p(x_i \mid \theta).$$

The data are fixed and the parameter varies; L is not a distribution over θ. The logarithm preserves the maximizer and turns products into sums.

Example 2.5: Bernoulli, including the boundary

$x_i \in \{0, 1\}$ with $P(x_i = 1) = \theta$; let $s = \sum x_i$, $f = n - s$. Then $\log L = s \log\theta + f \log(1-\theta)$. For $0 < s < n$,

$$\frac{d}{d\theta} \log L = \frac{s}{\theta} - \frac{f}{1-\theta} = 0 \iff \theta = \frac{s}{n}, \qquad \frac{d^2}{d\theta^2} \log L < 0.$$

If $s = 0$, $L = (1-\theta)^n$ is decreasing and the maximizer is 0; if $s = n$ it is 1. Both agree with s/n, but the derivative argument cannot be used there. Eight heads in ten tosses gives 0.8; two positive reviews out of two gives the extreme estimate 1.

Example 2.6: Gaussian

For $x_i \overset{\text{iid}}{\sim} N(\mu, \sigma^2)$, $\log L = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum(x_i - \mu)^2$, giving $\hat{\mu} = \bar{x}$ and $\hat{\sigma}^2 = \frac{1}{n}\sum(x_i - \bar{x})^2$ (biased; the unbiased version divides by $n - 1$).

4 Bayesian Inference and MAP

Motivation. Maximum likelihood can overfit with little data. Bayesian inference adds a prior and returns a posterior; its mode (MAP) is a single estimate that will turn out to be maximum likelihood plus a regularizer.

Definition 2.7: prior, posterior, evidence, MAP

With prior $p(\theta)$ and likelihood $p(D \mid \theta)$,

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}, \qquad p(D) = \int p(D \mid \theta)p(\theta)\, d\theta,$$

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_{\theta} \{\log p(D \mid \theta) + \log p(\theta)\}.$$

The evidence does not depend on θ. MAP is the posterior mode; the posterior mean is a different summary.

4.1 Beta–Bernoulli conjugacy

Theorem 2.8: Beta prior

If $\theta \sim \mathrm{Beta}(a, b)$, i.e. $p(\theta) = \theta^{a-1}(1-\theta)^{b-1}/B(a,b)$ with $B(a,b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$, and s successes, f failures are observed, then

$$\theta \mid D \sim \mathrm{Beta}(a+s,\, b+f), \qquad \mathbb{E}[\theta \mid D] = \frac{a+s}{a+b+n}, \qquad \hat{\theta}_{\mathrm{MAP}} = \frac{a+s-1}{a+b+n-2}$$

(the mode formula needs both posterior parameters above one).

Proof. $p(\theta \mid D) \propto \theta^s(1-\theta)^f \cdot \theta^{a-1}(1-\theta)^{b-1} = \theta^{(a+s)-1}(1-\theta)^{(b+f)-1}$, which is the $\mathrm{Beta}(a+s, b+f)$ kernel. For $\mathrm{Beta}(\alpha,\beta)$, $\mathbb{E}\theta = B(\alpha+1,\beta)/B(\alpha,\beta) = \alpha/(\alpha+\beta)$ using $\Gamma(z+1) = z\Gamma(z)$. The mode solves $\frac{\alpha-1}{\theta} = \frac{\beta-1}{1-\theta}$, i.e. $\theta = \frac{\alpha-1}{\alpha+\beta-2}$. ∎

The uniform prior is $\mathrm{Beta}(1,1)$; then the posterior mean $(s+1)/(n+2)$ is Laplace's rule of succession, and it equals the posterior predictive probability of a success, since $P(X_{\mathrm{new}} = 1 \mid D) = \int \theta\, p(\theta \mid D)\, d\theta$.

Example 2.9: the two sellers

Seller 1 has 90 positive and 10 negative reviews; seller 2 has 2 positive and 0 negative. With uniform priors

$$\theta_1 \mid D_1 \sim \mathrm{Beta}(91, 11), \qquad \theta_2 \mid D_2 \sim \mathrm{Beta}(3, 1).$$

Posterior predictive probabilities of a positive next review: $91/102 \approx 0.892$ versus $3/4 = 0.75$, so seller 1 is preferred, whereas the MLE (0.9 versus 1) prefers seller 2. A different question is which unknown rate is larger: since $\mathrm{Beta}(3,1)$ has CDF t^3,

$$P(\theta_1 > \theta_2 \mid D) = \mathbb{E}[\theta_1^3] = \frac{91 \cdot 92 \cdot 93}{102 \cdot 103 \cdot 104} \approx 0.713.$$

5 MAP as Regularization

Motivation. A Gaussian prior turns MAP into ℓ_2-penalized likelihood, so regularization has a probabilistic meaning.

5.1 Gaussian mean with a Gaussian prior

Let $X_1, \ldots, X_n \overset{\text{iid}}{\sim} N(\theta, 1)$ and prior $\theta \sim N(0, \tau^2)$.

Theorem 2.10: MAP is ridge, and it shrinks

$$\hat{\theta}_{\mathrm{MAP}} = \arg\min_{\theta} \sum_{i=1}^{n} (X_i - \theta)^2 + \lambda\theta^2, \quad \lambda = \frac{1}{\tau^2}, \qquad \hat{\theta}_{\mathrm{MAP}} = a\bar{X}, \quad a = \frac{n}{n+\lambda} = \frac{n\tau^2}{n\tau^2+1}.$$

Proof. $-\log p(\theta \mid X) = \frac{1}{2}\sum(X_i-\theta)^2 + \frac{\theta^2}{2\tau^2} + \text{const}$; multiplying by 2 gives the ridge objective. Setting the derivative $-2\sum(X_i-\theta) + 2\lambda\theta$ to zero gives $(n+\lambda)\theta = n\bar{X}$. ■

Remark 2.11: Laplace prior gives the lasso

With $p(\theta) \propto e^{-|\theta|/b}$ instead, $-\log$ posterior is $\frac{1}{2}\sum(X_i-\theta)^2 + \frac{1}{b}|\theta|$ and the MAP is the soft threshold $\operatorname{sign}(\bar{X})\max(|\bar{X}| - \frac{1}{nb}, 0)$ (Chapter 3).

6 Information and Entropy

Motivation. Classification losses are built from information theory. Entropy measures the average surprise of a distribution, and it is the baseline from which cross-entropy and KL divergence measure the cost of using the wrong distribution.

The information of an outcome is $h(x) = -\log p(x)$: rare outcomes carry more information, and for independent events probabilities multiply while information adds.

Definition 2.12: entropy

For a discrete random variable with pmf p,

$$H(X) = -\sum_{x} p(x)\log p(x) = \mathbb{E}[-\log p(X)], \qquad 0\log 0 := 0.$$

Natural logs give nats; base 2 gives bits.

Theorem 2.13: range of entropy

If X takes at most K values, $0 \le H(X) \le \log K$. $H = 0$ iff X is deterministic; $H = \log K$ iff X is uniform.

Proof. Each term $-p\log p \ge 0$ for $p \in [0,1]$, with equality iff $p \in \{0,1\}$. For the upper bound apply Jensen to the concave log: $H = \mathbb{E}\log\frac{1}{p(X)} \le \log\mathbb{E}\frac{1}{p(X)} = \log\sum_{x:p(x)>0} 1 \le \log K$, with equality iff $1/p(X)$ is constant on the support and the support has K points. ■

(For continuous densities the differential entropy can be negative; the statement above is for discrete variables.)

Definition 2.14: joint and conditional entropy

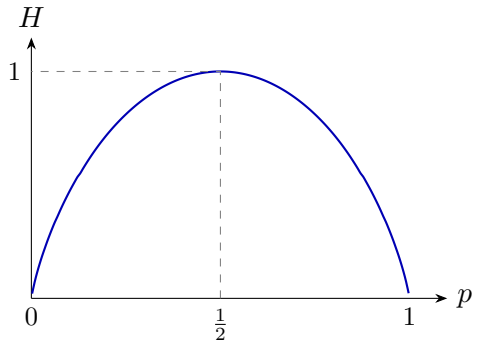
$$H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y),$$

$$H(Y \mid X) = -\sum_{x,y} p(x,y)\log p(y \mid x) = \sum_{x} p(x) H(Y \mid X = x).$$

Theorem 2.15: chain rule

$H(X,Y) = H(X) + H(Y \mid X)$.

Proof. Take $-\mathbb{E}$ of $\log p(X,Y) = \log p(X) + \log p(Y \mid X)$. ■



Entropy (in bits) of a Bernoulli(p) variable: zero for a certain outcome, maximal at $p = \frac{1}{2}$.

7 Kullback–Leibler Divergence

Motivation. How different are two distributions? KL divergence is the expected extra log-loss from coding with q instead of p. It is nonnegative and zero only at $q = p$, which is why minimizing it fits a model to data.

Definition 2.16: KL divergence

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_p\Big[\log \frac{p(X)}{q(X)}\Big].$$

Terms with $p(x) = 0$ contribute 0; if $p(x) > 0 = q(x)$ for some x the divergence is $+\infty$.

Theorem 2.17: Gibbs' inequality

$D_{\mathrm{KL}}(p \,\|\, q) \geq 0$, with equality iff $p = q$.

Proof. Sum over $S = \{x : p(x) > 0\}$. By Jensen for the concave log,

$$-D_{\mathrm{KL}}(p \,\|\, q) = \sum_{x \in S} p(x) \log \frac{q(x)}{p(x)} \leq \log \sum_{x \in S} p(x) \frac{q(x)}{p(x)} = \log \sum_{x \in S} q(x) \leq \log 1 = 0.$$

Equality forces (i) $q(x)/p(x) = c$ constant on S (Jensen) and (ii) $\sum_S q = 1$; then $c = \sum_S q = 1$, so $q = p$. Conversely $p = q$ gives $\log 1 = 0$ term by term. ■

Example 2.18: KL is not symmetric

For Bernoulli parameters, $D(r \,\|\, s) = r \log \frac{r}{s} + (1-r) \log \frac{1-r}{1-s}$. With $r = 0.9$, $s = 0.5$: $D(r\|s) \approx 0.368$ but $D(s\|r) \approx 0.511$ nats. KL is not a metric (no symmetry, no triangle inequality).

8 Mutual Information

Motivation. Mutual information is the KL divergence between a joint distribution and the product of its marginals: how much knowing X reduces uncertainty about Y. It is zero exactly under independence.

Definition 2.19: mutual information

$$I(X;Y) = D_{\mathrm{KL}}(p_{XY} \,\|\, p_X p_Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}.$$

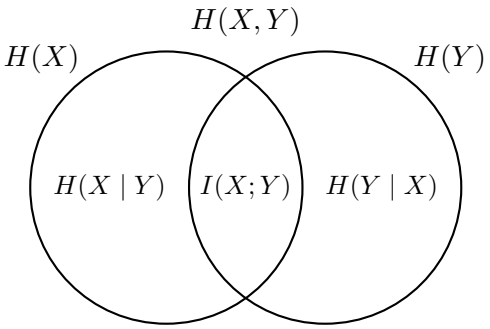
Theorem 2.20: identities

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y) \;\geq 0,$$

with $I = 0$ iff $X \perp Y$. Consequently $H(X \mid Y) \leq H(X)$ (conditioning reduces entropy).

Proof. $\log \frac{p(x,y)}{p(x)p(y)} = \log p(x \mid y) - \log p(x)$; take $\mathbb{E}$ and use $\sum_y p(x,y) = p(x)$ to get $-H(X \mid Y) + H(X)$. The last form follows from the chain rule. Nonnegativity and the equality case are Gibbs' inequality. ■

The entropy (Venn) diagram encodes $H(X,Y) = H(X \mid Y) + I(X;Y) + H(Y \mid X)$. For a fair bit X and $Y = X$, $I = 1$ bit; for independent fair bits, $I = 0$.



Entropy, joint and conditional entropy, and mutual information: the union of the two disks is $H(X,Y)$ and the overlap is $I(X;Y)$.

9 Cross-Entropy

Motivation. Cross-entropy is entropy plus KL divergence, so minimizing it over the model is the same as minimizing KL. This is why the standard classification loss is cross-entropy, and why it equals negative log-likelihood.

Definition 2.21: cross-entropy

$H(p,q) = -\sum_x p(x) \log q(x) = \mathbb{E}_p[-\log q(X)]$.

Theorem 2.22: cross-entropy = entropy + KL

$H(p,q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$. Hence for fixed p, $\arg\min_q H(p,q) = \arg\min_q D_{\mathrm{KL}}(p \,\|\, q) = p$.

Proof. $-\sum p \log q = -\sum p \log p + \sum p \log(p/q)$. The second statement uses Gibbs. ■

Remark 2.23: classification loss and soft labels

For a one-hot label $\mathbf{e}_y$, $H(\mathbf{e}_y) = 0$ and $H(\mathbf{e}_y, q) = D_{\mathrm{KL}}(\mathbf{e}_y \| q) = -\log q_y$: the classification loss. $H(p,q)$ is *linear in its first argument*, $H(\alpha p + \beta p', q) = \alpha H(p,q) + \beta H(p',q)$, because p enters only as coefficients. Therefore a soft label that averages n one-hot labels gives $H(\frac{1}{n}\sum_j \mathbf{e}_{y_j}, q) = \frac{1}{n}\sum_j (-\log q_{y_j})$: fitting the label distribution by KL is equivalent (up to the constant $H(\hat{p})$) to ordinary cross-entropy on every individual label.

Chapter 3. Probabilistic Regression, Regularization and Kernel Ridge

Linear regression from a probabilistic view, regularization, ridge regression, the kernel trick

1 Gaussian Likelihood Explains Squared Error

Motivation. Why squared error? If the noise is Gaussian, maximizing the likelihood is exactly minimizing the sum of squares. Each loss in the course corresponds in the same way to a noise model.

Theorem 3.1: MLE under Gaussian noise is least squares

If $y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \varepsilon_i$ with $\varepsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2)$, then

$$\log p(\mathbf{y} \mid X, \boldsymbol{\beta}, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2.$$

For fixed σ^2, maximizing the likelihood is minimizing the SSE, so $\hat{\boldsymbol{\beta}}_{\text{MLE}}$ solves the normal equations. Maximizing over σ^2 then gives $\hat{\sigma}^2 = \text{SSE}/n$.

Proof. The density of y_i is $(2\pi\sigma^2)^{-1/2}\exp\{-(y_i - \mathbf{x}_i^\top\boldsymbol{\beta})^2/(2\sigma^2)\}$; independence multiplies densities and the log turns the product into the displayed sum. Differentiating in σ^2: $-\frac{n}{2\sigma^2} + \frac{\text{SSE}}{2\sigma^4} = 0$. ■

2 Ridge Regression

Motivation. Least squares is unstable when $X^\top X$ is nearly singular or there are more features than samples. Adding $\lambda\|\mathbf{w}\|^2$ makes the problem well posed and shrinks the coefficients.

2.1 Definition and closed form

Definition 3.2: ridge

For $\lambda > 0$, $J(\boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$.

Theorem 3.3: ridge solution

$\nabla J = X^\top X\boldsymbol{\beta} - X^\top\mathbf{y} + \lambda\boldsymbol{\beta}$ and the unique minimizer is $\hat{\boldsymbol{\beta}}_\lambda = (X^\top X + \lambda\mathbf{I})^{-1}X^\top\mathbf{y}$, even when X is rank deficient.

Proof. $\mathbf{v}^\top(X^\top X + \lambda\mathbf{I})\mathbf{v} = \|X\mathbf{v}\|^2 + \lambda\|\mathbf{v}\|^2 > 0$ for $\mathbf{v} \neq \mathbf{0}$, so the matrix is positive definite, invertible, and J is strictly convex. ■

2.2 MAP interpretation

With prior $\boldsymbol{\beta} \sim N(\mathbf{0}, \tau^2\mathbf{I})$, $-\log p(\boldsymbol{\beta} \mid \mathbf{y}) = \frac{1}{2\sigma^2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{1}{2\tau^2}\|\boldsymbol{\beta}\|^2 + \text{const}$; multiplying by σ^2 gives ridge with $\lambda = \sigma^2/\tau^2$. A tight prior (small τ) means strong shrinkage.

3 Other Penalties and the Lasso

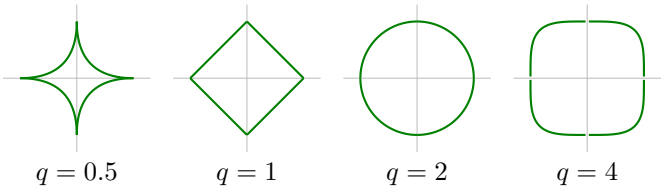
More generally one penalizes $\sum_j |\beta_j|^q$. $q = 2$ is ridge (round level sets, pure shrinkage); $q = 1$ is the lasso, whose diamond-shaped level sets have corners on the axes and produce exact zeros; $0 < q < 1$ is nonconvex.

Theorem 3.4: soft thresholding

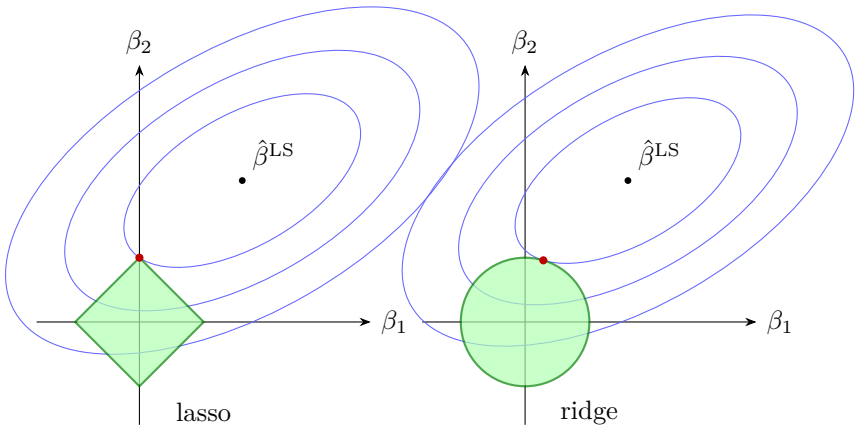
$\arg\min_b \ \frac{1}{2}(b-z)^2 + \lambda|b| = \operatorname{sign}(z)\max(|z|-\lambda, 0)$, whereas $\arg\min_b \frac{1}{2}(b-z)^2 + \frac{\lambda}{2}b^2 = z/(1+\lambda)$.

Proof. The objective is strictly convex; the optimality condition is $0 \in b - z + \lambda\,\partial|b|$, where $\partial|b| = \{\operatorname{sign} b\}$ for $b \neq 0$ and $[-1, 1]$ at 0. If $b > 0$: $b = z - \lambda$, valid iff $z > \lambda$. If $b < 0$: $b = z + \lambda$, valid iff $z < -\lambda$. If $b = 0$: need $z \in \lambda[-1, 1]$, i.e. $|z| \leq \lambda$. The ridge case is the stationarity condition $b - z + \lambda b = 0$. ■

Lasso moves every coefficient by a fixed amount and cuts small ones to zero (sparsity); ridge rescales proportionally and never produces exact zeros.



Unit balls $\sum_j |\beta_j|^q \leq 1$ of the penalties: $q = 1$ is the lasso, $q = 2$ ridge.



Constrained least squares: the smallest RSS contour meets the lasso diamond at a corner (a zero coefficient) and the ridge disk at a generic point.

4 Kernel Ridge Regression

Motivation. Nonlinear regression can be done by mapping inputs to a feature space and fitting ridge there. The solution depends on the data only through inner products, so a kernel lets us work with infinitely many features at the cost of an $n \times n$ linear system.

4.1 Feature-space and sample-space forms

Let $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^m$ be a feature map and, $\Phi = [\boldsymbol{\phi}(\mathbf{x}_1) \cdots \boldsymbol{\phi}(\mathbf{x}_n)] \in \mathbb{R}^{m\times n}$ (columns are samples). Ridge in feature space minimizes $J = \frac{1}{2}\|\mathbf{y} - \Phi^\top \mathbf{w}\|^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2$, so

$$\nabla J = -\Phi\mathbf{y} + (\lambda \mathbf{I}_m + \Phi\Phi^\top)\mathbf{w}, \qquad \hat{\mathbf{w}} = (\lambda \mathbf{I}_m + \Phi\Phi^\top)^{-1}\Phi\mathbf{y}.$$

Theorem 3.5: push-through identity

$(\lambda \mathbf{I}_m + \Phi\Phi^\top)^{-1}\Phi = \Phi(\lambda \mathbf{I}_n + \Phi^\top\Phi)^{-1}$. Hence with the Gram matrix $K = \Phi^\top\Phi$, $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^\top \boldsymbol{\phi}(\mathbf{x}_j)$,

$$\hat{\mathbf{w}} = \Phi\boldsymbol{\alpha}, \quad \boldsymbol{\alpha} = (K + \lambda \mathbf{I}_n)^{-1}\mathbf{y}, \qquad f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})^\top \hat{\mathbf{w}} = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x}) = \mathbf{k}_{\mathbf{x}}^\top (K + \lambda \mathbf{I}_n)^{-1}\mathbf{y}.$$

Proof. $\Phi(\lambda \mathbf{I}_n + \Phi^\top\Phi) = \lambda\Phi + \Phi\Phi^\top\Phi = (\lambda \mathbf{I}_m + \Phi\Phi^\top)\Phi$. Multiply on the left by $(\lambda \mathbf{I}_m + \Phi\Phi^\top)^{-1}$ and on the right by $(\lambda \mathbf{I}_n + \Phi^\top\Phi)^{-1}$ (both positive definite). The two identity matrices have different sizes. ■

Remark 3.6: the kernel trick and cost

$\boldsymbol{\phi}$ appears only through inner products, so only the kernel k is needed. The primal form solves an $m \times m$ system and the dual an $n \times n$ one: the kernel form wins when $m \gg n$ (e.g. the Gaussian kernel, whose feature space is infinite-dimensional). Merely mapping to higher dimensions is feature engineering; computing through k without forming $\boldsymbol{\phi}$ is the kernel trick. An alternative derivation substitutes $\mathbf{w} = \Phi\boldsymbol{\alpha}$ and obtains $K[(K + \lambda \mathbf{I})\boldsymbol{\alpha} - \mathbf{y}] = \mathbf{0}$; when K is singular one may not cancel K, but the displayed $\boldsymbol{\alpha}$ is a solution and all solutions give the same $\hat{\mathbf{w}}$.

4.2 Kernels

- Linear: $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$.
- Quadratic in 2D: $(\mathbf{x}^\top \mathbf{z})^2 = x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2$, so $\boldsymbol{\phi}(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2)$. In 1D, $(1 + xz)^2$ has $\boldsymbol{\phi}(x) = (1, \sqrt{2}x, x^2)$.
- Gaussian: $k(\mathbf{x}, \mathbf{z}) = \exp(-\|\mathbf{x} - \mathbf{z}\|^2/(2\sigma^2))$; σ is the bandwidth.
- Normalized: $\tilde{k}(\mathbf{x}, \mathbf{z}) = k(\mathbf{x}, \mathbf{z})/\sqrt{k(\mathbf{x}, \mathbf{x})k(\mathbf{z}, \mathbf{z})}$.

Theorem 3.7: Gram matrices are PSD

For any feature-map kernel and any $\mathbf{a} \in \mathbb{R}^n$, $\mathbf{a}^\top K\mathbf{a} = \|\sum_i a_i \boldsymbol{\phi}(\mathbf{x}_i)\|^2 \geq 0$.

Not every similarity function is a valid kernel; positive semidefiniteness of all Gram matrices is the requirement.

4.3 Worked examples

Example 3.8: linear kernel

$\mathbf{x}_1 = (1, 2)$, $\mathbf{x}_2 = (2, 0)$, $\mathbf{y} = (3, 1)$, $\lambda = 1$: $K = \begin{pmatrix} 5 & 2 \\ 2 & 4 \end{pmatrix}$, $(K + \mathbf{I})^{-1} = \frac{1}{26}\begin{pmatrix} 5 & -2 \\ -2 & 6 \end{pmatrix}$,

$\boldsymbol{\alpha} = (1/2, 0)$, $\hat{\mathbf{w}} = \frac{1}{2}\mathbf{x}_1 = (1/2, 1)$. At $\mathbf{x} = (1, 0)$: $\mathbf{k}_{\mathbf{x}} = (1, 2)$ and $f(\mathbf{x}) = 1/2 = \hat{\mathbf{w}}^\top \mathbf{x}$.

Example 3.9: quadratic kernel

$\mathbf{x}_1 = (1, 1)$, $\mathbf{x}_2 = (2, 0)$, $\mathbf{y} = (3, 2)$, $k = (\mathbf{x}^\top \mathbf{z})^2$, $\lambda = 1$. Then $K_{11} = 4$, $K_{12} = 4$, $K_{22} = 16$,

$$(K + \mathbf{I})^{-1} = \frac{1}{69}\begin{pmatrix} 17 & -4 \\ -4 & 5 \end{pmatrix}, \qquad \boldsymbol{\alpha} = \frac{1}{69}\begin{pmatrix} 43 \\ -2 \end{pmatrix}.$$

At $\mathbf{x}_t = (1, 0)$, $\mathbf{k}_{\mathbf{x}_t} = (1, 4)$ and $f(\mathbf{x}_t) = (43 - 8)/69 = 35/69 \approx 0.507$. Check in feature space: $\boldsymbol{\phi}(\mathbf{x}_1) = (1, 1, \sqrt{2})$, $\boldsymbol{\phi}(\mathbf{x}_2) = (4, 0, 0)$, $\hat{\mathbf{w}} = \Phi\boldsymbol{\alpha} = (35, 43, 43\sqrt{2})/69$ and $\boldsymbol{\phi}(\mathbf{x}_t) = (1, 0, 0)$ give the same $35/69$.

5 Selected exercises

Motivation. These exercises make three claims of the chapter precise: ridge is least squares on augmented data with a measurable "number of parameters", the lasso is soft thresholding when the design is orthonormal, and the familiar kernels are valid.

Exercise 3.1 (Ridge as augmented least squares; effective degrees of freedom)

Let $X \in \mathbb{R}^{n \times d}$, $X \neq 0$, $\lambda > 0$, and $J(\boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$.

(a) Let $\tilde{X} = \begin{pmatrix} X \\ \sqrt{\lambda}\,\mathbf{I}_d \end{pmatrix}$ and $\tilde{\mathbf{y}} = \begin{pmatrix} \mathbf{y} \\ \mathbf{0} \end{pmatrix}$. Show that $J(\boldsymbol{\beta}) = \frac{1}{2}\|\tilde{\mathbf{y}} - \tilde{X}\boldsymbol{\beta}\|^2$, that $\tilde{X}$ always has full column rank, and recover $\hat{\boldsymbol{\beta}}_\lambda$ from the normal equations of Chapter 1.

(b) The fitted values are $X\hat{\boldsymbol{\beta}}_\lambda = H_\lambda \mathbf{y}$ with $H_\lambda = X(X^\top X + \lambda\mathbf{I})^{-1}X^\top$. Show that $\mathrm{df}(\lambda) := \operatorname{tr} H_\lambda = \sum_{i=1}^{r} \frac{s_i^2}{s_i^2 + \lambda}$, where $s_1, \dots, s_r$ are the positive singular values of X.

(c) Show that df is strictly decreasing on $(0, \infty)$, tends to $r = \operatorname{rank} X$ as $\lambda \to 0^+$ and to 0 as $\lambda \to \infty$.

Solution. [Supplement]

(a) The squared norm of a stacked vector is the sum of the squared norms of its blocks, so $\|\tilde{\mathbf{y}} - \tilde{X}\boldsymbol{\beta}\|^2 = \|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \|\mathbf{0} - \sqrt{\lambda}\,\boldsymbol{\beta}\|^2 = 2J(\boldsymbol{\beta})$. If $\tilde{X}\mathbf{v} = \mathbf{0}$, its lower block gives $\sqrt{\lambda}\,\mathbf{v} = \mathbf{0}$, so $\mathbf{v} = \mathbf{0}$: $\tilde{X}$ has full column rank. Since $\tilde{X}^\top\tilde{X} = X^\top X + \lambda\mathbf{I}$ and $\tilde{X}^\top\tilde{\mathbf{y}} = X^\top\mathbf{y}$, the least squares theorem of Chapter 1 gives the unique minimizer $(X^\top X + \lambda\mathbf{I})^{-1}X^\top\mathbf{y} = \hat{\boldsymbol{\beta}}_\lambda$.

(b) Take a full SVD $X = USV^\top$ with $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{d \times d}$ orthogonal and $S \in \mathbb{R}^{n \times d}$ having $S_{ii} = s_i$ for $i \leq r$ and zeros elsewhere. Then $X^\top X + \lambda\mathbf{I} = V(S^\top S + \lambda\mathbf{I})V^\top$, whose inverse is $V(S^\top S + \lambda\mathbf{I})^{-1}V^\top$, and

$$H_\lambda = US(S^\top S + \lambda\mathbf{I}_d)^{-1}S^\top U^\top.$$

The middle matrix $S(S^\top S + \lambda\mathbf{I}_d)^{-1}S^\top$ is $n \times n$ diagonal with entries $s_i^2/(s_i^2 + \lambda)$ for $i \leq r$ and 0 otherwise. By cyclicity and $U^\top U = \mathbf{I}$, $\operatorname{tr} H_\lambda$ is the trace of this diagonal matrix, i.e. the sum of these entries.

(c) $r \geq 1$ because $X \neq 0$. Each term has derivative $-s_i^2/(s_i^2 + \lambda)^2 < 0$, so df is strictly decreasing. As $\lambda \to 0^+$ each term tends to 1, and as $\lambda \to \infty$ each tends to 0. For full column rank, $\mathrm{df}(0^+) = d = \operatorname{tr} H$ of Exercise 1.1: ridge interpolates continuously between d effective parameters and none. ■

Exercise 3.2 (Orthonormal design: soft, hard and proportional shrinkage)

Assume $X^\top X = \mathbf{I}_d$, let $\lambda > 0$ and $\mathbf{z} = X^\top \mathbf{y}$ (the least squares estimate).
(a) Show that $\frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = \frac{1}{2}\|\mathbf{z} - \boldsymbol{\beta}\|^2 + c$ with c independent of $\boldsymbol{\beta}$.
(b) Deduce that the lasso $\arg\min_{\boldsymbol{\beta}} \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_1$ is $\hat{\beta}_j = \operatorname{sign}(z_j)\max(|z_j| - \lambda, 0)$, and that ridge with penalty $\frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$ gives $\mathbf{z}/(1+\lambda)$.
(c) Show that best-subset selection $\arg\min_{\boldsymbol{\beta}} \frac{1}{2}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \frac{\lambda^2}{2}\|\boldsymbol{\beta}\|_0$, where $\|\boldsymbol{\beta}\|_0$ is the number of nonzero entries, is hard thresholding: $\hat{\beta}_j = z_j$ if $|z_j| > \lambda$ and $\hat{\beta}_j = 0$ if $|z_j| < \lambda$.

Solution. [Supplement]

(a) Using $X^\top X = \mathbf{I}$ and $X^\top \mathbf{y} = \mathbf{z}$,

$$\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = \mathbf{y}^\top\mathbf{y} - 2\boldsymbol{\beta}^\top\mathbf{z} + \boldsymbol{\beta}^\top\boldsymbol{\beta} = \|\mathbf{z} - \boldsymbol{\beta}\|^2 + (\mathbf{y}^\top\mathbf{y} - \mathbf{z}^\top\mathbf{z}).$$

(b) By (a) the lasso objective is $c + \sum_j \left[\frac{1}{2}(z_j - \beta_j)^2 + \lambda|\beta_j|\right]$, a sum of functions of distinct coordinates, so it is minimized by minimizing each term separately. The soft-thresholding theorem gives $\hat{\beta}_j$; the same theorem gives the ridge coordinate $z_j/(1+\lambda)$.

(c) The objective is again $c + \sum_j g_j(\beta_j)$ with $g_j(b) = \frac{1}{2}(z_j - b)^2 + \frac{\lambda^2}{2}\mathbf{1}\{b \neq 0\}$. At $b = 0$, $g_j = z_j^2/2$. Over $b \neq 0$, $g_j(b) \geq \lambda^2/2$, with equality at $b = z_j$ when $z_j \neq 0$. If $|z_j| > \lambda$, then $z_j \neq 0$ and $\lambda^2/2 < z_j^2/2$, so the unique minimizer is $b = z_j$. If $|z_j| < \lambda$, every $b \neq 0$ gives $g_j(b) \geq \lambda^2/2 > z_j^2/2$, so $b = 0$. (At $|z_j| = \lambda$ both values are optimal.) Hard thresholding keeps large coefficients unshrunk, the lasso shifts all survivors by λ, and ridge rescales everything and zeroes nothing. ■

Exercise 3.3 (Building valid kernels)

Call $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *valid* if every Gram matrix $K = [k(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1}^n$ is symmetric positive semidefinite.
(a) Show that if k_1, k_2 are valid and $c \geq 0$, then $k_1 + k_2$ and ck_1 are valid, and so is $k(\mathbf{x}, \mathbf{z}) = g(\mathbf{x})g(\mathbf{z})$ for any $g : \mathcal{X} \to \mathbb{R}$.
(b) Show that if $A, B \in \mathbb{R}^{n\times n}$ are symmetric PSD, so is the entrywise product $A \circ B$; deduce that $k_1 k_2$ is valid.
(c) Deduce that $(1 + \mathbf{x}^\top\mathbf{z})^p$ ($p \in \mathbb{N}$) and the Gaussian kernel $\exp(-\|\mathbf{x} - \mathbf{z}\|^2/(2\sigma^2))$ are valid on $\mathbb{R}^m$.

Solution. [Supplement]

(a) Symmetry is clear in each case. For $\mathbf{a} \in \mathbb{R}^n$: $\mathbf{a}^\top(K_1 + K_2)\mathbf{a} = \mathbf{a}^\top K_1\mathbf{a} + \mathbf{a}^\top K_2\mathbf{a} \geq 0$ and $\mathbf{a}^\top(cK_1)\mathbf{a} = c\,\mathbf{a}^\top K_1\mathbf{a} \geq 0$. For $k = g(\mathbf{x})g(\mathbf{z})$, $\mathbf{a}^\top K\mathbf{a} = \sum_{i,j} a_i a_j g(\mathbf{x}_i)g(\mathbf{x}_j) = \left(\sum_i a_i g(\mathbf{x}_i)\right)^2 \geq 0$.

(b) By the spectral theorem $B = \sum_k \mu_k \mathbf{q}_k\mathbf{q}_k^\top$ with $\mu_k \geq 0$; put $\mathbf{b}_k = \sqrt{\mu_k}\,\mathbf{q}_k$, so $B_{ij} = \sum_k b_{k,i}b_{k,j}$. Then

$$\mathbf{a}^\top(A \circ B)\mathbf{a} = \sum_k \sum_{i,j} (a_i b_{k,i}) A_{ij} (a_j b_{k,j}) = \sum_k (\mathbf{a} \circ \mathbf{b}_k)^\top A (\mathbf{a} \circ \mathbf{b}_k) \geq 0.$$

$A \circ B$ is symmetric, and the Gram matrix of $k_1 k_2$ is $K_1 \circ K_2$.

(c) *Step 1 (polynomial kernel).* The linear kernel is valid ($\mathbf{a}^\top K\mathbf{a} = \|\sum_i a_i\mathbf{x}_i\|^2$) and the constant 1 is valid by (a) with $g \equiv 1$. Hence $1 + \mathbf{x}^\top\mathbf{z}$ is valid by (a), and its pth power by (b) and induction on p.

Step 2 (exponential kernel). Let $e(\mathbf{x}, \mathbf{z}) = \exp(\mathbf{x}^\top\mathbf{z}/\sigma^2)$ and $e_N = \sum_{q=0}^N (\mathbf{x}^\top\mathbf{z}/\sigma^2)^q/q!$. By (a), (b) and Step 1, each e_N is valid. For fixed points, the Gram matrices E_N converge entrywise to E, so $\mathbf{a}^\top E\mathbf{a} = \lim_N \mathbf{a}^\top E_N\mathbf{a} \geq 0$.

Step 3 (Gaussian kernel). Expanding $\|\mathbf{x}-\mathbf{z}\|^2 = \|\mathbf{x}\|^2 - 2\mathbf{x}^\top\mathbf{z} + \|\mathbf{z}\|^2$ gives

$$\exp\Big(-\frac{\|\mathbf{x}-\mathbf{z}\|^2}{2\sigma^2}\Big) = g(\mathbf{x})\, e(\mathbf{x},\mathbf{z})\, g(\mathbf{z}), \qquad g(\mathbf{x}) = \exp\Big(-\frac{\|\mathbf{x}\|^2}{2\sigma^2}\Big),$$

which is valid by (b) applied to e and the kernel $g(\mathbf{x})g(\mathbf{z})$ of (a). ■

Chapter 4. Classification: Logistic Regression, Softmax and SVM

Logistic regression, multiclass classification, support vector machines

1 Odds, Logit and the Logistic Model

Motivation. For a binary label we need a model of a probability, a number in $[0,1]$, as a function of features. Modelling the log-odds as linear gives the logistic model, whose output is the sigmoid of a linear score.

For a probability p, the odds are $o = p/(1-p)$ ($p = 0.8$ gives odds 4) and

$$\operatorname{logit}(p) = \log \frac{p}{1-p} \in \mathbb{R}, \qquad \sigma(z) = \frac{1}{1+e^{-z}} = \operatorname{logit}^{-1}(z).$$

Definition 4.1: logistic regression

With augmented input $\mathbf{x} = (1, x_1, \dots, x_d)$ and $y \in \{0,1\}$,

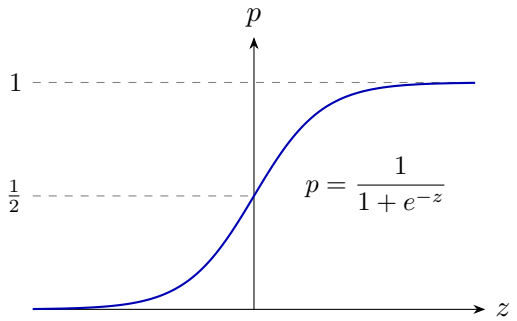
$$\log \frac{P(y=1 \mid \mathbf{x})}{P(y=0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}, \qquad P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}).$$

The score is linear and the probability transform nonlinear. Thresholding the probability at τ gives the hyperplane $\mathbf{w}^\top \mathbf{x} = \log \frac{\tau}{1-\tau}$ (at $\tau = 0.5$: $\mathbf{w}^\top \mathbf{x} = 0$).

Theorem 4.2: sigmoid identities

$\sigma'(z) = \sigma(z)(1-\sigma(z)) \le \frac{1}{4}$, $1 - \sigma(z) = \sigma(-z)$, $\frac{d}{dz}\log(1+e^z) = \sigma(z)$.

Proof. $\sigma'(z) = e^{-z}/(1+e^{-z})^2 = \sigma(z)\cdot\frac{e^{-z}}{1+e^{-z}}$ and $\frac{e^{-z}}{1+e^{-z}} = \frac{1}{e^z+1} = \sigma(-z) = 1-\sigma(z)$. The maximum of $s(1-s)$ on $[0,1]$ is $1/4$ at $s = 1/2$. ■



The logistic function, inverse of the logit $z = \log \frac{p}{1-p}$: log-odds z are mapped to probabilities p.

2 Likelihood, Gradient, Hessian and Convexity

Motivation. Logistic regression has no closed form, so we must optimize numerically. The Hessian is positive semidefinite, so the loss is convex: every local minimum is global and gradient methods are reliable.

For data $(\mathbf{x}_i, y_i)$ with $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$, $p(y_i \mid \mathbf{x}_i, \mathbf{w}) = p_i^{y_i}(1-p_i)^{1-y_i}$ and

$$J(\mathbf{w}) = -\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right] = \sum_i \left[\log(1+e^{z_i}) - y_i z_i\right], \quad z_i = \mathbf{w}^\top \mathbf{x}_i.$$

Theorem 4.3: logistic loss is convex

$\nabla J = \sum_i (p_i - y_i)\mathbf{x}_i = X^\top(\mathbf{p}-\mathbf{y})$ and $\nabla^2 J = \sum_i p_i(1-p_i)\mathbf{x}_i\mathbf{x}_i^\top = X^\top S X$, $S = \mathrm{diag}(p_i(1-p_i))$. Hence J is convex, and strictly convex if X has full column rank.

Proof. $\partial J/\partial z_i = \sigma(z_i) - y_i = p_i - y_i$ by the identities above, and $\partial z_i/\partial \mathbf{w} = \mathbf{x}_i$. Differentiating again, $\nabla p_i = p_i(1-p_i)\mathbf{x}_i$. For any $\mathbf{v}$, $\mathbf{v}^\top X^\top S X \mathbf{v} = \sum_i p_i(1-p_i)(\mathbf{x}_i^\top \mathbf{v})^2 \geq 0$; since $p_i(1-p_i) > 0$, the sum is positive whenever $X\mathbf{v} \neq \mathbf{0}$. ■

Example 4.4: one gradient step

$\mathbf{x} = (1,2)$, $y = 1$, $\mathbf{w} = \mathbf{0}$: $p = 1/2$, $\nabla J = (p-1)\mathbf{x} = (-1/2, -1)$. With $\eta = 0.1$, $\mathbf{w} = (0.05, 0.1)$, score 0.25, $p \approx 0.562$: the probability of the correct label increased.

3 Labels in $\{\pm 1\}$, Logistic Loss and BCE

Motivation. The same model can be written with labels $\{0,1\}$ (binary cross-entropy) or $\{\pm 1\}$ (logistic loss of the margin yf). The margin form connects logistic regression with the SVM and the other surrogate losses of this section.

With $Y \in \{\pm 1\}$ and score $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} + b$, $P(Y = +1 \mid \mathbf{x}) = \sigma(f)$ and $P(Y = -1 \mid \mathbf{x}) = \sigma(-f)$, so $-\log P(Y \mid \mathbf{x}) = \ell(Yf)$ with the **logistic loss** $\ell(z) = \log(1 + e^{-z})$. $Yf(\mathbf{x})$ is the *margin*: positive iff the sign of the score is correct.

Theorem 4.5: three names for one objective

Let $P(-1) = (1,0)$, $P(+1) = (0,1)$, $\mu(r) = (\sigma(-r), \sigma(r))$, $\tilde{Y} = (1+Y)/2 \in \{0,1\}$ and $\mathrm{BCE}(s,t) = -[t\log s + (1-t)\log(1-s)]$. For every $(\mathbf{a}, b)$ and every i,

$$D_{\mathrm{KL}}(P(Y_i) \,\|\, \mu(f(\mathbf{x}_i))) = \mathrm{BCE}\left(\sigma(f(\mathbf{x}_i)), \tilde{Y}_i\right) = \ell(Y_i f(\mathbf{x}_i)).$$

So KL minimization, BCE minimization and logistic regression are the same problem.

Proof. For a one-hot first argument, KL $= -\log$(model probability of the true class). If $Y = +1$ this is $-\log\sigma(r) = \log(1+e^{-r}) = \ell(r)$, and BCE with $t = 1$ is $-\log s$. If $Y = -1$ it is $-\log\sigma(-r) = \ell(-r)$, and BCE with $t = 0$ is $-\log(1-s) = -\log\sigma(-r)$. ■

Theorem 4.6: ℓ is convex

$\ell'(z) = \sigma(z) - 1$ and $\ell''(z) = \sigma(z)\sigma(-z) > 0$.

3.1 Surrogate risk bounds

For $(X,Y) \sim P$ and classifier $f = \mathrm{sign}(g)$, counting $g = 0$ as an error, the event $\{f(X) \neq Y\}$ implies $Yg(X) \leq 0$. For a nonnegative, nonincreasing loss ϕ this gives $\phi(Yg) \geq \phi(0)$ on errors and therefore

$$P(f(X) \neq Y) \leq \frac{\mathbb{E}[\phi(Yg(X))]}{\phi(0)}.$$

Logistic: $\phi(0) = \log 2$, so $R_{\log} < \varepsilon \Rightarrow P(f(X) = Y) > 1 - \varepsilon/\log 2$. Hinge $\max(0, 1-u)$ and exponential e^{-u} have $\phi(0) = 1$. Minimizing a convex surrogate thus controls the 0–1 error, which itself cannot be optimized by gradients.

4 Multiclass Classification and Softmax

Motivation. With K classes the sigmoid generalizes to the softmax, which turns K scores into a probability vector. Its cross-entropy loss has the same clean gradient (prediction minus target) as in the binary case.

For Q classes, $W \in \mathbb{R}^{Q\times d}$, scores $s_k = \mathbf{w}_k^\top \mathbf{x}$ and

$$p_k(\mathbf{x}) = \operatorname{softmax}(\mathbf{s})_k = \frac{e^{s_k}}{\sum_r e^{s_r}}, \qquad J(W) = -\sum_{i,k} y_{ik} \log p_k(\mathbf{x}_i).$$

Scores $(2, 1, 0)$ give approximately $(0.665, 0.245, 0.090)$.

Theorem 4.7: softmax calculus

With $\operatorname{LSE}(\mathbf{s}) = \log \sum_r e^{s_r}$ and $\mathbf{p} = \operatorname{softmax}(\mathbf{s})$:

(i) $\operatorname{softmax}(\mathbf{s} + c\mathbf{1}) = \operatorname{softmax}(\mathbf{s})$, $\operatorname{LSE}(\mathbf{s} + c\mathbf{1}) = \operatorname{LSE}(\mathbf{s}) + c$; stable evaluation $\operatorname{LSE}(\mathbf{s}) = M + \log \sum e^{s_r - M}$, $M = \max s_r$.

(ii) $\nabla \operatorname{LSE} = \mathbf{p}$; Jacobian $\partial p_k / \partial s_m = p_k(\delta_{km} - p_m)$, i.e. $\nabla^2 \operatorname{LSE} = \operatorname{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top \succeq 0$.

(iii) $\nabla_{\mathbf{s}}(-\log p_y) = \mathbf{p} - \mathbf{e}_y$; for the whole data set $\nabla_W J = (P - Y)^\top X$ (rows: classes).

(iv) $\log \operatorname{softmax}(\mathbf{s}) = \mathbf{s} - \operatorname{LSE}(\mathbf{s})\mathbf{1}$ and log-softmax is idempotent: $\log \operatorname{softmax}(\log \operatorname{softmax}(\mathbf{s})) = \log \operatorname{softmax}(\mathbf{s})$.

(v) $\max_k s_k \le \operatorname{LSE}(\mathbf{s}) \le \max_k s_k + \log Q$.

Proof. (i) e^c cancels.

(ii) Quotient rule; $\mathbf{v}^\top(\operatorname{diag}\mathbf{p} - \mathbf{p}\mathbf{p}^\top)\mathbf{v} = \sum p_k v_k^2 - (\sum p_k v_k)^2$ is the variance of v_K with $K \sim \mathbf{p}$.

(iii) $-\log p_y = -s_y + \operatorname{LSE}(\mathbf{s})$.

(iv) With $\boldsymbol{\ell} = \log \operatorname{softmax}(\mathbf{s})$, $e^{\ell_j} = p_j$ so $\operatorname{LSE}(\boldsymbol{\ell}) = \log \sum p_j = 0$.

(v) $e^M \le \sum e^{s_r} \le Q e^M$. ∎

Adding the same vector to all rows of W leaves the softmax unchanged, so W is identifiable only up to such shifts.

5 Support Vector Machines

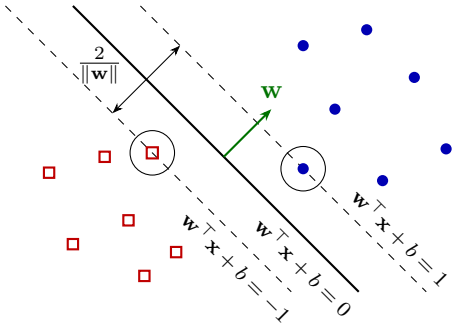
5.1 Geometry and the primal problem

Labels are $y_i \in \{\pm 1\}$, score $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. The distance from $\mathbf{x}_0$ to $\{\mathbf{w}^\top \mathbf{x} + b = 0\}$ is $|\mathbf{w}^\top \mathbf{x}_0 + b|/\|\mathbf{w}\|$ (move along the unit normal: $\mathbf{x}_0 - t\mathbf{w}/\|\mathbf{w}\|$ lies on the plane iff $t = (\mathbf{w}^\top \mathbf{x}_0 + b)/\|\mathbf{w}\|$). Rescaling $(\mathbf{w}, b)$ by $c > 0$ changes nothing, so for separable data normalize the closest functional margin to 1:

Definition 4.8: hard-margin SVM

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \ i = 1, \dots, n.$$

The support planes $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ lie at distance $1/\|\mathbf{w}\|$ (the margin) from the boundary; the slab width is $2/\|\mathbf{w}\|$.



Linear SVM: the margin between the hyperplanes $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ is $2/\|\mathbf{w}\|$; the circled points are support vectors.

5.2 Lagrangian, max–min inequality, weak duality

Theorem 4.9: max–min inequality

For any $F : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, $\sup_x \inf_y F(x, y) \leq \inf_y \sup_x F(x, y)$. Equality can fail: for $F = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (rows x, columns y) the left side is 0 and the right side 1.

Proof. For all x, y: $\inf_{y'} F(x, y') \leq F(x, y) \leq \sup_{x'} F(x', y)$. The left end does not depend on y and the right end not on x; take $\sup_x$ on the left, then $\inf_y$ on the right. ■

With $c_i = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ and $\alpha_i \geq 0$, $L(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 + \sum_i \alpha_i c_i$. Since $\sup_{\alpha \geq 0} \alpha c = 0$ if $c \leq 0$ and $+\infty$ if $c > 0$, $\sup_{\boldsymbol{\alpha} \geq \mathbf{0}} L$ equals the objective on feasible points and $+\infty$ elsewhere; hence $p^\star = \inf_{\mathbf{w},b} \sup_{\boldsymbol{\alpha}} L$. The dual is $d^\star = \sup_{\boldsymbol{\alpha}} \inf_{\mathbf{w},b} L$ and the max–min inequality gives **weak duality** $d^\star \leq p^\star$: every dual value is a lower bound on the primal optimum. For the SVM (convex, strictly feasible) strong duality $d^\star = p^\star$ holds.

5.3 The dual problem

Theorem 4.10: SVM dual

$\nabla_{\mathbf{w}} L = \mathbf{0} \Rightarrow \mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$; finiteness of $\inf_b L$ requires $\sum_i \alpha_i y_i = 0$. The dual is

$$\max_{\boldsymbol{\alpha} \geq \mathbf{0}} \ \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0.$$

Complementary slackness $\alpha_i[1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)] = 0$: $\alpha_i > 0$ only for support vectors, and $b = y_i - \mathbf{w}^\top \mathbf{x}_i$ for any such i.

Proof. b enters L linearly with coefficient $-\sum \alpha_i y_i$; if nonzero, $\inf_b = -\infty$. Substituting $\mathbf{w}$: $\tfrac{1}{2}\|\mathbf{w}\|^2 - \|\mathbf{w}\|^2 + \sum \alpha_i = \sum \alpha_i - \tfrac{1}{2}\|\mathbf{w}\|^2$. ■

Example 4.11: two points

$\mathbf{x}_1 = (0,0)$, $y_1 = -1$; $\mathbf{x}_2 = (2,2)$, $y_2 = +1$. The constraint forces $\alpha_1 = \alpha_2 = a$ and $\|\mathbf{x}_2\|^2 = 8$, so the dual is $2a - 4a^2$, maximized at $a = 1/4$. Then $\mathbf{w} = \frac{1}{4}(2,2) = (1/2, 1/2)$, $b = -1$ (from $\mathbf{x}_1$), the boundary is $x^{(1)} + x^{(2)} = 2$, $\|\mathbf{w}\| = 1/\sqrt{2}$, margin $\sqrt{2}$ (slab width $2\sqrt{2}$). Primal value $\frac{1}{2}\|\mathbf{w}\|^2 = 1/4$ equals the dual value $2a - 4a^2 = 1/4$.

Remark 4.12: soft margin and kernels

For non-separable data add slacks: $\min \frac{1}{2}\|\mathbf{w}\|^2 + C\sum \xi_i$ s.t. $y_i f(\mathbf{x}_i) \geq 1 - \xi_i$, $\xi_i \geq 0$, equivalently hinge loss $C\sum \max(0, 1 - y_i f(\mathbf{x}_i))$ plus ℓ_2. Its dual is identical except for the box constraint $0 \leq \alpha_i \leq C$. Replacing $\mathbf{x}_i^\top \mathbf{x}_j$ by a kernel gives the kernel SVM with score $\sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$. Logistic regression fits a likelihood; the SVM maximizes a geometric margin.

6 Selected exercises

Motivation. The exercises show how two-class softmax collapses to the logistic model, identify what the logistic loss estimates in the population, and read off the SVM margin from the dual variables.

Exercise 4.1 (Two-class softmax is logistic regression)

Let $Q = 2$, scores $s_k = \mathbf{w}_k^\top \mathbf{x}$, labels $y_i = y_{i1} \in \{0, 1\}$, $y_{i2} = 1 - y_i$, and $J(\mathbf{w}_1, \mathbf{w}_2)$ the softmax cross-entropy.
(a) Show that $p_1(\mathbf{x}) = \sigma((\mathbf{w}_1 - \mathbf{w}_2)^\top \mathbf{x})$ and $J(\mathbf{w}_1, \mathbf{w}_2) = J_{\log}(\mathbf{w}_1 - \mathbf{w}_2)$, where $J_{\log}$ is the logistic loss of Section 2.
(b) Show that $\nabla_{\mathbf{w}_1} J = -\nabla_{\mathbf{w}_2} J$ and that the Hessian of J in $(\mathbf{w}_1, \mathbf{w}_2)$ is singular.
(c) For $\lambda > 0$ let $(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_2)$ minimize $J + \frac{\lambda}{2}(\|\mathbf{w}_1\|^2 + \|\mathbf{w}_2\|^2)$. Show that $\hat{\mathbf{w}}_1 = -\hat{\mathbf{w}}_2$ and that $\hat{\mathbf{w}} = \hat{\mathbf{w}}_1 - \hat{\mathbf{w}}_2$ minimizes $J_{\log}(\mathbf{w}) + \frac{\lambda}{4}\|\mathbf{w}\|^2$.

Solution. [Supplement]

(a) Dividing numerator and denominator by e^{s_1},

$$p_1 = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} = \frac{1}{1 + e^{-(s_1 - s_2)}} = \sigma((\mathbf{w}_1 - \mathbf{w}_2)^\top \mathbf{x}), \qquad p_2 = 1 - p_1.$$

Hence $J = -\sum_i [y_i \log p_1(\mathbf{x}_i) + (1 - y_i)\log(1 - p_1(\mathbf{x}_i))]$, the logistic loss at $\mathbf{w} = \mathbf{w}_1 - \mathbf{w}_2$.

(b) By (a) and the chain rule, $\nabla_{\mathbf{w}_1} J = \nabla J_{\log}(\mathbf{w}_1 - \mathbf{w}_2)$ and $\nabla_{\mathbf{w}_2} J = -\nabla J_{\log}(\mathbf{w}_1 - \mathbf{w}_2)$. For any $\mathbf{v} \neq \mathbf{0}$, $t \mapsto J(\mathbf{w}_1 + t\mathbf{v}, \mathbf{w}_2 + t\mathbf{v})$ is constant, so with $\mathbf{u} = (\mathbf{v}, \mathbf{v})$ the Hessian $\mathcal{H}$ satisfies $\mathbf{u}^\top \mathcal{H} \mathbf{u} = 0$. J is convex (a convex function of a linear map), so $\mathcal{H} \succeq 0$, and then $\|\mathcal{H}^{1/2}\mathbf{u}\|^2 = 0$ gives $\mathcal{H}\mathbf{u} = \mathbf{0}$ with $\mathbf{u} \neq \mathbf{0}$.

(c) The penalized objective is strictly convex (convex plus strongly convex) and coercive, so it has a unique minimizer, characterized by

$$\nabla_{\mathbf{w}_1} J + \lambda \hat{\mathbf{w}}_1 = \mathbf{0}, \qquad \nabla_{\mathbf{w}_2} J + \lambda \hat{\mathbf{w}}_2 = \mathbf{0}.$$

Adding and using (b) gives $\lambda(\hat{\mathbf{w}}_1 + \hat{\mathbf{w}}_2) = \mathbf{0}$, so $\hat{\mathbf{w}}_1 = -\hat{\mathbf{w}}_2 = \hat{\mathbf{w}}/2$. Restricted to pairs $(\mathbf{w}/2, -\mathbf{w}/2)$ the objective equals $J_{\log}(\mathbf{w}) + \frac{\lambda}{2} \cdot 2\|\mathbf{w}/2\|^2 = J_{\log}(\mathbf{w}) + \frac{\lambda}{4}\|\mathbf{w}\|^2$. The minimizer over all pairs lies on this set, so $\hat{\mathbf{w}}$ minimizes the restricted objective. The penalty thus picks the symmetric representative among all equivalent softmax parameters. ■

Exercise 4.2 (What logistic regression estimates)

Let $(X, Y) \sim P$ with $Y \in \{\pm 1\}$ and $\eta(\mathbf{x}) = P(Y = +1 \mid X = \mathbf{x}) \in (0, 1)$. For measurable g let $R(g) = \mathbb{E}[\ell(Yg(X))]$ with $\ell(z) = \log(1 + e^{-z})$.
(a) For $\eta \in (0, 1)$ show that $C_\eta(t) = \eta\,\ell(t) + (1 - \eta)\ell(-t)$ is strictly convex with unique minimizer $t^\star = \operatorname{logit}(\eta)$.
(b) Deduce that $g^\star(\mathbf{x}) = \operatorname{logit}\eta(\mathbf{x})$ minimizes R over all measurable g, and that $\operatorname{sign} g^\star$ is the Bayes classifier (predict $+1$ iff $\eta > \frac{1}{2}$).
(c) Show that $C_\eta(t^\star) = -\eta \log \eta - (1 - \eta)\log(1 - \eta)$, so $R(g^\star)$ is the expected binary entropy of $\eta(X)$.

Solution. [Supplement]

(a) By the theorem "ℓ is convex", $\ell'(z) = \sigma(z) - 1 = -\sigma(-z)$ and $\ell'' > 0$. Hence $\frac{d}{dt}\ell(-t) = \sigma(t)$ and

$$C_\eta'(t) = -\eta\,\sigma(-t) + (1 - \eta)\sigma(t) = -\eta(1 - \sigma(t)) + (1 - \eta)\sigma(t) = \sigma(t) - \eta.$$

$C_\eta''(t) = \sigma'(t) > 0$, so C_η is strictly convex and its unique stationary point, $\sigma(t) = \eta$, i.e. $t = \operatorname{logit}(\eta)$, is the unique global minimizer.

(b) Conditioning on X, $\mathbb{E}[\ell(Yg(X)) \mid X] = C_{\eta(X)}(g(X))$, so $R(g) = \mathbb{E}\,C_{\eta(X)}(g(X))$. By (a), $C_{\eta(X)}(g(X)) \geq C_{\eta(X)}(g^\star(X))$ pointwise; taking expectations gives $R(g) \geq R(g^\star)$, where $R(g^\star) \leq C_\eta(0) = \log 2$ is finite and $g^\star$ is measurable as a composition of measurable maps. Finally $\operatorname{logit}\eta > 0$ iff $\eta > \frac{1}{2}$.

(c) From $\sigma(t^\star) = \eta$: $\ell(t^\star) = -\log\sigma(t^\star) = -\log\eta$ and $\ell(-t^\star) = -\log\sigma(-t^\star) = -\log(1 - \eta)$. Substituting into C_η gives the entropy. So minimizing the logistic loss estimates the conditional log-odds, not just the decision; this is why its outputs can be read as probabilities. ■

Exercise 4.3 (The margin from the dual variables)

Let the data be linearly separable with both labels present, let $(\mathbf{w}^\star, b^\star)$ solve the hard-margin primal and $\boldsymbol{\alpha}^\star$ the dual, with the KKT conditions of the dual theorem in force.
(a) Show that $\mathbf{w}^\star$ is unique, $\mathbf{w}^\star \neq \mathbf{0}$, and $b^\star$ is uniquely determined.
(b) Show that $\|\mathbf{w}^\star\|^2 = \sum_i \alpha_i^\star$; hence the margin is $\left(\sum_i \alpha_i^\star\right)^{-1/2}$ and the primal and dual values coincide.
(c) Check (b) on the two-point example.

Solution. [Supplement]

(a) *Step 1 (uniqueness of $\mathbf{w}^\star$).* The feasible set F is an intersection of half-spaces in $(\mathbf{w}, b)$, hence convex. Suppose $(\mathbf{w}_1, b_1), (\mathbf{w}_2, b_2) \in F$ are both optimal with value $p^\star$ and $\mathbf{w}_1 \neq \mathbf{w}_2$. Their midpoint is feasible and, by the parallelogram identity,

$$\tfrac{1}{2}\left\|\tfrac{\mathbf{w}_1 + \mathbf{w}_2}{2}\right\|^2 = \tfrac{1}{4}\|\mathbf{w}_1\|^2 + \tfrac{1}{4}\|\mathbf{w}_2\|^2 - \tfrac{1}{8}\|\mathbf{w}_1 - \mathbf{w}_2\|^2 < p^\star,$$

a contradiction.

Step 2 ($\mathbf{w}^\star \neq \mathbf{0}$). With $\mathbf{w} = \mathbf{0}$ the constraints read $y_i b \geq 1$; a point with $y_i = +1$ forces $b \geq 1$ and one with $y_i = -1$ forces $b \leq -1$. So $\mathbf{w}^\star \neq \mathbf{0}$.

Step 3 ($b^\star$). Since $\mathbf{w}^\star = \sum_i \alpha_i^\star y_i \mathbf{x}_i \neq \mathbf{0}$, some $\alpha_i^\star > 0$. Complementary slackness gives $y_i(\mathbf{w}^{\star\top}\mathbf{x}_i + b^\star) = 1$, and $y_i^2 = 1$ gives $b^\star = y_i - \mathbf{w}^{\star\top}\mathbf{x}_i$, a function of $\mathbf{w}^\star$ alone.

(b) Summing complementary slackness over i and using $\sum_i \alpha_i^\star y_i = 0$,

$$\sum_i \alpha_i^\star = \sum_i \alpha_i^\star y_i(\mathbf{w}^{\star\top}\mathbf{x}_i + b^\star) = \mathbf{w}^{\star\top}\sum_i \alpha_i^\star y_i \mathbf{x}_i + b^\star \sum_i \alpha_i^\star y_i = \|\mathbf{w}^\star\|^2.$$

The margin is $1/\|\mathbf{w}^\star\|$. The primal value is $\frac{1}{2}\|\mathbf{w}^\star\|^2$, and the dual value is $\sum_i \alpha_i^\star - \frac{1}{2}\|\mathbf{w}^\star\|^2 = \frac{1}{2}\|\mathbf{w}^\star\|^2$.

(c) There $\alpha_1 = \alpha_2 = \frac{1}{4}$, so $\sum_i \alpha_i = \frac{1}{2} = \|(1/2, 1/2)\|^2$ and the margin is $(1/2)^{-1/2} = \sqrt{2}$, as found directly. ■

Chapter 5. Neural Networks and Backpropagation

Basic concepts of neural networks, multilayer perceptrons, backpropagation

1 From Hand-Designed Features to Learned Functions

A classifier maps an observation $\mathbf{x}$ to an output $y = f(\mathbf{x}; \boldsymbol{\theta})$. Classical pipelines fix features by hand (color histograms, filter banks) and learn only a final classifier; deep learning trains the representation and the classifier end to end from labeled data. Two properties of features recur: a feature is *invariant* to a transformation if it does not change, and *equivariant* if it changes in the same way (Chapter 7).

2 Neurons, Layers and Shapes

Definition 5.1: multilayer perceptron

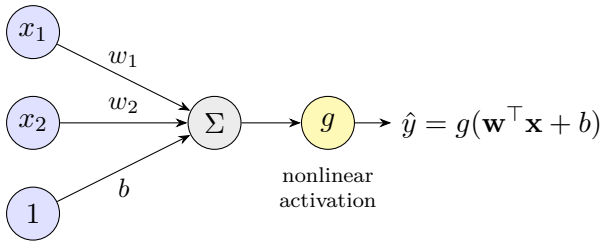
A neuron computes $h = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. A fully connected layer stacks neurons:

$$\mathbf{z}^{(\ell)} = W_\ell \mathbf{a}^{(\ell-1)} + \mathbf{b}_\ell, \qquad \mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)}), \qquad \mathbf{a}^{(0)} = \mathbf{x},$$

with $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}_\ell \in \mathbb{R}^{n_\ell}$, and σ applied elementwise. An MLP is a composition of such layers; a 1000-neuron layer is one layer, not 1000 layers.

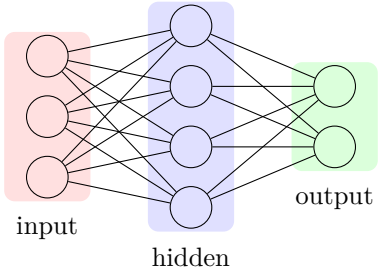
Example 5.2: parameter count

A two-layer classifier $\mathbf{s} = W_2\,\mathrm{ReLU}(W_1\mathbf{x})$ on CIFAR-10 images ($32 \times 32 \times 3 = 3072$ inputs), hidden width 100, 10 classes: $W_1 \in \mathbb{R}^{100\times 3072}$, $W_2 \in \mathbb{R}^{10\times 100}$, so $307{,}200 + 1000 = 308{,}200$ weights (plus 110 biases).



A single neuron: an affine combination of the inputs followed by a nonlinear activation g.

$$f(\mathbf{x}) = \mathrm{softmax}\left(W_2 \max(0, W_1 \mathbf{x})\right)$$



A two-layer network: every unit of a layer is connected to every unit of the next.

3 Why Nonlinearity and Depth

Without activations layers collapse: $W_2(W_1\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2 = (W_2W_1)\mathbf{x}+(W_2\mathbf{b}_1+\mathbf{b}_2)$ is still affine, however many layers there are, and an affine score yields linear decision boundaries. Hidden layers are learned embeddings: e.g. $\phi(x_1,x_2)=(x_1,x_2,x_1^2+x_2^2)$ separates an inner from an outer circle by a plane. A network learns such combinations instead of receiving them by hand; depth composes them hierarchically (strokes $\to$ loops $\to$ digits). Depth increases expressiveness but does not by itself guarantee trainability or generalization (Chapters 6–8).

4 Activation Functions and Outputs

Activation	Definition	Derivative
Sigmoid	$\sigma(z)=(1+e^{-z})^{-1}\in(0,1)$	$\sigma(1-\sigma)\in(0,1/4]$
Tanh	$\tanh z = 2\sigma(2z)-1\in(-1,1)$	$1-\tanh^2 z\in(0,1]$
ReLU	$\max(0,z)$	$\mathbf{1}\{z>0\}$
Leaky ReLU	$\max(az,z)$, $0<a<1$	1 or a
ELU	z $(z\geq 0)$, $a(e^z-1)$ $(z<0)$	1 or ae^z
Maxout	$\max_k(\mathbf{w}_k^\top\mathbf{x}+b_k)$	routes to the max

Outputs: softmax for mutually exclusive classes (loss $-\log p_y$, score gradient $\mathbf{p}-\mathbf{e}_y$), sigmoid for binary probabilities, identity for regression.

Example 5.3: forward pass

$\mathbf{x}=(1,-2)$, $W_1=\begin{pmatrix}1 & 1\\ 2 & -1\end{pmatrix}$, $W_2=\mathbf{I}$, no biases, ReLU hidden layer: $\mathbf{z}_1=(-1,4)$, $\mathbf{a}_1=(0,4)$, scores $(0,4)$, softmax $\approx(0.018,0.982)$.

5 Computation Graphs and the Scalar Chain Rule

Training repeats: zero the gradients, forward pass, loss, backward pass (backpropagation computes derivatives), optimizer update. Backpropagation applies the chain rule on a computation graph: each node multiplies the *upstream* derivative (from later operations) by its *local* derivative.

Example 5.4: $f=(x+y)z$

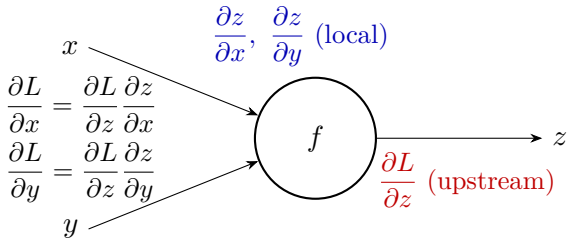
At $(x,y,z)=(-2,5,-4)$: $q=x+y=3$, $f=qz=-12$. Local derivatives $\partial f/\partial q=z=-4$, $\partial f/\partial z=q=3$, $\partial q/\partial x=\partial q/\partial y=1$, hence $\partial f/\partial x=\partial f/\partial y=-4$, $\partial f/\partial z=3$.

Example 5.5: sigmoid neuron

$f=\sigma(w_0x_0+w_1x_1+w_2)$ with $w_0=2, x_0=-1, w_1=-3, x_1=-2, w_2=-3$: $z=1$, $f\approx 0.731$, $\partial f/\partial z=f(1-f)\approx 0.197$. Then $\partial f/\partial w_0=0.197x_0=-0.197$, $\partial f/\partial x_0=0.197w_0=0.393$, $\partial f/\partial w_1=-0.393$, $\partial f/\partial x_1=-0.590$, $\partial f/\partial w_2=0.197$. Decomposing σ into $z\to -z\to e^{-z}\to 1+e^{-z}\to 1/(\cdot)$ gives the same numbers; treating σ as one node is cheaper.

Remark 5.6: gate patterns

Add gate: distributes the upstream derivative to both inputs. Multiply gate: each input receives upstream $\times$ the other input. Copy (a value used twice): derivatives add. Max gate: routes the derivative to the maximal input. Hence a shared parameter accumulates gradients from all its uses.



Backpropagation at one node: the downstream gradient is the upstream gradient times the local gradient.

6 Vector Calculus Rules for Backpropagation

Motivation. Backpropagation is the chain rule organized so that each layer passes one vector backwards. A handful of vector rules (for linear maps, elementwise nonlinearities and losses) suffice to differentiate any feedforward network.

Gradient convention: $\nabla_{\mathbf{x}} L$ has the shape of $\mathbf{x}$; the Jacobian of $\mathbf{y} \in \mathbb{R}^m$ in $\mathbf{x} \in \mathbb{R}^n$ is $m \times n$ with entries $\partial y_i / \partial x_j$.

Theorem 5.7: rules R1–R6

R1. Sum rule: the Jacobian of a sum is the sum of Jacobians; terms not depending on the variable contribute 0.

R2. Identity: $\partial \mathbf{x} / \partial \mathbf{x} = \mathbf{I}$.

R3. Chain rule: $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (outer Jacobian on the left).

R4. Affine map: $\partial(A\mathbf{x} + \mathbf{b}) / \partial \mathbf{x} = A$.

R5. Elementwise map: $\partial \sigma(\mathbf{z}) / \partial \mathbf{z} = \operatorname{diag}(\sigma'(\mathbf{z}))$, so $\operatorname{diag}(\sigma'(\mathbf{z}))\mathbf{v} = \sigma'(\mathbf{z}) \odot \mathbf{v}$.

R6. Gradient chain rule (scalar L): $\nabla_{\mathbf{x}} L = \left(\frac{\partial \mathbf{u}}{\partial \mathbf{x}}\right)^{\top} \nabla_{\mathbf{u}} L$.

Proof. R6: by R3 the row vector $\partial L / \partial \mathbf{x} = (\partial L / \partial \mathbf{u})(\partial \mathbf{u} / \partial \mathbf{x})$; transposing reverses the order. ■

Theorem 5.8: affine layer

For $\mathbf{z} = W\mathbf{a} + \mathbf{b}$ and $\boldsymbol{\delta} = \nabla_{\mathbf{z}} L$:

$$\nabla_W L = \boldsymbol{\delta}\, \mathbf{a}^{\top}, \qquad \nabla_{\mathbf{b}} L = \boldsymbol{\delta}, \qquad \nabla_{\mathbf{a}} L = W^{\top} \boldsymbol{\delta}.$$

For an activation layer $\mathbf{h} = \sigma(\mathbf{z})$: $\nabla_{\mathbf{z}} L = \nabla_{\mathbf{h}} L \odot \sigma'(\mathbf{z})$.

Proof. $\nabla_{\mathbf{a}} L$: R6 with R4. $\nabla_{\mathbf{b}} L$: R6 with $\partial \mathbf{z} / \partial \mathbf{b} = \mathbf{I}$ (R1, R2). $\nabla_W L$: row i of W, $\mathbf{w}_i^{\top}$, affects L only through $z_i = \mathbf{w}_i^{\top} \mathbf{a} + b_i$, whose gradient in $\mathbf{w}_i$ is $\mathbf{a}$ (R4); thus $\nabla_{\mathbf{w}_i} L = \delta_i \mathbf{a}$, and stacking rows gives $\boldsymbol{\delta} \mathbf{a}^{\top}$. Equivalently, componentwise $\partial L / \partial W_{ij} = \delta_i a_j$. Activation: R6 with R5, and a diagonal matrix is symmetric. ■

Remark 5.9: differentials and trace

Matrix gradients can also be read from differentials: if $dL = \operatorname{tr}(K\,dW)$ then $\nabla_W L = K^\top$. For $\mathbf{z} = W\mathbf{a}$, $dL = \boldsymbol{\delta}^\top (dW)\mathbf{a} = \operatorname{tr}(\mathbf{a}\boldsymbol{\delta}^\top dW)$, so $\nabla_W L = \boldsymbol{\delta}\mathbf{a}^\top$.

6.1 Backpropagation through an MLP

With $\boldsymbol{\delta}_\ell = \nabla_{\mathbf{z}^{(\ell)}} L$ (an $n_\ell \times 1$ column), the recursion is

$$\boldsymbol{\delta}_\ell = (W_{\ell+1}^\top \boldsymbol{\delta}_{\ell+1}) \odot \sigma'(\mathbf{z}^{(\ell)}), \qquad \nabla_{W_\ell} L = \boldsymbol{\delta}_\ell\, \mathbf{a}^{(\ell-1)\top}, \qquad \nabla_{\mathbf{b}_\ell} L = \boldsymbol{\delta}_\ell.$$

It follows from applying R6 twice: through the affine map ($W_{\ell+1}^\top$) and through the activation (elementwise product). In a batch, outer products are summed (or averaged) over samples.

Example 5.10: forward example continued backward

Take class 1 as correct: $\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{e}_1 \approx (-0.982, 0.982)$, $\nabla_{W_2} L = \boldsymbol{\delta}_2 \mathbf{a}_1^\top = \begin{pmatrix} 0 & -3.928 \\ 0 & 3.928 \end{pmatrix}$. The first hidden pre-activation was negative, so $\boldsymbol{\delta}_1 = (0, 0.982)$ and $\nabla_{W_1} L = \boldsymbol{\delta}_1 \mathbf{x}^\top = \begin{pmatrix} 0 & 0 \\ 0.982 & -1.964 \end{pmatrix}$: the inactive unit receives no gradient (a "dead ReLU" for this input). For elementwise ReLU, upstream $(4, -1, 5, 9)$ at $\mathbf{x} = (1, -2, 3, -1)$ becomes $(4, 0, 5, 0)$ with no Jacobian ever formed.

7 Worked Results

Motivation. Two standard computations show what the rules reveal: why cross-entropy trains a sigmoid neuron faster than squared error, and how a residual connection keeps the backward signal from vanishing.

7.1 Single neurons: MSE versus cross-entropy

Theorem 5.11: gradient bounds

For $y = \sigma(wx + b)$ and $C = \frac{1}{2}(y - z)^2$: $\nabla C = y(1-y)(y-z)(x, 1)$ and $\|\nabla C\| < \frac{1}{4}\sqrt{1 + x^2}\,(1 + |z|)$. For cross-entropy $C = -[z \log y + (1 - z) \log(1 - y)]$: $\nabla C = (y - z)(x, 1)$.

Proof. Chain rule with $\sigma' = y(1 - y) \le 1/4$ and $|y - z| \le |y| + |z| < 1 + |z|$ since $0 < y < 1$. For cross-entropy, $\partial C/\partial t = -[z(1 - y) - (1 - z)y] = y - z$: the factor σ' cancels. ∎

For a saturated wrong neuron ($z = 1$, $wx + b = -10$), the MSE gradient carries the factor $\sigma' \approx 4.5 \times 10^{-5}$ while the cross-entropy gradient is of order one: MSE with sigmoid suffers a learning slowdown. GD reads $(w_{n+1}, b_{n+1}) = (w_n, b_n) - \eta \nabla C$.

7.2 Residual MLP

Consider $y_\ell = \sigma(A_\ell y_{\ell-1} + b_\ell) + y_{\ell-1}$ for $2 \le \ell \le L - 1$, $y_1 = \sigma(A_1 x + b_1)$, $y_L = A_L y_{L-1} + b_L$ (scalar). Put $z_\ell = A_\ell y_{\ell-1} + b_\ell$, $D_\ell = \operatorname{diag}(\sigma'(z_\ell))$ and $g_\ell = \partial y_L / \partial y_\ell \in \mathbb{R}^{1 \times m}$.

Theorem 5.12: residual Jacobians and gradients

$$\frac{\partial y_\ell}{\partial y_{\ell-1}} = I + D_\ell A_\ell \ (2 \le \ell \le L-1), \qquad \frac{\partial y_L}{\partial y_{L-1}} = A_L, \qquad g_\ell = A_L \prod_{j=L-1}^{\ell+1} (I + D_j A_j),$$

$$\frac{\partial y_L}{\partial b_\ell} = g_\ell D_\ell, \qquad \frac{\partial y_L}{\partial A_\ell} = D_\ell g_\ell^\top y_{\ell-1}^\top \quad \left(\tfrac{\partial y_L}{\partial (A_\ell)_{ij}} = (g_\ell)_i \sigma'((z_\ell)_i)(y_{\ell-1})_j\right),$$

and $\partial y_L/\partial b_L = 1$, $\partial y_L/\partial A_L = y_{L-1}^\top$.

Proof. R1 splits y_ℓ into $\sigma(z_\ell)$ and $y_{\ell-1}$; R2 gives I; R3 with R5, R4 gives $D_\ell A_\ell$. For b_ℓ, the residual term does not depend on b_ℓ and $\partial z_\ell/\partial b_\ell = I$, so $\partial y_\ell/\partial b_\ell = D_\ell$; chain with g_ℓ. For A_ℓ, the ith row affects y_L only through $(z_\ell)_i$, giving the componentwise formula; it factorizes into an i-part and a j-part, i.e. an outer product. ■

If $A_j = 0$ or $\sigma'(z_j) = 0$ for some intermediate j, the factor is $I + 0 = I$, not 0: the identity path carries the gradient, so lower-layer gradients need not vanish. Without the residual, the factor is $D_j A_j = 0$ and all lower gradients are exactly zero. Expanding $\prod(I + D_j A_j) = I + \sum D_j A_j + \cdots$ exhibits the skip path explicitly (Chapter 7).

8 Selected exercises

Motivation. The exercises prove that a hidden layer is genuinely needed for XOR and justify the finite-difference test used to check a backward pass.

Exercise 5.1 (XOR needs a hidden layer)

Let $A = \{(0,0),(1,1)\}$ and $B = \{(0,1),(1,0)\}$.
(a) Show that no affine $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ satisfies $f > 0$ on B and $f < 0$ on A, nor the reverse.
(b) Show that $g(\mathbf{x}) = \mathrm{ReLU}(x_1 + x_2) - 2\,\mathrm{ReLU}(x_1 + x_2 - 1)$ equals 0 on A and 1 on B.
(c) Show that $\{\mathbf{x} \in \mathbb{R}^2 : g(\mathbf{x}) > \frac{1}{2}\}$ is the open strip $\{\frac{1}{2} < x_1 + x_2 < \frac{3}{2}\}$, which is not a half-plane.

Solution. [Supplement]

(a) Suppose $f > 0$ on B and $f < 0$ on A. From B: $w_2 + b > 0$ and $w_1 + b > 0$, so $w_1 + w_2 + 2b > 0$. From A: $b < 0$ and $w_1 + w_2 + b < 0$, so $w_1 + w_2 + 2b < 0$, a contradiction. The reverse case is the same argument applied to $-f$.

(b) g depends on $\mathbf{x}$ only through $t = x_1 + x_2$, via $h(t) = \mathrm{ReLU}(t) - 2\,\mathrm{ReLU}(t-1)$. On A, $t \in \{0, 2\}$ and $h(0) = 0$, $h(2) = 2 - 2 = 0$; on B, $t = 1$ and $h(1) = 1$.

(c) Piecewise, $h(t) = 0$ for $t \le 0$, $h(t) = t$ for $0 \le t \le 1$, and $h(t) = 2 - t$ for $t \ge 1$. Hence $h(t) > \frac{1}{2}$ iff $\frac{1}{2} < t < \frac{3}{2}$. The complement of a half-plane is a half-plane, hence convex. The complement of the strip contains $(0,0)$ and $(1,1)$ but not their midpoint $(\frac{1}{2}, \frac{1}{2})$, so it is not convex and the strip is not a half-plane. Two ReLU units with fixed weights already produce a non-linear decision boundary. ■

Exercise 5.2 (Checking a backward pass by finite differences)

Let $\varphi : \mathbb{R} \to \mathbb{R}$ be three times continuously differentiable on $[x - h, x + h]$, $h > 0$, and $M_k = \max_{[x-h,x+h]} |\varphi^{(k)}|$.
(a) Show that $\left|\frac{\varphi(x+h) - \varphi(x)}{h} - \varphi'(x)\right| \le \frac{h}{2} M_2$.

(b) Show that $\frac{\varphi(x+h)-\varphi(x-h)}{2h} - \varphi'(x) = \frac{h^2}{6}\varphi'''(\xi)$ for some $\xi \in [x-h, x+h]$, so the error is at most $\frac{h^2}{6}M_3$.
(c) For $\varphi = \exp$ at $x = 0$ and $h = 10^{-3}$, compute both errors to leading order. Explain how this checks $\partial L/\partial\theta_j$ computed by backpropagation.

Solution. [Supplement]

(a) Taylor's theorem with Lagrange remainder gives $\varphi(x+h) = \varphi(x) + h\varphi'(x) + \frac{h^2}{2}\varphi''(\xi)$ with $\xi \in (x, x+h)$. Rearranging, the error equals $\frac{h}{2}\varphi''(\xi)$, bounded by $\frac{h}{2}M_2$.

(b) *Step 1 (two expansions).* To third order,

$$\varphi(x \pm h) = \varphi(x) \pm h\varphi'(x) + \frac{h^2}{2}\varphi''(x) \pm \frac{h^3}{6}\varphi'''(\xi_\pm), \qquad \xi_+ \in (x, x+h),\ \xi_- \in (x-h, x).$$

Step 2 (subtract). The even terms cancel: $\varphi(x+h) - \varphi(x-h) = 2h\varphi'(x) + \frac{h^3}{6}(\varphi'''(\xi_+) + \varphi'''(\xi_-))$. Dividing by $2h$, the error is $\frac{h^2}{6} \cdot \frac{1}{2}(\varphi'''(\xi_+) + \varphi'''(\xi_-))$.

Step 3 (one point). The average of two values of the continuous φ''' lies between them, so by the intermediate value theorem it equals $\varphi'''(\xi)$ for some ξ between ξ_- and ξ_+.

(c) Here $\varphi'(0) = 1$. The forward quotient is $(e^h - 1)/h = 1 + \frac{h}{2} + \frac{h^2}{6} + \cdots$, with error about 5.0×10^{-4}; the central quotient is $\sinh(h)/h = 1 + \frac{h^2}{6} + \cdots$, with error about 1.7×10^{-7}. To check backpropagation, apply (b) to $\varphi(t) = L(\boldsymbol{\theta} + t\mathbf{e}_j)$, whose derivative at $t = 0$ is $\partial L/\partial\theta_j$: two forward passes at $\boldsymbol{\theta} \pm h\mathbf{e}_j$ give an independent estimate accurate to $O(h^2)$, to be compared (in relative error) with the backpropagated value. In floating point h cannot be taken too small, since rounding errors of size about ϵ_{mach}/h then dominate. ■

Chapter 6. Training and Optimization of Deep Networks

Mini-batch SGD, activations, weight initialization, learning rates, batch normalization; Taylor expansion and the descent lemma, SGD, momentum, Nesterov, AdaGrad, RMSProp, Adam

1 Full, Stochastic and Mini-Batch Gradients

Motivation. Training minimizes an average loss over millions of examples, and computing the exact gradient at each step is too expensive. Gradients on small random batches are unbiased and cheap, which is the basis of all practical training.

For $f(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} f_i(\boldsymbol{\theta})$, the full gradient costs N per-sample gradients. Choosing I uniformly, $\mathbf{g} = \nabla f_I$ is **unbiased**: $\mathbb{E}[\mathbf{g} \mid \boldsymbol{\theta}] = \frac{1}{N}\sum_i \nabla f_i = \nabla f$. A mini-batch averages b such gradients.

Theorem 6.1: mini-batch variance

If the b indices are drawn independently with replacement and a single draw has conditional covariance Σ, the mini-batch gradient has covariance Σ/b. Without replacement it is $\frac{N-b}{b(N-1)}\Sigma$ (with Σ normalized by N).

Proof. With replacement the b terms are iid with covariance Σ; the covariance of their average is $\frac{1}{b^2}\cdot b\Sigma$. Without replacement the pairwise covariances are $-\Sigma/(N-1)$, which gives the finite-population correction. ■

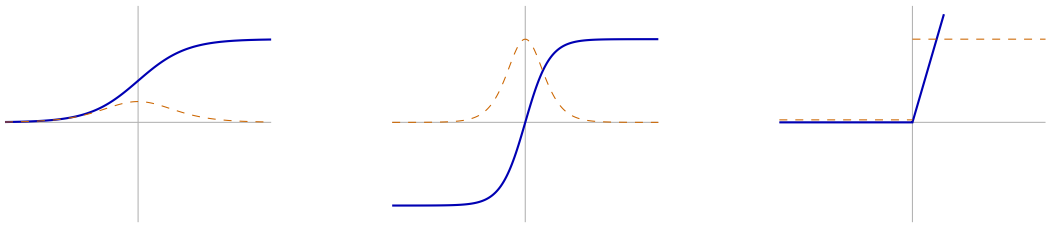
Example 6.2: unbiased but noisy

$f_i(w) = \frac{1}{2}(w - y_i)^2$ with $\mathbf{y} = (1, 3, 5)$ at $w = 0$: per-sample gradients $-1, -3, -5$, full gradient -3. A single random gradient averages to -3; a two-sample batch without replacement gives $-2, -3, -4$, again averaging to -3.

One *iteration* is one parameter update; one *epoch* is one pass through the data (1000 samples, batch 100: 10 iterations per epoch). Full batch: deterministic but expensive; single sample: cheap but noisy; mini-batch: a compromise.

2 Activation Functions Revisited

Sigmoid. Saturates: $\sigma' \le 1/4$ and $\sigma' \to 0$ for large $|z|$. Outputs are positive, so for the next layer $h = W\mathbf{z}$ (with $\mathbf{z}$ the positive sigmoid outputs) $\partial L/\partial W_{ij} = \delta_i z_j$ has the sign of δ_i for every j: all weights into a neuron move in the same direction in one step, producing zig-zag paths. **Tanh.** Zero-centered and odd, $\tanh' \in (0, 1]$, but still saturates near ± 1. **ReLU.** Cheap, derivative 1 on the positive side (no saturation there), but units with negative input get zero gradient (dead ReLU) and outputs are not centered. Leaky ReLU, ELU and maxout keep a nonzero slope or add flexibility.



Activation functions (solid) and their derivatives (dashed): the sigmoid and tanh *saturate, ReLU has derivative* 0 *or* 1.

3 Weight Initialization

3.1 Why the scale matters

`W = s * np.random.randn(Din, Dout)` draws iid $N(0, s^2)$ weights. For $z_j = \sum_{i=1}^{D_{\text{in}}} W_{ji} x_i$ with zero-mean independent weights of variance σ_W^2, independent of inputs with $\mathbb{E}x_i^2 = q$,

$$\mathbb{E}z_j = 0, \qquad \operatorname{Var} z_j = D_{\text{in}} \sigma_W^2 q$$

(cross terms vanish because $\mathbb{E}[W_{ji}W_{jk}] = 0$ for $i \neq k$). For six tanh layers of width 4096:

- $s = 0.01$: $D_{\text{in}} s^2 = 0.41$; activations shrink towards 0 layer by layer, and weight gradients $\propto$ inputs become tiny.
- $s = 0.05$: $D_{\text{in}} s^2 = 10.24$; pre-activations have standard deviation ≈ 3, tanh saturates (histograms pile up at ± 1), $\tanh' \approx 0$, and the backpropagated signal $\partial L/\partial \mathbf{z} = \sigma'(\cdot) W^\top \partial L/\partial \mathbf{h}$ is almost zero: no learning, although the loss is still large. A zero gradient caused by saturation is not a sign of optimality.

3.2 Zero and symmetric initialization

Theorem 6.3: zero initialization freezes a tanh MLP

For $f(\mathbf{x}) = A_L y_{L-1} + b_L$, $y_\ell = \tanh(A_\ell y_{\ell-1} + b_\ell)$, all parameters initialized to 0 and GD on $\frac{1}{2}\sum_i (f(X_i) - Y_i)^2$: $A_1, \ldots, A_L, b_1, \ldots, b_{L-1}$ stay 0 forever, and $b_L \to \bar{Y}$ if $0 < \eta < 2/N$.

Proof. Induction on the claim "all parameters except b_L are zero". Forward: $\tanh(0) = 0$ so every hidden output is 0 and $f = b_L$. Backward: $\partial/\partial A_L = \sum_i \delta_L^{(i)} y_{L-1}^\top = 0$; $\boldsymbol{\delta}_{L-1} = (A_L^\top \delta_L) \odot \tanh'(0) = 0$ since $A_L = 0$, and inductively all $\boldsymbol{\delta}_\ell = 0$, so $\partial A_\ell = \boldsymbol{\delta}_\ell y_{\ell-1}^\top = 0$, $\partial b_\ell = 0$. Hence GD does not move them. For b_L: $J = \frac{1}{2}\sum (b_L - Y_i)^2$, $b^{(t+1)} - \bar{Y} = (1 - \eta N)(b^{(t)} - \bar{Y})$. ■

Theorem 6.4: duplicate neurons stay duplicates

For $f(x) = \sum_{j=1}^{p} u_j \sigma(a_j x + b_j)$ trained by Adam (or GD, momentum, RMSProp), if $(a_{p-1}, b_{p-1}, u_{p-1}) = (a_p, b_p, u_p)$ at initialization, they remain equal for all iterations.

Proof. $\partial f/\partial u_j = \sigma(a_j x + b_j)$, $\partial f/\partial a_j = u_j \sigma'(a_j x + b_j) x$, $\partial f/\partial b_j = u_j \sigma'(a_j x + b_j)$; the loss gradient multiplies these by $\partial \ell/\partial f$, which is shared by all neurons. So equal parameters give equal gradient coordinates. Induct on the statement "the paired coordinates of $\boldsymbol{\theta}^k$, m_1^k, m_2^k agree": it holds at $k = 0$ ($m_1^0 = m_2^0 = 0$), and every Adam operation (EMAs, bias corrections, elementwise division) acts coordinatewise. The optimizer state must be part of the induction hypothesis. ■

Hence identical neurons compute the same function forever: random initialization is needed to break symmetry.

3.3 Xavier and He initialization

For an approximately linear activation, preserving the second moment requires $D_{\text{in}}\sigma_W^2 = 1$, i.e. $\sigma_W = 1/\sqrt{D_{\text{in}}}$ (often called "Xavier"; Glorot's compromise $2/(D_{\text{in}} + D_{\text{out}})$ also balances the backward pass).

Theorem 6.5: He initialization

Let $y^+ = A\mathbf{x}$, $\mathbf{x} = \text{ReLU}(\mathbf{y})$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$, where $\mathbf{y}$ is independent of A, its coordinates have mean 0, variance 1, are uncorrelated, and $\mathbf{y} \overset{d}{=} -\mathbf{y}$. Then

(i) $\mathbb{E}x_j^2 = \frac{1}{2}$; (ii) $\mathbb{E}y^+ = 0$; (iii) $\mathbb{E}(y_i^+)^2 = \frac{n_{\text{in}}\sigma_A^2}{2}$;

(iv) $y_1^+, \dots, y_{n_{\text{out}}}^+$ are uncorrelated; (v) $y^+ \overset{d}{=} -y^+$.

Thus $\sigma_A^2 = 2/n_{\text{in}}$ preserves the second moment through ReLU layers.

Proof. (i) $\text{ReLU}(y)^2 = y^2\mathbf{1}\{y > 0\}$; symmetry makes the positive and negative halves of $\mathbb{E}y^2 = 1$ equal ($y = 0$ contributes nothing).

(ii) Independence and $\mathbb{E}A_{ij} = 0$.

(iii) $\mathbb{E}(y_i^+)^2 = \sum_{j,k} \mathbb{E}[A_{ij}A_{ik}]\mathbb{E}[x_j x_k] = \sum_j \sigma_A^2 \mathbb{E}x_j^2$.

(iv) For $i \neq k$ all $\mathbb{E}[A_{ij}A_{kl}] = 0$.

(v) $(A, \mathbf{y}) \overset{d}{=} (-A, \mathbf{y})$ since A is independent of $\mathbf{y}$ and symmetric; apply $F(A, \mathbf{y}) = A\,\text{ReLU}(\mathbf{y})$. ■

Remark 6.6: refinements

Leaky ReLU with slope α: $\mathbb{E}\phi(y)^2 = \frac{1+\alpha^2}{2}$ and $\sigma_A^2 = \frac{2}{(1+\alpha^2)n_{\text{in}}}$. ReLU output is not centered: for $z \sim N(0, v)$, $\mathbb{E}\,\text{ReLU}(z) = \sqrt{v/(2\pi)}$, $\mathbb{E}\,\text{ReLU}(z)^2 = v/2$, so He preserves the second moment, not the variance. Backward, $\partial\ell/\partial\mathbf{x} = A^\top \partial\ell/\partial\mathbf{y}^+$ has variance $n_{\text{out}}\sigma_A^2$: forward and backward preservation ask for n_{in} versus n_{out}.

Theorem 6.7: sigmoid layer

For $\mathbf{y} = \sigma(A\mathbf{x})$ with $\sigma(t) \approx \frac{1}{2} + \frac{t}{4}$, $\sigma' \approx \frac{1}{4}$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$ independent of $\mathbf{x}$, and x_j uncorrelated with mean $\frac{1}{2}$ and variance 1: $\mathbb{E}y_i = \frac{1}{2}$, $\text{Var}\, y_i = \frac{5n_{\text{in}}\sigma_A^2}{64}$. If the upstream gradients $\partial\ell/\partial y_i$ have mean 0, variance 1, are uncorrelated and independent of A, then $\partial\ell/\partial x_j \approx \frac{1}{4}\sum_i A_{ij}\partial\ell/\partial y_i$ has mean 0 and variance $\frac{n_{\text{out}}\sigma_A^2}{16}$.

Proof. $\text{Var}(A_{ij}x_j) = \mathbb{E}A_{ij}^2\mathbb{E}x_j^2 = \sigma_A^2(1 + \frac{1}{4})$ and the terms are uncorrelated; multiply by $\frac{1}{16}$. The backward computation is identical with $\mathbb{E}(\partial\ell/\partial y_i)^2 = 1$. ■

Theorem 6.8: convolution layer

For a 3×3, padding-1 convolution $Y_{cij} = \sum_{p,q,r} w_{cpqr}\tilde{X}_{p,i+q-2,j+r-2} + b_c$ with zero-mean unit-variance uncorrelated inputs, $w \overset{\text{iid}}{\sim} N(0, \sigma_w^2)$, $b \overset{\text{iid}}{\sim} N(0, \sigma_b^2)$ independent: $\mathbb{E}Y_{cij} = 0$

everywhere, and $\mathbb{E}Y_{cij}^2 = \sigma_w^2 \cdot \#\{\text{in-range inputs}\} + \sigma_b^2$, i.e. $9C_{\text{in}}\sigma_w^2 + \sigma_b^2$ in the interior, $6C_{\text{in}}\sigma_w^2 + \sigma_b^2$ on edges and $4C_{\text{in}}\sigma_w^2 + \sigma_b^2$ at corners.

Proof. Expand the square: weight–weight terms leave $\sigma_w^2 \mathbb{E}\tilde{X}_a^2$ on the diagonal, weight–bias terms vanish, and $\mathbb{E}b_c^2 = \sigma_b^2$. Padded entries are zero. ■

3.4 Variance of a sigmoid output

For smooth f and X with mean μ, Taylor's theorem $f(X) = f(\mu) + f'(\mu)(X-\mu) + \frac{1}{2}f''(\xi)(X-\mu)^2$ gives $\operatorname{Var} f(X) = f'(\mu)^2 \operatorname{Var} X + \text{Error}$, the error collecting terms with f''. For $X \sim N(0,1)$ and $Y = \sigma(wX + b)$, $\operatorname{Var} Y \approx w^2\sigma'(b)^2$: small weights or large $|b|$ (saturation) give small output variance. An exact bound also holds: if f is L-Lipschitz, $\operatorname{Var} f(X) \le \mathbb{E}(f(X) - f(\mu))^2 \le L^2 \operatorname{Var} X$, so $\operatorname{Var} Y \le w^2/16$.

4 Learning-Rate Schedules

The learning rate controls the update size: too large diverges, too small is slow, and a large early rate followed by a smaller late rate refines the solution and reduces gradient noise.

- Step: multiply by 0.1 at fixed epochs (ResNet: 30, 60, 90), giving $\eta_0, 0.1\eta_0, 0.01\eta_0, 0.001\eta_0$.
- Cosine: $\eta_t = \frac{\eta_0}{2}[1 + \cos(\pi t/T)]$; linear: $\eta_t = \eta_0(1 - t/T)$; inverse square root: $\eta_t = \eta_0/\sqrt{t}$.
- Linear warmup: increase from 0 to $\eta_{\max}$ over the first iterations (e.g. 5000), then decay.

5 Batch Normalization

5.1 Training-time computation

For a mini-batch $X \in \mathbb{R}^{B \times D}$ (rows: samples, columns: features),

$$\mu_j = \frac{1}{B}\sum_i X_{ij}, \quad v_j = \frac{1}{B}\sum_i (X_{ij} - \mu_j)^2, \quad \hat{X}_{ij} = \frac{X_{ij} - \mu_j}{\sqrt{v_j + \epsilon}}, \quad Y_{ij} = \gamma_j \hat{X}_{ij} + \beta_j.$$

γ, β are learned, so the network can choose the output scale and center (with $\gamma_j = \sqrt{v_j + \epsilon}$, $\beta_j = \mu_j$ it recovers the input for that batch).

Example 6.9: two features

$X = \begin{pmatrix} 1 & 10 \\ 3 & 14 \end{pmatrix}$: $\mu = (2, 12)$, $v = (1, 4)$, $\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$ (ϵ ignored). With $\gamma = (2, 3)$, $\beta = (-1, 5)$, $Y = \begin{pmatrix} -3 & 2 \\ 1 & 8 \end{pmatrix}$.

Theorem 6.10: invariances

With $\epsilon = 0$:
(i) each column of $\hat{X}$ has mean 0 and mean square 1;

(ii) for $z = \mathbf{w}^\top \mathbf{x} + b$ the BN output does not depend on b and is invariant under $\mathbf{w} \mapsto c\mathbf{w}$, $c > 0$;
(iii) consequently $\langle \nabla_{\mathbf{w}} L, \mathbf{w} \rangle = 0$.

Proof. (ii) $z - \mu = \mathbf{w}^\top(\mathbf{x} - \bar{\mathbf{x}})$ removes b; scaling $\mathbf{w}$ scales both $z - \mu$ and $\sqrt{v}$ by c.
(iii) $h(c) = L(c\mathbf{w})$ is constant, so $0 = h'(1) = \nabla L(\mathbf{w})^\top \mathbf{w}$. ■

5.2 Test time

At inference the batch may have size one, and the output must not depend on the other samples. During training BN keeps running averages

$$\mu^{\text{run}} \leftarrow m\,\mu^{\text{run}} + (1-m)\mu_{\text{batch}}, \qquad (\sigma^2)^{\text{run}} \leftarrow m\,(\sigma^2)^{\text{run}} + (1-m)\sigma^2_{\text{batch}}$$

(an exponential moving average, as in Adam; PyTorch's `momentum` argument is $1 - m$), and at test time uses

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j^{\text{run}}}{\sqrt{(\sigma_j^2)^{\text{run}} + \epsilon}}, \qquad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

With a batch of one and batch statistics, $\mu = x$ and $v = 0$ would give $\hat{x} = 0$: all information lost. In code: `model.train()` versus `model.eval()`.

Theorem 6.11: fusion at inference

At test time BN is the affine map $z \mapsto az + c$ with $a_j = \gamma_j / \sqrt{v_j^{\text{run}} + \epsilon}$, $c_j = \beta_j - a_j \mu_j^{\text{run}}$, so $\text{BN}(W\mathbf{x} + \mathbf{b}) = \text{diag}(\mathbf{a})W\mathbf{x} + (\text{diag}(\mathbf{a})\mathbf{b} + \mathbf{c})$ can be folded into the preceding layer.

Remark 6.12: why BN helps

Reported benefits: easier optimization, better gradient flow, larger learning rates, less sensitivity to initialization, mild regularization through batch noise. The original "internal covariate shift" explanation has been challenged; a smoother loss landscape is an alternative view. In training mode BN couples samples in the backward pass: with $g_i = \partial L / \partial y_i$ and $r = \sqrt{v + \epsilon}$, $\partial L / \partial x_i = \frac{\gamma}{r}(g_i - \bar{g} - \hat{x}_i \overline{g\hat{x}})$.

6 Gradient Descent and Taylor's Theorem

Motivation. Why does stepping against the gradient decrease the loss? A first-order Taylor expansion says that for a small enough step it must; the rest of the chapter quantifies "small enough".

We minimize $f : \mathbb{R}^d \to \mathbb{R}$; the iterate $\mathbf{x}_t$ is the whole parameter vector. GD is $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$, and $-\nabla f$ is the steepest-descent direction since $\nabla f^\top \mathbf{v} \geq -\|\nabla f\|$ for unit $\mathbf{v}$ (Cauchy–Schwarz).

Theorem 6.13: Taylor along a segment

With $\mathbf{d} = \mathbf{y} - \mathbf{x}$ and $h(t) = f(\mathbf{x} + t\mathbf{d})$,

$$f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x} + t\mathbf{d})^\top \mathbf{d}\, dt, \qquad f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{d} + \int_0^1 (1-t)\, \mathbf{d}^\top \nabla^2 f(\mathbf{x} + t\mathbf{d})\mathbf{d}\, dt.$$

Proof. The first is $h(1) - h(0) = \int_0^1 h'$. For the second, $h(1) = h(0) + h'(0) + \int_0^1 \int_0^t h''(s)\, ds\, dt$ and exchanging the order over $0 \leq s \leq t \leq 1$ gives $\int_0^1 (1-s) h''(s)\, ds$. ■

7 Smoothness and the Descent Lemma

Motivation. If the gradient is L-Lipschitz, the loss lies below a quadratic around each point. This descent lemma gives a safe step size $1/L$ and a guaranteed decrease per step, the standard starting point of convergence proofs.

Definition 6.14: L-smoothness

f is L-smooth if $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \le L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$. For $f \in C^2$ this is equivalent to $-L\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$ (all Hessian eigenvalues in $[-L, L]$).

Theorem 6.15: descent lemma

If f is L-smooth, $f(\mathbf{y}) \le f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y} - \mathbf{x}) + \frac{L}{2}\|\mathbf{y} - \mathbf{x}\|^2$.

Proof. Subtract $\nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 \nabla f(\mathbf{x})^\top \mathbf{d}\, dt$ from the first Taylor formula, then use Cauchy–Schwarz and smoothness:

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x}))^\top \mathbf{d}\, dt \le \int_0^1 Lt\|\mathbf{d}\|^2 dt = \frac{L}{2}\|\mathbf{d}\|^2.$$

■

Theorem 6.16: GD on smooth functions

GD satisfies $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta(1 - \frac{L\eta}{2})\|\nabla f(\mathbf{x}_t)\|^2$, so it decreases f when $0 < \eta < 2/L$. With $\eta = 1/L$ and $f^\star = \inf f > -\infty$,

$$\min_{0 \le t < T} \|\nabla f(\mathbf{x}_t)\|^2 \le \frac{1}{T}\sum_{t<T} \|\nabla f(\mathbf{x}_t)\|^2 \le \frac{2L\,(f(\mathbf{x}_0) - f^\star)}{T}.$$

Proof. Put $\mathbf{y} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$ in the descent lemma. With $\eta = 1/L$ the coefficient is $1/(2L)$; summing over t telescopes the left side to $f(\mathbf{x}_0) - f(\mathbf{x}_T) \le f(\mathbf{x}_0) - f^\star$. ■

This is a stationarity guarantee: for nonconvex f a small gradient may sit at a saddle point or a local minimum. For $f(x) = x^2/2$ ($L = 1$), $x_{t+1} = (1 - \eta)x_t$ converges iff $0 < \eta < 2$; for $f = \frac{1}{2}\mathbf{x}^\top H\mathbf{x}$ each eigen-direction contracts by $1 - \eta\lambda_i$, so a small eigenvalue (poor conditioning) means slow progress while $\eta < 2/\lambda_{\max}$ is required for stability.

8 Stochastic Gradient Descent

Motivation. With noisy gradients the iterates no longer settle down at a fixed step size. The analysis shows how the noise floor depends on the step size and why decreasing step sizes are needed for convergence.

SGD uses $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \mathbf{g}_t$ with $\mathbb{E}_t \mathbf{g}_t = \nabla f(\mathbf{x}_t)$, where $\mathbb{E}_t$ conditions on the history up to $\mathbf{x}_t$.

Theorem 6.17: second moment is not variance

$\mathbb{E}_t\|\mathbf{g}_t\|^2 = \|\nabla f(\mathbf{x}_t)\|^2 + \mathbb{E}_t\|\mathbf{g}_t - \nabla f(\mathbf{x}_t)\|^2$.

Proof. Expand $\mathbf{g}_t = \nabla f + (\mathbf{g}_t - \nabla f)$; the cross term has conditional mean zero. ■

Theorem 6.18: $T^{-1/2}$ rate for SGD

If f is L-smooth, $\mathbb{E}_t\|\mathbf{g}_t\|^2 \le G$ and $\Delta = f(\mathbf{x}_0) - f^\star$, then for a constant step

$$\frac{1}{T}\sum_{t<T}\mathbb{E}\|\nabla f(\mathbf{x}_t)\|^2 \le \frac{\Delta}{\eta T} + \frac{L\eta G}{2}, \qquad \text{and with } \eta = \sqrt{\tfrac{2\Delta}{LGT}}: \quad \le \sqrt{\frac{2L\Delta G}{T}}.$$

Proof. Descent lemma: $f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\nabla f(\mathbf{x}_t)^\top \mathbf{g}_t + \frac{L\eta^2}{2}\|\mathbf{g}_t\|^2$. Conditional expectation gives $\mathbb{E}_t f(\mathbf{x}_{t+1}) \le f(\mathbf{x}_t) - \eta\|\nabla f(\mathbf{x}_t)\|^2 + \frac{L\eta^2}{2}G$. Take total expectation (tower property), sum over $t < T$ (telescoping), use $\mathbb{E}f(\mathbf{x}_T) \ge f^\star$ and divide by ηT. Finally $\frac{a}{\eta} + b\eta$ is minimized at $\eta = \sqrt{a/b}$ with value $2\sqrt{ab}$. ■

The noise term forces $\eta \sim T^{-1/2}$, slower than GD's T^{-1}. Since mini-batches of size B divide the noise variance by B, larger batches reduce G and permit larger steps.

9 Momentum

Motivation. Gradient descent zigzags in narrow valleys. Momentum averages past gradients so that oscillating components cancel and consistent ones accumulate; Nesterov's variant evaluates the gradient at a look-ahead point.

9.1 Heavy-ball momentum

Definition 6.19: momentum

$\mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{x}_t)$, $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}$, $\mathbf{v}_0 = \mathbf{0}$, $0 \le \beta < 1$.

Unrolling, $\mathbf{v}_{t+1} = -\eta\sum_{k=0}^{t}\beta^{t-k}\nabla f(\mathbf{x}_k)$: an exponentially weighted sum of past gradients (a velocity). For a constant gradient $\mathbf{g}$, $\|\mathbf{v}_t\| \to \eta\|\mathbf{g}\|/(1-\beta)$, an effective step $1/(1-\beta)$ times larger ($10\times$ for $\beta = 0.9$). Persistent directions accumulate; oscillating directions cancel, which reduces zig-zagging in narrow valleys. Physically the gradient acts as a force and $1-\beta$ as friction: in the buffer form $\mathbf{m}_{t+1} = \mathbf{g}_t + \beta\mathbf{m}_t$ one has $\mathbf{m}_{t+1} - \mathbf{m}_t = \mathbf{g}_t - (1-\beta)\mathbf{m}_t$.

Theorem 6.20: momentum on a quadratic

For $f(x) = \frac{h}{2}x^2$, $x_{t+1} = (1+\beta-\eta h)x_t - \beta x_{t-1}$; the iteration converges iff both roots of $r^2 - (1+\beta-\eta h)r + \beta = 0$ lie in the open unit disk, which for $0 \le \beta < 1$ holds iff $0 < \eta h < 2(1+\beta)$. Complex roots (modulus $\sqrt{\beta}$) mean damped oscillation.

Proof. Substitute $\mathbf{v}_{t+1} = x_{t+1} - x_t$ and $\mathbf{v}_t = x_t - x_{t-1}$ into the velocity update. For the root condition use the Jury/Schur criterion for $r^2 + a_1 r + a_0$: $|a_0| < 1$, $1 + a_1 + a_0 > 0$, $1 - a_1 + a_0 > 0$, which here read $\beta < 1$, $\eta h > 0$, $\eta h < 2(1+\beta)$. ■

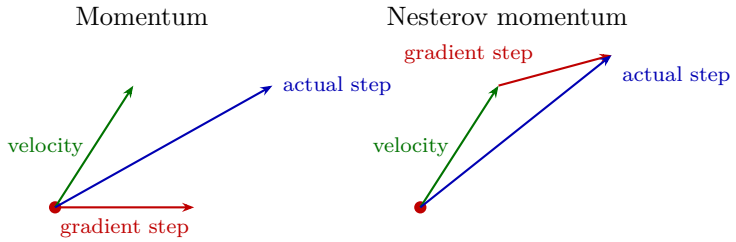
9.2 Nesterov momentum

Nesterov evaluates the gradient at the look-ahead point:

$$\mathbf{y}_t = \mathbf{x}_t + \beta\mathbf{v}_t, \qquad \mathbf{v}_{t+1} = \beta\mathbf{v}_t - \eta\nabla f(\mathbf{y}_t), \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}.$$

Example 6.21: a numerical illustration

$f = x^2/2$, $x_0 = 1$, $\eta = 0.1$, $\beta = 0.9$. Both methods give $v_1 = -0.1$, $x_1 = 0.9$. Momentum: $v_2 = 0.9(-0.1) - 0.1(0.9) = -0.18$, $x_2 = 0.72$. Nesterov: $y_1 = 0.81$, $v_2 = -0.09 - 0.081 = -0.171$, $x_2 = 0.729$: it measures the slope where it is about to go and brakes earlier (not necessarily closer to the optimum at every step).



Momentum adds the gradient step taken at the current point to the velocity; Nesterov evaluates the gradient after first moving along the velocity.

10 Adaptive Methods

Motivation. Different parameters often need different step sizes. AdaGrad, RMSProp and Adam rescale each coordinate by a running estimate of its gradient magnitude; this section derives their update rules and the bias correction in Adam.

All operations below are coordinatewise.

Definition 6.22: AdaGrad, RMSProp, Adam

AdaGrad: $\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$, $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta\, \mathbf{g}_t \oslash (\sqrt{\mathbf{r}_t} + \epsilon)$.
RMSProp: $\mathbf{r}_t = \rho \mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t \odot \mathbf{g}_t$, same update.
Adam: $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$, $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t \odot \mathbf{g}_t$, $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$, $\mathbf{x}_t = \mathbf{x}_{t-1} - \eta\, \hat{\mathbf{m}}_t \oslash (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$. Typical $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$.

Remark 6.23: what each piece does

AdaGrad's accumulator grows forever, so its effective rate $\eta/\sqrt{r_t}$ shrinks (for a constant gradient g the step is $\eta/\sqrt{t}$). RMSProp forgets old gradients; for constant $g = 2$, $\rho = 0.9$, $\eta = 0.1$, $r_1 = 0.4$, $r_2 = 0.76$, steps $0.316, 0.229$, tending to 0.1. Adam = momentum's numerator (direction) + RMSProp's denominator $\sqrt{\mathbf{v}}$ (per-coordinate scale) + bias correction. (Nesterov plus Adam is NAdam.)

10.1 Exponential moving averages and bias correction

Theorem 6.24: EMA of iid gradients

If $g_0, g_1, \ldots$ are iid with mean μ and variance s^2, and $m_{k+1} = \beta m_k + (1-\beta) g_k$, $m_0 = 0$, then

$$\mathbb{E} m_k = (1-\beta^k)\mu, \qquad \operatorname{Var} m_k = s^2 \frac{1-\beta}{1+\beta}(1-\beta^{2k}),$$

with limits μ and $s^2 \frac{1-\beta}{1+\beta}$. The corrected $\hat{m}_k = m_k/(1-\beta^k)$ is unbiased with variance

$s^2 \frac{(1-\beta)(1+\beta^k)}{(1+\beta)(1-\beta^k)}$ (equal to s^2 at $k=1$).

Proof. m_k depends only on $g_0, \dots, g_{k-1}$, so it is independent of g_k. Mean by induction: $\beta(1-\beta^k)\mu + (1-\beta)\mu = (1-\beta^{k+1})\mu$. Variance: $\operatorname{Var} m_{k+1} = \beta^2 \operatorname{Var} m_k + (1-\beta)^2 s^2$; writing $(1-\beta)^2 = \frac{1-\beta}{1+\beta}(1-\beta^2)$, the induction step is $\beta^2(1-\beta^{2k}) + (1-\beta^2) = 1-\beta^{2k+2}$. Correction: divide by $(1-\beta^k)^2$ and use $1-\beta^{2k} = (1-\beta^k)(1+\beta^k)$. ■

Without correction Adam's first step is $\frac{1-\beta_1}{\sqrt{1-\beta_2}} \operatorname{sign}(g_1)\eta \approx 3.16\,\eta$; with correction $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2$ and the step is exactly $\eta \operatorname{sign}(g_1)$. The correction removes the zero-initialization bias at the price of a larger early variance.

10.2 Special cases and invariances

Theorem 6.25: Adam contains RMSProp and signSGD

For Adam without bias correction: $\beta_1 = 0$ gives RMSProp with $\rho = \beta_2$; $\beta_1 = \beta_2 = 0$, $\epsilon = 0$ give signSGD $\theta_{k+1} = \theta_k - \eta \operatorname{sign}(g_k)$ (if no coordinate of g_k is 0). With $\epsilon = 0$, Adam's iterates are unchanged when the loss is multiplied by $c > 0$, while SGD's step scales by c.

Proof. $\beta_1 = 0$ makes $m_{k+1} = g_k$. With $\beta_2 = 0$ also, $\sqrt{v_{k+1}} = |g_k|$. Scaling $g \mapsto cg$ scales m by c and $\sqrt{v}$ by c. ■

Chapter 7. Convolutional Neural Networks and Architectures

Convolution, stride, padding, pooling, equivariance; AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

1 Why a New Layer

An image $X \in \mathbb{R}^{H\times W\times C}$ fed to a fully connected layer is flattened, and $z = A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{M\times HWC}$ has $M(HWC+1)$ parameters: for $32\times 32\times 3$ input and $M = 1000$, already 3,073,000. Flattening loses no numbers, but an unrestricted matrix does not encode which pixels are neighbors or that a pattern can appear anywhere. Two assumptions about images motivate convolution:

- **Locality**: nearby pixels jointly form edges, corners and textures, so a unit can look at a small patch (local connectivity).
- **Stationarity**: the same pattern may occur at any position, so the same detector should be used everywhere (weight sharing).

Definition 7.1: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1, j-\tau_2, c}$. A feature extractor F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ (the output moves with the input) and a classifier G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$ (the output does not change). More generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on an image of 0s and 90s a window with r bright pixels outputs $10r$. Averaging suppresses isolated pixels but blurs edges; a 5×5 average smooths more. A CNN *learns* the kernel; learned weights may be negative and need not sum to one.

2.2 Single and multiple channels

Definition 7.2: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b} \in \mathbb{R}^{D}$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

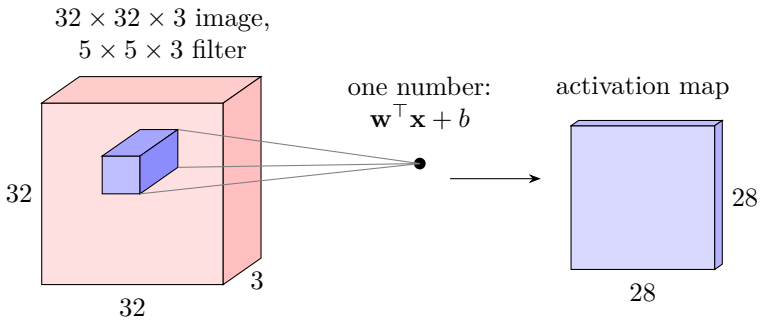
where $\tilde{X}$ is the padded input. Each output is a dot product of a $K\times K\times C$ patch with a filter spanning *all* input channels, plus a bias. The array $Z_{\cdot,\cdot,d}$ is the feature map of filter d; stacking the D maps gives D output channels. This operation is cross-correlation (no kernel flip); since kernels are learned, the distinction does not matter for expressiveness.

Example 7.3: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$, $b = 1$, $S = 1$, $P = 0$: $Z = \begin{pmatrix} 4 & 8 \\ 1 & 1 \end{pmatrix}$. The same four weights and bias are reused at the four positions.

Example 7.4: a 5×5 example

With a 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3 \\ -4 & 7 & 4 \\ 2 & -5 & 1 \end{pmatrix}$ ($b = 0$): unpadded $Z = \begin{pmatrix} 51 & 66 & 20 \\ 31 & 49 & 101 \\ 15 & 53 & -2 \end{pmatrix}$; with $P = 1$ the output is 5×5 and its central 3×3 block equals the unpadded output (those windows contain the same pixels).



A filter spans the full depth of the input; sliding it over all spatial positions gives one 28×28 activation map.

2.3 Parameter counts

One $5 \times 5 \times 3$ filter has 75 weights and one bias; six filters on $32 \times 32 \times 3$ give a $28 \times 28 \times 6$ output with $6(75 + 1) = 456$ parameters. For the same input and output shape:

Layer	Parameters
Fully connected	$4704(3072 + 1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75 + 1) = 357{,}504$
Convolution (shared)	$6(75 + 1) = 456$

Locality removes long-range connections; sharing removes independent copies. As a matrix acting on the flattened input, convolution is sparse with repeated entries (a 1D length-2 filter (a, b) on $(x_1, \ldots, x_4)$ gives a banded matrix with only two free numbers).

3 Stride, Padding and Output Size

Motivation. Designing a CNN requires knowing the shape of every layer's output. A single formula relates input size, kernel size, padding and stride.

Theorem 7.5: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. The padded length is $N + 2P$. A window starting at tS ends at $tS + K - 1$, which must be $\leq N + 2P - 1$, i.e. $t \leq (N + 2P - K)/S$. The admissible integers are $t = 0, \ldots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1 \Rightarrow 5$, $S = 2 \Rightarrow 3$, $S = 3 \Rightarrow 2$ (the last does not "fit" exactly; a border is left unused). For $S = 1$ and odd K, $P = (K - 1)/2$ preserves the size (padding $1, 2, 3$ for $K = 3, 5, 7$). Zero padding is an explicit boundary assumption, not recovered information.

Example 7.6: further examples

Input $52 \times 52 \times 128$, $D = 256$.
(1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$.
(2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$. Parameters, activations (HWD) and multiply–accumulates ($\approx HWDK^2C$) are different costs.

Stacking: $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ with ten $5 \times 5 \times 6$ filters ($10(150 + 1) = 1510$ parameters). Two stacked stride-1 5×5 layers see a 9×9 patch (receptive field $5 + 4$). With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

For an unpadded stride-1 layer with upstream $G_{i,j,d} = \partial L / \partial Z_{i,j,d}$,

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position contributes to the update of the shared weight (the backward counterpart of weight sharing).

5 Shift Equivariance

Motivation. Shifting an image should shift the features, not change them. Convolution has exactly this property, which is the mathematical reason CNNs need far fewer parameters than fully connected networks on images.

Theorem 7.7: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1, j+v-\tau_2, c} = [C_W X]_{i-\tau_1, j-\tau_2, d}$; the coefficients do not depend on the absolute position (i, j). A circular shift permutes positions, and an average is invariant under permutations. ■

Remark 7.8: when exactness fails

With zero padding, content shifted across the border is lost and zeros enter, so equality holds only away from the boundary. A stride-S layer (or pooling) is compatible only with shifts that are multiples of S: for width-2, stride-2 max pooling M, $M(T_2x) = T_1M(x)$ but $M(T_1x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 convolution. At each position $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$ with $A \in \mathbb{R}^{D\times C}$ shared across positions: it mixes channels, not neighbors. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64 + 1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u,jS+v,c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling reduces resolution and gives limited robustness to small shifts within a window. Backward: max pooling routes the gradient to the maximizer; average pooling spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW}\sum_{i,j} X_{i,j,c}$ maps $H \times W \times C \to C$; a classifier on top needs $Q(C + 1)$ instead of $Q(HWC + 1)$ parameters. It discards spatial arrangement entirely.

7 Algebraic Facts about CNN Layers

Motivation. The building blocks of CNNs (convolution, ReLU, pooling) satisfy simple algebraic identities. They tell us when layers can be reordered or merged without changing the network.

Theorem 7.9: positive homogeneity

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each component is positively homogeneous: linear maps for all λ; $\max(0, \lambda t) = \lambda \max(0, t)$ and $\max_i \lambda x_i = \lambda \max_i x_i$ for $\lambda \geq 0$ (scaling by $\lambda > 0$ preserves order and signs; $\lambda = 0$ is trivial); dropout multiplies by a fixed mask. Compositions of positively homogeneous maps are positively homogeneous: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break the property, e.g. $\mathrm{ReLU}(x+1)$ at $x = 0$. ■

Theorem 7.10: conv–MP–ReLU orders coincide

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. For nondecreasing ϕ, $\max_i \phi(x_i) = \phi(\max_i x_i)$: $\phi(x_i) \leq \phi(\max x)$ for all i with equality at the maximizer. ReLU is nondecreasing and elementwise, and max pooling is a windowwise max, so $\mathrm{ReLU} \circ \mathrm{MP} = \mathrm{MP} \circ \mathrm{ReLU}$; finally $\mathrm{ReLU} \circ \mathrm{ReLU} = \mathrm{ReLU}$. (For average pooling only avg ReLU $\geq$ ReLU avg holds, by convexity.) ■

Theorem 7.11: convolutions that implement given maps

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k,l) = (2+\Delta i, 2+\Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'}\delta_{k2}\delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colorwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.

8 Vocabulary and Costs

A network maps an image to class scores and then probabilities, $X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. For a convolution with D filters $K \times K \times C$ producing $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2C + 1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

An FC layer $\mathbb{R}^n \to \mathbb{R}^m$ has $m(n+1)$ parameters. Top-1 error asks whether the highest score is correct; top-5 error asks whether the true label is missing from the five highest scores.

9 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output
Input	RGB	$227 \times 227 \times 3$
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$ $((227-11)/4+1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144 parameters, whereas Conv1 has $96(11^2 \cdot 3 + 1) = 34{,}944$. Design choices:

- **ReLU**: avoids sigmoid/tanh saturation on the positive side.
- **Large first stride**: $227 \to 55$ immediately reduces memory and computation (at the cost of detail).
- **Local response normalization**: $a_{i,j,c} \mapsto a_{i,j,c}/\left(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2\right)^{\beta}$ normalizes across neighboring channels (not batch normalization).
- **Dropout** 0.5 in the FC layers against overfitting.

10 ZFNet and VGG

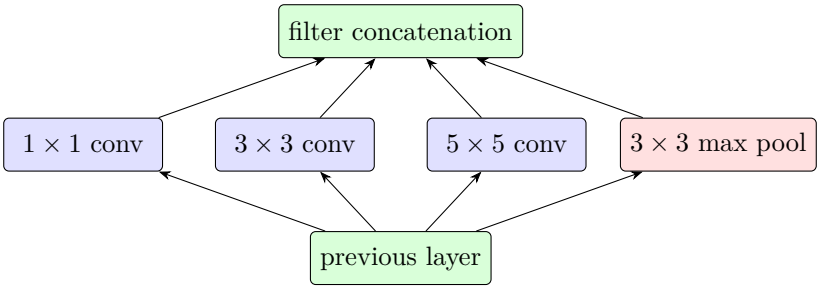
ZFNet tunes AlexNet's kernel sizes and strides. VGG uses deeper stacks of small 3×3 convolutions: two stride-1 3×3 layers have a 5×5 receptive field with $18C^2$ weights (versus $25C^2$), three have 7×7 with $27C^2$ (versus $49C^2$), and interleaved ReLUs add nonlinearity. Receptive fields obey $r_\ell = r_{\ell-1} + (K_\ell - 1)j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

11 GoogLeNet and the Inception Module

Motivation. Which filter size should a layer use? Inception uses several in parallel and concatenates the results, with 1×1 convolutions to keep the cost under control.

11.1 Naive module

For input $28\times28\times256$, parallel branches with padding chosen to keep 28×28: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128+192+96+256=672$ channels. Weights: $256\cdot128+9\cdot256\cdot192+25\cdot256\cdot96=1{,}089{,}536$.



The naive Inception module: parallel convolutions of several sizes and a pooling branch, concatenated along the depth.

11.2 Bottleneck module

Insert 1×1 reductions to 64 channels before the 3×3 and 5×5 convolutions and after pooling: outputs $128+192+96+64=480$ channels and

$$32{,}768+(16{,}384+110{,}592)+(16{,}384+153{,}600)+16{,}384=346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 7.12: when a bottleneck saves weights

A direct $K\times K$ map $C\to D$ costs K^2CD; reducing to R channels first costs $CR+K^2RD$. This is cheaper iff $R<\dfrac{K^2CD}{C+K^2D}$ (for $K=5, C=256, D=96$ the threshold is ≈ 231).

The 1×1 convolution is a channel-mixing matrix applied at every position (see above). GoogLeNet stacks such modules (22 weight layers) and replaces the large FC stack by global average pooling plus a final classifier.

11.3 Split–transform–merge

A $256 \to 128 \to 128 \to 256$ conv–ReLU block (1×1, 3×3 with padding 1, 1×1, all with biases) has $32{,}896 + 147{,}584 + 33{,}024 = 213{,}504$ parameters. Thirty-two parallel $256 \to 4 \to 4 \to 256$ paths summed at the end have $32(1028 + 148 + 1280) = 78{,}592$ parameters; the 3×3 cost drops by the group factor 32. Implementation: three `nn.ModuleLists` of 32 convolutions, ReLU inside each path, then a sum (equivalently grouped convolutions with `groups=32`).

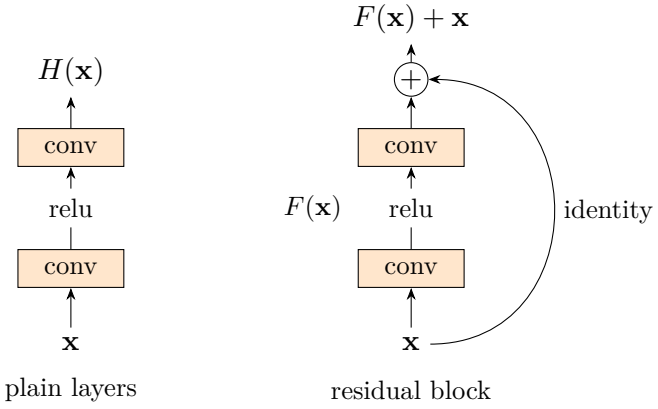
12 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: this is not overfitting but an optimization (degradation) problem. A deeper network that can represent identity layers contains the shallower one, so its *optimal* training loss is not worse, yet gradient methods may not find it. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; if every Jacobian has norm at most $\rho < 1$ the gradient shrinks like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 7.13: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^\top \mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient even when J_F is small, and the block can represent the identity by $F = 0$: the network learns a residual $F = H - \mathrm{id}$ rather than the whole map H. This is a reparametrization, not a guarantee (e.g. $J_F = -\mathbf{I}$ cancels the gradient; a ReLU after the sum alters the identity). If shapes differ, use a projection shortcut $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (e.g. a 1×1 or strided convolution). Unlike Inception's concatenation, residual addition keeps the channel count. A bottleneck block $256 \to 64 \to 64 \to 256$ (1×1, 3×3, 1×1) has $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights versus $18 \cdot 256^2 = 1{,}179{,}648$ for two full-width 3×3 layers. In the CIFAR-10 plots, residual networks remove the training-error degradation seen in plain networks. The full backpropagation formulas for a residual MLP are derived in Chapter 5.



A residual block learns $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$; with $F = 0$ it is the identity, and gradients pass through the shortcut.

13 Saliency Maps

To see which pixels a prediction is sensitive to, differentiate the selected class score with respect to the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor's theorem $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: a large $|G|$ means a small change of that pixel changes the score a lot, and the sign says in which direction. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, the gradient $(2, -3, 0.1)$ ranks x_2 first. Saliency is local sensitivity, not a complete causal explanation.

Chapter 8. Universal Approximation, Barron's Theorem and Generalization

Lectures 10/21, 10/26: measures, normed spaces, Cybenko's theorem; Barron's theorem and proof; Rademacher complexity

1 Three Sources of Error

Let f^* be the ideal target, $\mathcal{F}$ the class of functions a given architecture can represent, f_{app} the best element of $\mathcal{F}$, f_{gen} the best element on the finite training sample (empirical risk minimizer) and f_{opt} what the optimizer actually returns. The total error splits as

$$\underbrace{f^* \to f_{\text{app}}}_{\text{approximation}} + \underbrace{f_{\text{app}} \to f_{\text{gen}}}_{\text{generalization (estimation)}} + \underbrace{f_{\text{gen}} \to f_{\text{opt}}}_{\text{optimization}}.$$

Chapter 6 dealt with optimization; this chapter shows that the approximation error of one-hidden-layer networks can be made arbitrarily small (universality) and, for Barron functions, quantifies it.

2 Preliminaries from Analysis

2.1 Measures

Definition 8.1: measures on a compact set $K \subset \mathbb{R}^n$

The **Borel sets** of K form the smallest collection containing all open subsets of K and closed under complements in K and countable unions. A (nonnegative) **measure** μ assigns $\mu(A) \in [0, \infty]$ to Borel sets with

$$\mu(\emptyset) = 0, \qquad \mu\Big(\bigcup_{j=1}^{\infty} A_j\Big) = \sum_{j=1}^{\infty} \mu(A_j) \quad \text{for pairwise disjoint } A_j.$$

A **signed measure** is the same with real (possibly negative) values, e.g. a difference of two finite measures. $\mu = 0$ means $\mu(A) = 0$ for every Borel set A. $M(I_n)$ denotes the finite signed regular Borel measures on the unit cube $I_n = [0, 1]^n$.

2.2 Normed spaces and density

Definition 8.2: linear and normed spaces

A real **linear space** is a set closed under addition and multiplication by reals obeying the usual vector-space rules; e.g. $C(K) = \{f : K \to \mathbb{R} \text{ continuous}\}$, since $af + bg \in C(K)$ with $(af + bg)(x) = af(x) + bg(x)$. A **normed linear space** is a linear space X with $\|\cdot\| : X \to [0, \infty)$ such that

$$\|u\| = 0 \iff u = 0, \qquad \|cu\| = |c|\|u\|, \qquad \|u + v\| \le \|u\| + \|v\|.$$

On $C(K)$ we use the sup norm $\|f\|_\infty = \max_{x \in K} |f(x)|$. A subset $D \subseteq X$ is **dense** in X if for every $f \in X$ and $\varepsilon > 0$ there is $g \in D$ with $\|f - g\| < \varepsilon$.

Density in $(C(I_n), \|\cdot\|_\infty)$ is exactly uniform approximation: every continuous function can be approximated within ε at every point simultaneously.

3 Cybenko's Universal Approximation Theorem

Motivation. Can neural networks represent the functions we want to learn? Cybenko's theorem says yes: one hidden layer with a sigmoidal activation can approximate any continuous function on a compact set uniformly. The proof is an existence argument from functional analysis (Hahn–Banach and Riesz).

Definition 8.3: sigmoidal and discriminatory

$\sigma : \mathbb{R} \to [0,1]$ is **sigmoidal** if $\lim_{t\to-\infty} \sigma(t) = 0$ and $\lim_{t\to+\infty} \sigma(t) = 1$. For $\mu \in M(I_n)$, σ is **discriminatory** if

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \quad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R} \qquad \Longrightarrow \qquad \mu = 0.$$

Theorem 8.4: Cybenko 1989 (UAT)

Let σ be any continuous sigmoidal function. Then finite sums

$$G(\mathbf{x}) = \sum_{j=1}^{N} \alpha_j\, \sigma(\mathbf{w}_j^\top \mathbf{x} + \theta_j), \qquad \mathbf{w}_j \in \mathbb{R}^n,\ \alpha_j, \theta_j \in \mathbb{R},\ N \in \mathbb{N},$$

are dense in $C(I_n)$: for every continuous f and $\varepsilon > 0$ some G satisfies $\sup_{\mathbf{x}\in I_n} |f(\mathbf{x}) - G(\mathbf{x})| < \varepsilon$.

The proof combines three ingredients: a Hahn–Banach consequence (Theorem 8.5), the Riesz representation theorem (Theorem 8.6), and the discriminatory property (Theorem 8.9).

3.1 The functional-analytic lemmas

Theorem 8.5: a consequence of Hahn–Banach

Let W be a linear, non-dense subspace of a normed linear space X. Then there is a bounded linear functional L on X with $L \neq 0$ on X and $L|_W = 0$.

Proof. The closure $\overline{W}$ is a closed subspace and $\overline{W} \neq X$, so pick $x_0 \notin \overline{W}$; then $d = \inf_{w\in\overline{W}} \|x_0 - w\| > 0$. On $\overline{W} \oplus \mathbb{R}x_0$ define $\ell(w + tx_0) = t$. It is linear, vanishes on W, $\ell(x_0) = 1$, and bounded: $|t|d \le |t|\|x_0 + w/t\| = \|w + tx_0\|$ for $t \neq 0$, so $\|\ell\| \le 1/d$. The Hahn–Banach theorem extends ℓ to X with the same norm. ∎

Theorem 8.6: Riesz representation theorem

Let $K \subset \mathbb{R}^n$ be compact. For every bounded linear functional F on $C(K)$ there is a unique finite signed (regular Borel) measure μ on K with

$$F(f) = \int_K f(\mathbf{x})\, d\mu(\mathbf{x}) \qquad \text{for all } f \in C(K).$$

(For positive functionals the measure is nonnegative.)

Theorem 8.7: an annihilating measure

Let W be a linear, non-dense subspace of $C(I_n)$. Then there is $\mu \in M(I_n)$, $\mu \neq 0$, with $\int_{I_n} h\, d\mu = 0$ for all $h \in W$.

Proof [Supplement]. Theorem 8.5 with $X = C(I_n)$ gives a bounded linear $L \neq 0$ with $L|_W = 0$. Riesz represents $L(f) = \int f\, d\mu$; $\mu \neq 0$ because $L \neq 0$; and for $h \in W$, $\int h\, d\mu = L(h) = 0$. ■

3.2 Density for discriminatory functions

Theorem 8.8: density for discriminatory activations

If σ is continuous and discriminatory, the sums G above are dense in $C(I_n)$.

Proof. Since σ is continuous, $W = \{G\}$ is a linear subspace of $C(I_n)$ (sums and scalar multiples of such sums are again such sums). Suppose W is not dense. By Theorem 8.7 there is $\mu \neq 0$ with $\int h\, d\mu = 0$ for all $h \in W$, in particular

$$\int_{I_n} \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu(\mathbf{x}) = 0 \qquad \text{for all } \mathbf{w} \in \mathbb{R}^n,\ \theta \in \mathbb{R}.$$

Because σ is discriminatory, $\mu = 0$: a contradiction. ■

3.3 Continuous sigmoidals are discriminatory

Theorem 8.9: continuous sigmoidals are discriminatory

Every continuous sigmoidal function is discriminatory for all $\mu \in M(I_n)$.

Proof [Supplement]. Assume $\int \sigma(\mathbf{w}^\top \mathbf{x} + \theta)\, d\mu = 0$ for all $\mathbf{w}, \theta$. Fix $\mathbf{w}, \theta, \varphi$ and let $\sigma_\lambda(\mathbf{x}) = \sigma(\lambda(\mathbf{w}^\top \mathbf{x} + \theta) + \varphi)$. As $\lambda \to \infty$, $\sigma_\lambda \to 1$ on the open half-space $\Pi = \{\mathbf{w}^\top \mathbf{x} + \theta > 0\}$, $\to 0$ on $\{\mathbf{w}^\top \mathbf{x} + \theta < 0\}$ and $= \sigma(\varphi)$ on the hyperplane $H = \{\mathbf{w}^\top \mathbf{x} + \theta = 0\}$. The σ_λ are bounded by 1, so dominated convergence gives

$$0 = \int \sigma_\lambda\, d\mu \ \longrightarrow\ \mu(\Pi) + \sigma(\varphi)\mu(H) \qquad \text{for every } \varphi.$$

Letting $\varphi \to -\infty$ and $\varphi \to +\infty$ yields $\mu(\Pi) = 0$ and $\mu(H) = 0$ for all half-spaces and hyperplanes. Fix $\mathbf{w}$ and define $F(h) = \int h(\mathbf{w}^\top \mathbf{x})\, d\mu(\mathbf{x})$ for bounded measurable $h : \mathbb{R} \to \mathbb{R}$. F is linear and bounded, and $F(\mathbf{1}_{[\theta,\infty)}) = \mu(\Pi) + \mu(H) = 0$, $F(\mathbf{1}_{(\theta,\infty)}) = 0$, hence F vanishes on indicators of intervals, on simple functions, and by uniform approximation on all bounded measurable h. Taking $h = \cos$ and $h = \sin$ gives $\int e^{i\mathbf{w}^\top \mathbf{x}}\, d\mu(\mathbf{x}) = 0$ for every $\mathbf{w}$: the Fourier transform of μ vanishes, so $\mu = 0$. ■

Proof of Cybenko's theorem. Theorem 8.9 shows σ is discriminatory; Theorem 8.8 gives density. ■

Remark 8.10: what the theorem says and does not say

One hidden layer with enough neurons approximates any continuous function on a compact set uniformly (the cube can be replaced by any compact K after rescaling). The theorem is an existence result: it gives no bound on N, no algorithm to find the weights, and no statement about generalization. Later work extends it to every continuous non-polynomial activation,

including ReLU (Leshno et al. 1993). Intuition in 1D: a steep sigmoid is almost a step, a difference of two steps is a bump, and a sum of bumps approximates any continuous function.

4 Barron's Theorem: a Dimension-Free Rate

4.1 Fourier preliminaries

For $f \in L^1(\mathbb{R}^d)$ the Fourier transform is $\hat f(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} f(\mathbf{x}) e^{-2\pi i \boldsymbol{\xi}^\top \mathbf{x}}\, d\mathbf{x}$. The Sobolev space $H^s(\mathbb{R}^d)$ consists of f with $\int |\hat f(\boldsymbol{\xi})|^2 (1+|\boldsymbol{\xi}|^2)^s \, d\boldsymbol{\xi} < \infty$.

Definition 8.11: Barron class

For $C > 0$,

$$\Gamma_C = \Big\{ f \in L^1(\mathbb{R}^d) : \ \|\hat f\|_{L^1(\mathbb{R}^d)} < \infty, \ \int_{\mathbb{R}^d} |2\pi\boldsymbol{\xi}|\, |\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi} < C \Big\}.$$

On a domain Ω one also uses the spectral Barron norm $\|f\|_{\mathcal{B}^s(\Omega)} = \inf_{f_e|_\Omega = f} \int_{\mathbb{R}^d} (1+|\boldsymbol{\xi}|)^s |\hat f_e(\boldsymbol{\xi})|\, d\boldsymbol{\xi}$, the infimum over all extensions f_e of f beyond Ω.

Theorem 8.12 (Theorem 1, Barron 1993)

Let σ be sigmoidal, $f \in \Gamma_C$ and $B_1^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| \le 1\}$. Then for every $c > 4C^2$ and every $N \in \mathbb{N}$ there is a network f_N with N neurons such that

$$\frac{1}{|B_1^d|} \int_{B_1^d} |f(\mathbf{x}) - f_N(\mathbf{x})|^2 \, d\mathbf{x} < \frac{c}{N}.$$

The rate $O(1/N)$ does not depend on the dimension d. The price is the assumption $f \in \Gamma_C$, a condition on the Fourier spectrum.

Remark 8.13: curse of dimensionality

Approximating a general function with s bounded derivatives in d variables to accuracy ε by a fixed linear family needs on the order of $\varepsilon^{-d/s}$ terms, which explodes with d. Barron's bound avoids this: the number of neurons needed for mean squared error ε is about c/ε for every d. The dimension enters only through the constant C.

5 Proof of Barron's Theorem

The proof has three steps. Lemma 1 (Maurey) says that an element of a closed convex hull is approximated at rate $1/N$ by averages of N elements. Lemma 2 places every Barron function in such a hull, namely the hull of scaled half-space indicators. A final step replaces the indicators by sigmoids.

5.1 Convex hulls and Maurey's lemma

Definition 8.14: convex hull

For a subset G of a vector space,

$$\operatorname{co}(G) = \Big\{ \sum_{j=1}^{n} \alpha_j x_j : \ n \in \mathbb{N},\ x_j \in G,\ \alpha_j \geq 0,\ \sum_{j=1}^{n} \alpha_j = 1 \Big\},$$

and $\overline{\operatorname{co}}(G)$ denotes its closure in a given norm.

Theorem 8.15 (Lemma 1, Maurey)

Let H be a Hilbert space and $G \subset H$ with $\|g\|_H \leq B$ for all $g \in G$, for some $B > 0$. Let $f \in \overline{\operatorname{co}}(G)$. Then for every $N \in \mathbb{N}$ and every $c > B^2$ there exist $g_1, \dots, g_N \in G$ with

$$\Big\| f - \frac{1}{N} \sum_{i=1}^{N} g_i \Big\|_H^2 \leq \frac{c}{N}.$$

Proof. Fix N and $\varepsilon > 0$. Since $f \in \overline{\operatorname{co}}(G)$ there are $\alpha_1, \dots, \alpha_m \in [0,1]$ summing to 1 and $h_1, \dots, h_m \in G$ such that

$$f^* = \sum_{j=1}^{m} \alpha_j h_j \quad \text{satisfies} \quad \|f - f^*\|_H < \varepsilon.$$

Claim. There are $g_1, \dots, g_N \in \{h_1, \dots, h_m\}$ with $\big\| f^* - \frac{1}{N} \sum_i g_i \big\|_H^2 \leq B^2/N$.

Let $X_1, \dots, X_N$ be iid random elements with $P(X_i = h_j) = \alpha_j$. Then $\mathbb{E}[X_i] = \sum_j P(X_i = h_j)\, h_j = f^*$. Writing $f^* = \frac{1}{N} \sum_{j=1}^{N} f^*$ and using $\|\sum_j v_j\|^2 = \sum_j \|v_j\|^2 + \sum_{i \neq j} \langle v_i, v_j \rangle$,

$$\mathbb{E}\Big\| f^* - \frac{1}{N} \sum_{j=1}^{N} X_j \Big\|^2 = \frac{1}{N^2}\, \mathbb{E}\Big[\sum_{j=1}^{N} \|f^* - X_j\|^2 + \sum_{i \neq j} \langle f^* - X_i, f^* - X_j \rangle \Big].$$

For $i \neq j$, independence gives $\mathbb{E}\langle X_i, X_j \rangle = \langle \mathbb{E}X_i, \mathbb{E}X_j \rangle = \langle f^*, f^* \rangle$, so

$$\mathbb{E}\langle f^* - X_i, f^* - X_j \rangle = \langle f^*, f^* \rangle - \langle f^*, \mathbb{E}X_j \rangle - \langle \mathbb{E}X_i, f^* \rangle + \mathbb{E}\langle X_i, X_j \rangle = 0.$$

The N diagonal terms have the same distribution, hence

$$\mathbb{E}\Big\| f^* - \frac{1}{N} \sum_{j} X_j \Big\|^2 = \frac{1}{N} \mathbb{E}\|f^* - X_1\|^2 = \frac{1}{N} \mathbb{E}[\|f^*\|^2 - 2\langle f^*, X_1 \rangle + \|X_1\|^2]$$

$$= \frac{1}{N} (\mathbb{E}\|X_1\|^2 - \|f^*\|^2) \leq \frac{B^2}{N},$$

using $\mathbb{E}\langle f^*, X_1 \rangle = \|f^*\|^2$ and $\|X_1\| \leq B$. A random variable cannot exceed its mean for every outcome, so some realization $g_i = X_i(\omega) \in \{h_1, \dots, h_m\}$ satisfies the claim.

By the triangle inequality,

$$\Big\| f - \frac{1}{N} \sum_{i} g_i \Big\|_H \leq \|f - f^*\|_H + \Big\| f^* - \frac{1}{N} \sum_{i} g_i \Big\|_H < \varepsilon + \frac{B}{\sqrt{N}}.$$

Since $c > B^2$, choosing $\varepsilon < (\sqrt{c} - B)/\sqrt{N}$ makes the right-hand side less than $\sqrt{c/N}$; squaring gives the lemma. ∎

This is what Theorem 1 needs: if f lies in the closed convex hull of a set of single neurons bounded by B in the normalized L^2 norm, an average of N neurons approximates it with error c/N. So it suffices to show that every $f \in \Gamma_C$ lies in the closed convex hull of affinely transformed Heaviside functions.

5.2 Barron functions lie in a convex hull of half-space indicators

On the unit ball use the normalized norm

$$\|g\|_{L_0^2(B_1^d)} = \Big(\frac{1}{|B_1^d|}\int_{B_1^d} |g(\mathbf{x})|^2\, d\mathbf{x}\Big)^{1/2},$$

and the set of scaled half-space indicators

$$G_C = \Big\{\mathbf{x}\in B_1^d \mapsto \gamma\, \mathbf{1}_{\mathbb{R}_+}(\langle \mathbf{a},\mathbf{x}\rangle + b) :\ \mathbf{a}\in\mathbb{R}^d,\ b\in\mathbb{R},\ |\gamma|\le 2C\Big\},$$

where $\mathbf{1}_{\mathbb{R}_+}(t) = 1$ for $t \ge 0$ and 0 otherwise (the Heaviside function). Every $g \in G_C$ has $\|g\|_{L_0^2(B_1^d)} \le |\gamma| \le 2C$.

Theorem 8.16 (Lemma 2, Barron 1993)

Let $d \in \mathbb{N}$, $C > 0$ and $f \in \Gamma_C$. Then

$$f|_{B_1^d} - f(\mathbf{0}) \in \overline{\mathrm{co}}(G_C),$$

where the closure is taken in $L_0^2(B_1^d)$.

Proof. Write $K = \overline{\mathrm{co}}(G_C)$ (closure in $L_0^2(B_1^d)$) and $h = f|_{B_1^d} - f(\mathbf{0})$. As a closed convex hull, K is a closed convex subset of the Hilbert space $L_0^2(B_1^d)$. The lecture gave a sketch: the integral representation of Step 1 and the idea that a Riemann sum of it is a convex combination. Parts marked [Supplement] are not in the lecture notes; they fill the gaps of the sketch, following the structure of Barron's original proof.

Step 1 (integral representation). Since $f \in \Gamma_C$, both f and $\hat f$ are in $L^1(\mathbb{R}^d)$, so Fourier inversion holds:

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} \hat f(\boldsymbol{\xi})(e^{2\pi i\langle \mathbf{x},\boldsymbol{\xi}\rangle} - 1)\, d\boldsymbol{\xi}.$$

Write $\hat f(\boldsymbol{\xi}) = |\hat f(\boldsymbol{\xi})| e^{i\kappa(\boldsymbol{\xi})}$, where $\kappa(\boldsymbol{\xi})$ is the phase. Then

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} |\hat f(\boldsymbol{\xi})|(e^{i(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi}))} - e^{i\kappa(\boldsymbol{\xi})})\, d\boldsymbol{\xi}.$$

For real-valued f the left side is real, so we may take real parts:

$$f(\mathbf{x}) - f(\mathbf{0}) = \int_{\mathbb{R}^d} \frac{\cos\big(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi})\big) - \cos\kappa(\boldsymbol{\xi})}{|2\pi\boldsymbol{\xi}|}\, |2\pi\boldsymbol{\xi}|\, |\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi}.$$

At $\boldsymbol{\xi} = \mathbf{0}$ the original integrand vanishes, so the division is harmless. Let $C_f = \int |2\pi\boldsymbol{\xi}||\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi} < C$ and define the probability measure Λ on $\mathbb{R}^d$ by $d\Lambda(\boldsymbol{\xi}) = C_f^{-1}|2\pi\boldsymbol{\xi}||\hat f(\boldsymbol{\xi})|\, d\boldsymbol{\xi}$ (the lecture writes C for C_f; if $C_f = 0$ then $\hat f = 0$ a.e. off the origin, $h = 0 \in K$ and there is nothing to prove). With

$$q_{\boldsymbol{\xi}}(\mathbf{x}) = \frac{\cos\big(2\pi\langle \mathbf{x},\boldsymbol{\xi}\rangle + \kappa(\boldsymbol{\xi})\big) - \cos\kappa(\boldsymbol{\xi})}{|2\pi\boldsymbol{\xi}|}, \qquad k_{\boldsymbol{\xi}} = C_f q_{\boldsymbol{\xi}},$$

we obtain

$$f(\mathbf{x}) - f(\mathbf{0}) = C_f \int_{\mathbb{R}^d} q_{\boldsymbol{\xi}}(\mathbf{x})\, d\Lambda(\boldsymbol{\xi}) = \int_{\mathbb{R}^d} k_{\boldsymbol{\xi}}(\mathbf{x})\, d\Lambda(\boldsymbol{\xi}). \tag{5}$$

Step 2 (each $k_{\boldsymbol{\xi}}$ lies in K). [Supplement]

For $\boldsymbol{\xi} = \mathbf{0}$ put $k_{\mathbf{0}} = 0$, which is the element of G_C with $\gamma = 0$. Fix $\boldsymbol{\xi} \ne \mathbf{0}$ and set $\mathbf{u} = \boldsymbol{\xi}/|\boldsymbol{\xi}|$ and $t = \langle \mathbf{u},\mathbf{x}\rangle$. By Cauchy–Schwarz, $t \in [-1,1]$ on the ball. Then $k_{\boldsymbol{\xi}}(\mathbf{x}) = r(t)$ with

$$r(t) = C_f \frac{\cos\big(2\pi|\boldsymbol{\xi}|t + \kappa\big) - \cos\kappa}{2\pi|\boldsymbol{\xi}|}, \qquad r(0) = 0, \qquad |r'(t)| = C_f|\sin\big(2\pi|\boldsymbol{\xi}|t + \kappa\big)| \le C_f,$$

so $|r(s) - r(t)| \le C_f|s-t|$ by the mean value theorem.

Staircase. For $m \in \mathbb{N}$ let $t_k = k/m$ $(k = 0, \dots, m)$ and

$$s_m(t) = \sum_{k=1}^{m} \alpha_k \mathbf{1}_{\mathbb{R}_+}(t - t_k) + \sum_{k=1}^{m} \alpha'_k \mathbf{1}_{\mathbb{R}_+}(-t - t_k), \qquad \alpha_k = r(t_k) - r(t_{k-1}), \;\; \alpha'_k = r(-t_k) - r(-t_{k-1}).$$

If $t \in [t_j, t_{j+1})$ with $t \ge 0$, every term of the second sum is 0 (its threshold t_k is positive), and the first sum telescopes to $r(t_j) - r(0) = r(t_j)$. Symmetrically, for $t < 0$ the first sum vanishes and the second gives $r(-t_j)$ for $-t \in [t_j, t_{j+1})$. In both cases t is within $1/m$ of the grid point used, so

$$\sup_{t \in [-1,1]} |r(t) - s_m(t)| \le \frac{C_f}{m}.$$

Coefficients. Each $|\alpha_k|, |\alpha'_k| \le C_f/m$, so $S := \sum_k (|\alpha_k| + |\alpha'_k|) \le 2C_f \le 2C$.

Convex combination. In terms of $\mathbf{x}$, $\mathbf{1}_{\mathbb{R}_+}(t - t_k) = \mathbf{1}_{\mathbb{R}_+}(\langle \mathbf{u}, \mathbf{x}\rangle - t_k)$ and $\mathbf{1}_{\mathbb{R}_+}(-t - t_k) = \mathbf{1}_{\mathbb{R}_+}(\langle -\mathbf{u}, \mathbf{x}\rangle - t_k)$ are half-space indicators. List the $2m$ coefficients as β_j and the matching indicators as e_j. If $S = 0$ then $s_m = 0 \in G_C$. Otherwise

$$s_m(\langle \mathbf{u}, \cdot \rangle) = \sum_j \frac{|\beta_j|}{S} \Big(S \, \operatorname{sign}(\beta_j) \, e_j \Big),$$

with weights $|\beta_j|/S \ge 0$ summing to 1 and each bracket in G_C because $S \le 2C$. So $s_m(\langle \mathbf{u}, \cdot \rangle) \in \operatorname{co}(G_C)$.

Limit. The normalized norm is bounded by the sup norm, $\|g\|_{L_0^2(B_1^d)} \le \sup_{B_1^d} |g|$, because the normalized norm averages $|g|^2$ over the ball against the uniform probability measure. Hence $\|k_{\boldsymbol{\xi}} - s_m(\langle \mathbf{u}, \cdot \rangle)\|_{L_0^2} \le C_f/m \to 0$, and $k_{\boldsymbol{\xi}} \in K$.

Step 3 (the average stays in K). The lecture expresses this step by a Riemann sum: partition $\mathbb{R}^d$ into cubes $I_{\boldsymbol{\theta}}$ ($\boldsymbol{\theta} \in \frac{1}{n}\mathbb{Z}^d$); then $\sum_{\boldsymbol{\theta}} k_{\boldsymbol{\theta}} \Lambda(I_{\boldsymbol{\theta}})$ is a convex combination of elements of K approximating the integral (5). Making this rigorous needs a truncation of the infinite sum and continuity of $\boldsymbol{\xi} \mapsto k_{\boldsymbol{\xi}}$, which can fail because the phase κ may jump where $\hat{f} = 0$. The following argument avoids both issues. [Supplement]

Projection. K is nonempty, closed and convex in a Hilbert space, so h has a nearest point $p \in K$, characterized by

$$\langle h - p,\, k - p \rangle \le 0 \qquad \text{for all } k \in K.$$

Averaging. Since Λ is a probability measure, (5) gives $h(\mathbf{x}) - p(\mathbf{x}) = \int \big(k_{\boldsymbol{\xi}}(\mathbf{x}) - p(\mathbf{x})\big)\, d\Lambda(\boldsymbol{\xi})$. Take the inner product with $h - p$ and exchange the integrals over B_1^d and over Λ by Fubini; this is allowed because $|k_{\boldsymbol{\xi}}| \le C_f$ (Step 2, since $|r(t)| \le C_f|t| \le C_f$) and L^2 functions on the ball are integrable. With Step 2,

$$\|h - p\|_{L_0^2}^2 = \int_{\mathbb{R}^d} \langle h - p,\, k_{\boldsymbol{\xi}} - p \rangle \, d\Lambda(\boldsymbol{\xi}) \le 0.$$

Hence $h = p \in K$. ∎

5.3 Completing the proof of Theorem 1

Proof of Barron's theorem. Let $f \in \Gamma_C$. By Lemma 8.16, $f|_{B_1^d} - f(\mathbf{0}) \in \overline{\operatorname{co}}(G_C)$, and every $g \in G_C$ has $\|g\|_{L_0^2(B_1^d)} \le 2C$. Lemma 8.15 with $H = L_0^2(B_1^d)$ and $B = 2C$ gives, for every $N \in \mathbb{N}$ and $c > 4C^2$, parameters $|\gamma_i| \le 2C$, $\mathbf{a}_i$, b_i with

$$\frac{1}{|B_1^d|} \int_{B_1^d} \Big| f(\mathbf{x}) - f(\mathbf{0}) - \frac{1}{N} \sum_{i=1}^{N} \gamma_i \mathbf{1}_{\mathbb{R}_+}\big(\langle \mathbf{a}_i, \mathbf{x} \rangle + b_i\big) \Big|^2 d\mathbf{x} < \frac{c}{N}.$$

Finally a sigmoid becomes a step under scaling: $\sigma(\lambda t) \to \mathbf{1}_{\mathbb{R}_+}(t)$ as $\lambda \to \infty$ for $t \neq 0$, and the hyperplanes $\{\langle \mathbf{a}_i, \mathbf{x}\rangle + b_i = 0\}$ have measure zero. (A hyperplane is a null set only when $\mathbf{a}_i \neq \mathbf{0}$. If $\mathbf{a}_i = \mathbf{0}$ and $b_i = 0$ the set is all of $\mathbb{R}^d$; that term is the constant γ_i, which is also obtained with $\mathbf{a}_i = \mathbf{0}$, $b_i = 1$, so we may assume this case does not occur.) [Supplement]By dominated convergence, replacing each indicator by $\sigma\big(\lambda(\langle \mathbf{a}_i, \mathbf{x}\rangle + b_i)\big)$ with λ large keeps the error below c/N. The result

$$f_N(\mathbf{x}) = f(\mathbf{0}) + \sum_{i=1}^{N} \frac{\gamma_i}{N}\, \sigma\big(\lambda\langle \mathbf{a}_i, \mathbf{x}\rangle + \lambda b_i\big)$$

is a one-hidden-layer network with N neurons; the constant $f(\mathbf{0})$ is the output bias. ∎

6 Generalization and Rademacher Complexity

6.1 Population and empirical risk

Let $\mathbf{x} \in \mathbb{R}^d$ be the input, $y \in \mathcal{Y}$ the output, and $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ iid training data from a distribution P. For a loss ℓ and a model class $\mathcal{F}$ we want

$$\arg\min_{f \in \mathcal{F}} \ \mathbb{E}_{(\mathbf{x},y)\sim P}\big[\ell(y, f(\mathbf{x}))\big] \quad \text{(population risk)},$$

but can only compute

$$\arg\min_{f \in \mathcal{F}} \ \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f(\mathbf{x}_i)) \quad \text{(empirical risk)}.$$

The generalization error of Section 1 is controlled by how far empirical averages can be from expectations uniformly over the class, $\sup_{f\in\mathcal{F}} \big|\frac{1}{n}\sum_i f(\mathbf{z}_i) - \mathbb{E} f\big|$, applied to the class of losses $\mathbf{z} = (\mathbf{x}, y) \mapsto \ell(y, f(\mathbf{x}))$.

6.2 Rademacher complexity

Definition 8.17: Rademacher variables and complexity

Random variables $\xi_1, \dots, \xi_n$ are **Rademacher** if they are iid with $P(\xi_i = 1) = P(\xi_i = -1) = \frac{1}{2}$. For a class $\mathcal{F}$ and iid samples $\mathbf{z}_1, \dots, \mathbf{z}_n$, the **Rademacher complexity** is

$$\mathrm{Rad}_n(\mathcal{F}) = \mathbb{E}\Big[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \xi_i f(\mathbf{z}_i)\Big],$$

the expectation being over both the ξ_i and the $\mathbf{z}_i$.

It measures how well the class can correlate with random signs: a rich class fits noise and has large complexity.

Theorem 8.18: uniform deviation bound

By symmetrization, $\mathbb{E}\sup_{f\in\mathcal{F}} \big|\frac{1}{n}\sum_i f(\mathbf{z}_i) - \mathbb{E} f\big| \le 2\,\mathrm{Rad}_n(\mathcal{F} \cup -\mathcal{F})$. If moreover every $f \in \mathcal{F}$ takes values in $[0, 1]$, then with probability at least $1 - \delta$

$$\sup_{f\in\mathcal{F}} \Big|\frac{1}{n}\sum_{i=1}^{n} f(\mathbf{z}_i) - \mathbb{E} f\Big| \le 2\,\mathrm{Rad}_n(\mathcal{F} \cup -\mathcal{F}) + \sqrt{\frac{\log(2/\delta)}{2n}}.$$

The lecture records this as "$\le 2\,\mathrm{Rad}(\mathcal{F}) + 1/\sqrt{2n}$", suppressing the confidence factor.

6.3 Two-layer networks

For $\mathcal{F} = \{f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_j \beta_j \sigma(\mathbf{w}_j^\top \mathbf{x})\}$ with $\sum_j |\beta_j| \le Q$, $\|\mathbf{w}_j\| \le W$, inputs $\|\mathbf{x}\| \le R$ and σ 1-Lipschitz with $\sigma(0) = 0$ (e.g. tanh), the contraction lemma gives

$$\mathrm{Rad}_n(\mathcal{F}) \le \frac{2QWR}{\sqrt{n}} = O\Big(\frac{1}{\sqrt{n}}\Big).$$

The bound depends on the size Q of the outer weights, not on the number of neurons. Together with Barron's theorem, the approximation error decays like $1/N$ and the generalization error like $1/\sqrt{n}$, and neither rate depends on d.

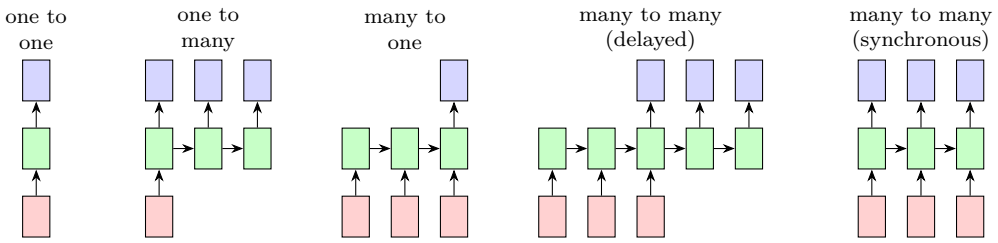
Chapter 9. Recurrent Neural Networks and LSTM

Lectures 10/28, 11/2: RNNs, backpropagation through time, vanishing and exploding gradients, LSTM, language modeling, image captioning (naive)

1 Why Recurrent Networks

A feedforward network maps one fixed-size input to one fixed-size output. Many tasks instead consume or produce **sequences**, often of variable length. A recurrent network handles them by reading one element at a time and carrying an **internal (hidden) state** that is updated as the sequence is processed. The input/output patterns seen in lecture:

Pattern	Input → output	Examples
one to one	single → single	vanilla NN / FFN
one to many	single → sequence	image to video, image captioning
many to one	sequence → single	video classification, action recognition
many to many (delayed)	read all, then emit	translation (English to Korean), video to caption
many to many (synchronous)	emit at every step	video segmentation, video editing, style transfer



Inputs (red), hidden states (green) and outputs (blue); horizontal arrows carry the state.

In the delayed pattern the outputs start only after the whole input has been read (e.g. inputs at $t = 1, 2, 3$, outputs at $t = 4, 5, 6$).

2 The Recurrence

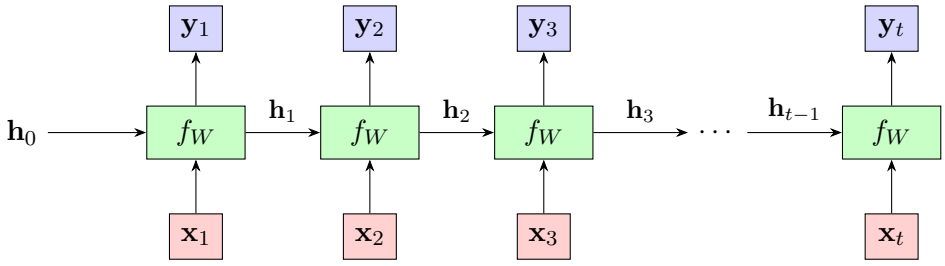
Definition 9.1: recurrent neural network

Given inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots$, an RNN computes hidden states and outputs by

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t), \qquad \mathbf{y}_t = f_{W_{hy}}(\mathbf{h}_t), \qquad t \geq 1,$$

starting from an initial state $\mathbf{h}_0$ (usually $\mathbf{h}_0 = \mathbf{0}$). The functions f_W (new state from old state and current input) and $f_{W_{hy}}$ (output from state) and their parameters are **the same at every time step**.

Unrolling the loop over time gives an ordinary computational graph with one copy of the cell per time step, all copies sharing parameters. Weight sharing is what lets one network process sequences of any length.



The unrolled RNN: every copy of the cell uses the same f_W, and the output map $f_{W_{hy}}$ is shared as well.

Definition 9.2: vanilla RNN

With an elementwise activation σ (typically tanh),

$$\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}), \qquad \hat{\mathbf{y}}_t = W^{\text{out}}\mathbf{h}_t, \qquad \mathbf{h}_0 = \mathbf{0}.$$

Written out for the first two steps:

$$\mathbf{h}_1 = \sigma(W\mathbf{x}_1 + \mathbf{b}), \quad \hat{\mathbf{y}}_1 = W^{\text{out}}\mathbf{h}_1; \qquad \mathbf{h}_2 = \sigma(U\mathbf{h}_1 + W\mathbf{x}_2 + \mathbf{b}), \quad \hat{\mathbf{y}}_2 = W^{\text{out}}\mathbf{h}_2.$$

The same U, W, $\mathbf{b}$, W^{out} appear at both steps. The letters for the two weight matrices are not standardized: the next section (following the handwritten lecture notes) writes W for the recurrent weight and U for the input weight.

3 Backpropagation Through Time

3.1 The idea

The loss of a sequence is a sum of per-step losses, $L = \sum_{t=1}^{\tau} L_t$. Each L_t depends on the parameters through the current step and, via the hidden state, through every earlier step. **Backpropagation through time (BPTT)** is ordinary backpropagation on the unrolled graph. One forward pass runs through the entire sequence to compute L; one backward pass runs through the entire sequence to compute the gradient. Because parameters are shared, the gradient with respect to a parameter is the sum of the contributions from all of its copies.

Remark 9.3: truncated BPTT

For long sequences the full backward pass is expensive. **Truncated BPTT** carries the hidden state forward through the whole sequence but backpropagates only through a window of the most recent steps. Gradients from beyond the window are ignored.

3.2 A worked two-step example

Consider the scalar RNN

$$h_t = \tanh(Wh_{t-1} + Ux_t + b), \qquad Y_t = Vh_t + c, \qquad \theta = (W, U, V, b, c),$$

with inputs $(x_1, \dots, x_\tau)$, outputs $(Y_1, \dots, Y_\tau)$, targets $(Z_1, \dots, Z_\tau)$ and loss $L = \sum_t L_t$ with $L_t = \frac{1}{2}(Y_t - Z_t)^2$. To minimize L we need $\nabla_\theta L$. The chain rule now acts both within each step and across time, which is what makes this harder than for an MLP. Take $\tau = 2$ and all quantities one-dimensional. The forward pass is

$$\begin{aligned} a_1 &= Wh_0 + Ux_1 + b, \quad h_1 = \tanh a_1, \quad Y_1 = Vh_1 + c, \\ a_2 &= Wh_1 + Ux_2 + b, \quad h_2 = \tanh a_2, \quad Y_2 = Vh_2 + c, \end{aligned} \qquad L = \tfrac{1}{2}(Y_1 - Z_1)^2 + \tfrac{1}{2}(Y_2 - Z_2)^2.$$

Local derivatives. Using $\tanh'(a) = \operatorname{sech}^2 a = 1 - \tanh^2 a$, and writing ∂ for the derivative with respect to the explicit occurrence (other arguments held fixed):

$$\frac{\partial h_t}{\partial W} = (1 - h_t^2)h_{t-1}, \qquad \frac{\partial h_t}{\partial U} = (1 - h_t^2)x_t, \qquad \frac{\partial h_t}{\partial b} = 1 - h_t^2, \qquad \frac{\partial h_2}{\partial h_1} = (1 - h_2^2)W.$$

Since L_t depends on h_t only through Y_t,

$$\frac{\partial L_t}{\partial h_t} = \frac{\partial L_t}{\partial Y_t}\frac{\partial Y_t}{\partial h_t} = (Y_t - Z_t)V.$$

Output parameters. L_t depends on V and c only through Y_t, so

$$\frac{\partial L}{\partial V} = \sum_{t=1}^{2}(Y_t - Z_t)h_t, \qquad \frac{\partial L}{\partial c} = \sum_{t=1}^{2}(Y_t - Z_t).$$

Recurrent weight. L_1 sees W through h_1. L_2 sees W directly through h_2 and again through h_1, which feeds h_2:

$$\frac{\partial L}{\partial W} = \frac{\partial L_1}{\partial h_1}\frac{\partial h_1}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial W} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial W},$$

that is,

$$\frac{\partial L}{\partial W} = (Y_1 - Z_1)V(1 - h_1^2)h_0 + (Y_2 - Z_2)V(1 - h_2^2)h_1 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)h_0.$$

Input weight and bias [Supplement]. They have the same three-path structure, with h_{t-1} replaced by x_t and by 1 respectively:

$$\frac{\partial L}{\partial U} = (Y_1 - Z_1)V(1 - h_1^2)x_1 + (Y_2 - Z_2)V(1 - h_2^2)x_2 + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2)x_1,$$
$$\frac{\partial L}{\partial b} = (Y_1 - Z_1)V(1 - h_1^2) + (Y_2 - Z_2)V(1 - h_2^2) + (Y_2 - Z_2)V(1 - h_2^2)W(1 - h_1^2).$$

With $h_0 = 0$ the first and third terms of $\partial L/\partial W$ vanish, but the other two gradients keep all three terms.

3.3 General formulation

Let the input be a sequence $\tau(X) = (u_1, \ldots, u_L)$ with $u_\ell \in \mathbb{R}^m$. The state update is a map $q_{\tilde\theta} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$, e.g. $q_{\tilde\theta}(h, u) = \phi(Wh + Uu + c)$ with $\tilde\theta = (W, U, c) \in \mathbb{R}^p$:

$$h_0 = \mathbf{0}, \qquad h_\ell = q_{\tilde\theta}(h_{\ell-1}, u_\ell) \quad (\ell = 1, \ldots, L), \qquad f_\theta(X) = Ah_L + b,$$

with $A \in \mathbb{R}^{1\times n}$, $b \in \mathbb{R}$, $h_\ell \in \mathbb{R}^n$ and $\theta = (A, b, \tilde\theta)$. For a loss $\mathcal{L}(f_\theta(X), Y)$,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial f}(f_\theta(X), Y)\,\frac{\partial f_\theta(X)}{\partial \theta}, \qquad \frac{\partial f_\theta(X)}{\partial A} = h_L^\top, \quad \frac{\partial f_\theta(X)}{\partial b} = 1, \quad \frac{\partial f_\theta(X)}{\partial \tilde\theta} = A\frac{dh_L}{d\tilde\theta}.$$

Here $dh_L/d\tilde\theta$ is the total derivative, while $\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell)$ below holds $h_{\ell-1}$ fixed. Write $J_k = \partial_h q_{\tilde\theta}(h_{k-1}, u_k) \in \mathbb{R}^{n\times n}$ for the state-to-state Jacobian.

Theorem 9.4: BPTT formula

For $\ell \le L$,

$$\frac{\partial h_L}{\partial h_\ell} = J_L J_{L-1} \cdots J_{\ell+1}, \qquad \frac{dh_L}{d\tilde\theta} = \sum_{\ell=1}^{L}\frac{\partial h_L}{\partial h_\ell}\,\partial_{\tilde\theta} q_{\tilde\theta}(h_{\ell-1}, u_\ell),$$

(the empty product for $\ell = L$ is the identity). Consequently

$$\frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial \tilde{\theta}} = \sum_{\ell=1}^{L} \frac{\partial \mathcal{L}(f_\theta(X), Y)}{\partial h_\ell} \, \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell), \qquad \frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} \frac{\partial h_L}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} J_L \cdots J_{\ell+1}.$$

Proof. Step 1 (state Jacobian). For $k > \ell$, h_k depends on h_ℓ only through h_{k-1}, so $\partial h_k / \partial h_\ell = J_k \, \partial h_{k-1} / \partial h_\ell$. Starting from $\partial h_\ell / \partial h_\ell = I$ and iterating gives the product.

Step 2 (one-step recursion). h_ℓ depends on $\tilde{\theta}$ directly and through $h_{\ell-1}$, so the chain rule gives

$$\frac{dh_\ell}{d\tilde{\theta}} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell) + J_\ell \frac{dh_{\ell-1}}{d\tilde{\theta}}, \qquad \frac{dh_0}{d\tilde{\theta}} = 0.$$

Step 3 (unrolling). Induction on L. For $L = 1$ the formula reads $dh_1 / d\tilde{\theta} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_0, u_1)$, which is Step 2. If the formula holds for $L - 1$, Step 2 and Step 1 give

$$\frac{dh_L}{d\tilde{\theta}} = \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{L-1}, u_L) + \sum_{\ell=1}^{L-1} J_L \frac{\partial h_{L-1}}{\partial h_\ell} \, \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell) = \sum_{\ell=1}^{L} \frac{\partial h_L}{\partial h_\ell} \, \partial_{\tilde{\theta}} q_{\tilde{\theta}}(h_{\ell-1}, u_\ell).$$

Step 4 (loss). $\mathcal{L}$ depends on $\tilde{\theta}$ only through h_L. Multiplying by $\partial \mathcal{L} / \partial h_L$ and using $\partial \mathcal{L} / \partial h_\ell = (\partial \mathcal{L} / \partial h_L)(\partial h_L / \partial h_\ell)$ gives the last two identities. ■

The sum over ℓ is the parameter-sharing sum: each time step contributes the local derivative of its own copy of q, weighted by how strongly h_ℓ influences the loss.

4 Vanishing and Exploding Gradients

4.1 RNNs are very deep networks

Sequence lengths of hundreds or thousands are common, so the factor $\partial h_L / \partial h_\ell = J_L \cdots J_{\ell+1}$ in Theorem 9.4 is a product of up to $L - \ell$ Jacobians. Multiplying many numbers is unstable. If most factors exceed 1 the product blows up (**exploding gradients**). If most are below 1 it tends to 0 (**vanishing gradients**, the bigger problem). Only factors close to 1 give a product of reasonable size. For the scalar tanh RNN of Section 3.2 each factor is $\partial h_t / \partial h_{t-1} = (1 - h_t^2) W$ [Supplement].

Theorem 9.5: a sufficient condition for vanishing gradients [Supplement]

Let $q_{\tilde{\theta}}(h, u) = \tanh(Wh + Uu + c)$ and let $\|\cdot\|$ be the spectral norm. Then

$$\left\| \frac{\partial h_L}{\partial h_\ell} \right\| \le \|W\|^{L-\ell}.$$

In particular, if $\|W\| < 1$ the gradient signal from step L to step ℓ decays geometrically in $L - \ell$.

Proof. Step 1. With $a_k = W h_{k-1} + U u_k + c$, the Jacobian is $J_k = \operatorname{diag}(1 - \tanh^2(a_k)) \, W = \operatorname{diag}(1 - h_k^2) \, W$.

Step 2. The diagonal entries lie in $[0, 1]$, so $\|\operatorname{diag}(1 - h_k^2)\| \le 1$ and $\|J_k\| \le \|W\|$.

Step 3. Submultiplicativity of the spectral norm and Theorem 9.4 give $\|J_L \cdots J_{\ell+1}\| \le \prod_{k=\ell+1}^{L} \|J_k\| \le \|W\|^{L-\ell}$. ■

The converse fails: $\|W\| > 1$ does not force explosion, because saturated units ($h_k^2 \approx 1$) shrink the factors. Exploding gradients need $\|W\|$ large and units away from saturation. [Supplement]

4.2 Exploding gradients and gradient clipping

A very large gradient means the output is very sensitive to small parameter changes in some direction. Sometimes such gradients are unworkable and the architecture must be changed. Sometimes, however, the direction is fine and only the length is the problem. Then one uses **gradient clipping** with a scalar threshold $v > 0$:

$$g \leftarrow \min\left(1, \frac{v}{\|g\|}\right) g = \begin{cases} g, & \|g\| \le v, \\ \dfrac{v}{\|g\|}\, g, & \text{otherwise.} \end{cases}$$

The clipped vector has the same direction as g and length $\min(\|g\|, v)$. The slide writes max in the first expression; the case split shows that min is meant. On a loss surface with a steep cliff, an unclipped step can jump far away from the valley, while a clipped step stays close.

4.3 Vanishing gradients

In an RNN $\partial \mathcal{L}/\partial h_L$ may be of reasonable size while

$$\frac{\partial \mathcal{L}}{\partial h_\ell} = \frac{\partial \mathcal{L}}{\partial h_L} \frac{\partial h_L}{\partial h_{L-1}} \frac{\partial h_{L-1}}{\partial h_{L-2}} \cdots \frac{\partial h_{\ell+1}}{\partial h_\ell}$$

is tiny. Then small changes of h_ℓ barely affect $\mathcal{L}$. Since

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = \frac{\partial \mathcal{L}}{\partial h_\ell} \frac{\partial h_\ell}{\partial u_\ell},$$

small changes of the input u_ℓ barely affect $\mathcal{L}$ either: loosely, the network does not remember u_ℓ at step L. (This is not exact, since large changes of u_ℓ may still matter.) In any case the gradient signal from time L is lost by time ℓ, so the model cannot learn which information to keep at time ℓ.

5 Long Short-Term Memory

Motivation. Plain RNNs cannot learn long-range dependencies because gradients vanish over many time steps. The LSTM adds a memory cell updated additively through gates, so gradients can flow across long time spans.

5.1 The LSTM cell

The vanilla RNN computes $h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$. The LSTM keeps a second state, the **cell state** c_t (long memory), next to the hidden state h_t, and controls it with gates.

Definition 9.6: LSTM

With σ the logistic sigmoid and $\odot$ elementwise multiplication,

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

The four gates have the following roles.

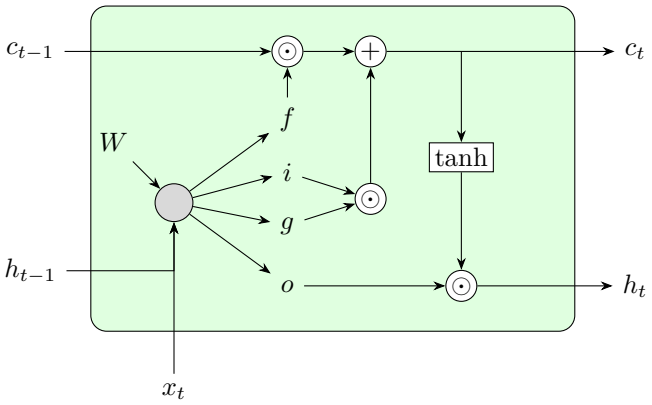
- Input gate i_t: how much of the new content g_t to write into the cell.

- Forget gate f_t: how much of the old cell c_{t-1} to carry over.
- Output gate o_t: how much of the cell to reveal in h_t.
- Candidate ("gate gate") g_t: the new content computed from the current input.

Stacking the four pre-activations gives the compact form

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix},$$

with one matrix W of size $4h \times 2h$ when the hidden state and the input both have dimension h (in general $4h \times (h+m)$ for input dimension m). Its output has length $4h$ and is split into four blocks of length h. Another common notation concatenates $[h_{t-1}, x_t]$, writes $\tilde{C}_t$ for g_t and $*$ for $\odot$.



The LSTM cell: W acts on the stacked (h_{t-1}, x_t); the cell state runs along the top.

5.2 Gradient flow through the cell

Theorem 9.7: the cell path

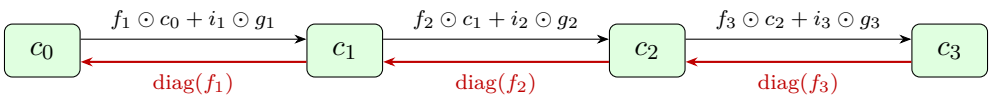
Holding the gates fixed, $\dfrac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t)$. Along the path that goes only through cell states,

$$\frac{\partial c_L}{\partial c_\ell} = \operatorname{diag}\Big(\prod_{k=\ell+1}^{L} f_k\Big) \qquad \text{(product taken entrywise)}.$$

Proof. Step 1. In $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ the second term does not contain c_{t-1}, and the first is the diagonal linear map $c_{t-1} \mapsto \operatorname{diag}(f_t) c_{t-1}$.

Step 2. Chaining these Jacobians and multiplying diagonal matrices entrywise gives the product formula. ■

Compare the vanilla RNN, where each factor is $\operatorname{diag}(1 - h_t^2)W$: the same matrix W is multiplied in at every step and the tanh derivative can only shrink it. In the LSTM the factor is the forget gate itself, chosen by the network at each step, with no repeated matrix product and no squashing derivative. If $f_t \approx 1$ the gradient passes through many steps almost unchanged ("uninterrupted gradient flow"). For example, with $f = 1$ and $i = 0$ the content of the cell is preserved indefinitely. For a vanilla RNN it is much harder to learn a recurrent matrix W that preserves information in the hidden state.



Forward (black) and backward (red) along the cell states: the backward signal is only multiplied by forget gates.

Remark 9.8: what the LSTM does not guarantee

The full Jacobian $\partial c_L / \partial c_\ell$ also has paths through h_{t-1} and the gates, and f_t may be far from 1. So the LSTM does not rule out vanishing or exploding gradients. It gives the model an easier way to learn long-range dependencies.

6 Applications

The lecture covered two applications: language modeling with RNNs, and image captioning. As a further example it mentioned SIRD epidemic models. There the dynamics are an ODE whose infection rate α changes over time, and an LSTM can learn the sequence of α from data.

6.1 Language modeling

A sentence is a sequence of discrete symbols (tokens). Sentence generation, machine translation and text summarization are all instances of modeling language with a network f_θ.

Definition 9.9: one-hot encoding (tokenization)

With a vocabulary of n tokens, token t is represented by $\mathbf{x}_t \in \mathbb{R}^n$ having a single 1 at the index of that token and zeros elsewhere.

Definition 9.10: RNN language model

Sentence generation is next-token prediction. At step t:

$$\begin{aligned} \mathbf{z}_t &= W_e \mathbf{x}_t + \mathbf{b}_e, & W_e &\in \mathbb{R}^{d \times n}, \\ \mathbf{h}_t &= \mathrm{LSTM}(\mathbf{z}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}; \theta), & \mathbf{h}_t &\in \mathbb{R}^d, \\ \hat{\mathbf{x}}_{t+1} &= W_d \mathbf{h}_t + \mathbf{b}_d, & W_d &\in \mathbb{R}^{n \times d}. \end{aligned}$$

The three lines are the continuous word embedding, the recurrent network (whose state is the continuous output embedding), and the classification layer giving scores over the vocabulary. Generation starts from the start-of-sentence token `<SOS>` as $\mathbf{x}_0$ and ends at the end-of-sentence token `<EOS>`.

Training. The model reads the true sentence shifted by one, `<SOS>`, $\mathbf{x}_1, \ldots, \mathbf{x}_{T-1}$, and at each step predicts the next token. The loss is a word-level classification loss,

$$\sum_{t=1}^{T} \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t), \qquad \mathcal{L}(\hat{\mathbf{x}}_t, \mathbf{x}_t) = -\log \mathrm{softmax}(\hat{\mathbf{x}}_t)_{k_t},$$

where k_t is the index of the true token. The model's sentence probability factors as

$$p_\theta(\mathbf{x}_1, \ldots, \mathbf{x}_T) = \prod_{t=1}^{T} p_\theta(\mathbf{x}_t \mid \mathbf{x}_0, \ldots, \mathbf{x}_{t-1}).$$

So minimizing the summed cross-entropy is maximum likelihood for this probability (Chapter 2).

Inference. At each step take the output scores $S(\mathbf{x}_t)$ and choose a token by one of two rules.

- Greedy (deterministic): $\mathbf{x}_t = \arg\max S(\mathbf{x}_t)$.
- Probabilistic (stochastic): sample $\mathbf{x}_t \sim S(\mathbf{x}_t)$ from the softmax probabilities.

Feed the chosen token back as the next input, and stop when `<EOS>` is sampled. Greedy decoding always returns the same sentence. Sampling can also pick less likely words, so repeated runs give different sentences.

Machine translation reads a sentence in one language and generates its translation in another. This is the delayed many-to-many pattern.

6.2 Image captioning

The task is to describe an image $\mathbf{x}$ with a sentence $\mathbf{y}$. It splits into two steps.

1. Recognize the visual patterns in the image with a CNN g_ϕ (Chapter 7).
2. Generate a sentence conditioned on the image with an RNN language model.

The model is

$$f_\theta(\mathbf{y} \mid g_\phi(\mathbf{x})) :$$

the image features $g_\phi(\mathbf{x})$ condition the language model that produces $\mathbf{y}$, e.g. "man in black shirt is playing guitar." The lecture listed naive captioning and captioning with attention as the two variants. Attention-based captioning opens Chapter 10.

Naive captioning. Take a CNN pretrained on ImageNet and keep the feature vector of its last fully connected layer (dimension 4096 for VGG, 2048 for ResNet) for a $224 \times 224 \times 3$ input. A linear layer maps this vector to the word-embedding dimension, and the result is fed to the LSTM language model as if it were the first word. Then `<sos>` is fed, and generation continues as in Definition 9.10 ("Man", "in", ..., "guitar", `<eos>`). Training uses (image, sentence) pairs and the same word-level cross-entropy loss as sentence generation.

Example 9.11: the captioning loss (lecture example)

Here $\mathbf{y}_t$ is the true one-hot vector and $\hat{\mathbf{y}}_t$ the model's probability distribution over all words. Take the vocabulary {man, plays, guitar, `<EOS>`, `<SOS>`} and the target "man plays guitar". The inputs `<SOS>`, man, plays, guitar have targets man, plays, guitar, `<EOS>`, with one-hot vectors $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3, \mathbf{e}_4 \in \mathbb{R}^5$. If at step 3 the model outputs $\hat{\mathbf{y}}_3 = (0.05, 0.1, 0.8, 0.04, 0.01)$, then

$$\mathcal{L}_3 = -\sum_{k=1}^{5} y_{3,k} \log \hat{y}_{3,k} = -\log 0.8 \approx 0.223,$$

since only the true entry survives. The total loss is $\mathcal{L} = \sum_{k=1}^{4} \mathcal{L}_k$.

A practical issue. A strong language model can produce plausible sentences from the previous words alone, ignoring the image. The previous words are at dependency distance 1 from the next word, whereas the image enters only at the first step and must be carried through the whole hidden-state chain. So the conditioning on the image has to be strengthened.

- **Shortcuts.** Feed the image feature at every step as an extra input. This reduces the dependency distance to the image to 1, but the model can still learn to ignore it.

- **Attention.** Let the model look at different image regions for different words (Chapter 10). Using the same weight for every region (uniform attention) gives back a single global feature.

Chapter 10. Attention and Transformers

Lecture 11/2: attention-based captioning, spatial and channel attention, inductive bias, scaled dot-product, cross- and self-attention, positional encoding, masking, multi-head attention, Transformer encoder, layer normalization

1 Attention-Based Image Captioning

Motivation. Naive captioning (Chapter 9) feeds one global image vector once, and a strong language model learns to ignore it. Attention lets each generated word look at the image regions it describes, by recomputing a weighted summary of the spatial feature map at every step.

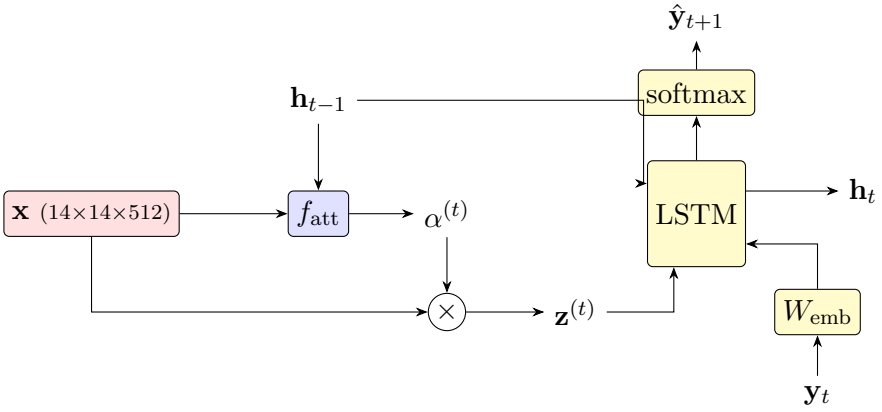
Instead of the fully connected feature vector, keep the last convolutional feature map of the pretrained CNN: for a $224 \times 224 \times 3$ input, conv5 of VGG has shape $14 \times 14 \times 512$. Write it as $L = 14 \cdot 14 = 196$ feature vectors $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$, one per spatial location. An **attention map** for a word is a nonnegative matrix $\alpha^{(t)} \in \mathbb{R}^{14\times 14}$ with the same spatial size as the feature map and $\sum_{i,j} \alpha^{(t)}_{i,j} = 1$. There is one per generated word ($\alpha^{(1)}$ for "a", $\alpha^{(2)}$ for "bird", ...), and it summarizes the image as $\mathbf{z}^{(t)} = \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} \in \mathbb{R}^{512}$. In sizes: $\mathbf{x} \in \mathbb{R}^{14\times 14\times 512}$, $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$, $\alpha^{(t)} \in \mathbb{R}^{14\times 14}$. Two questions follow: how to compute the attention, and how to use it to predict the word.

Definition 10.1: attention-based captioning (Show, Attend and Tell)

At step t, with previous hidden state $\mathbf{h}_{t-1}$ and current input word $\mathbf{y}_t$:

$$
\begin{aligned}
e^{(t)} &= f_{\text{att}}(\mathbf{x}, \mathbf{h}_{t-1}) && \text{(alignment scores, one per location)},\\
\alpha^{(t)}_{i,j} &= \frac{\exp(e^{(t)}_{i,j})}{\sum_{i',j'} \exp(e^{(t)}_{i',j'})} && \text{(attention weights)},\\
\mathbf{z}^{(t)} &= \sum_{i,j} \alpha^{(t)}_{i,j} \mathbf{x}_{i,j} && \text{(context vector)},\\
\mathbf{h}_t &= \text{LSTM}(\mathbf{h}_{t-1}, W_{\text{emb}}\mathbf{y}_t, \mathbf{z}^{(t)}) && \text{(modified LSTM)},\\
\hat{\mathbf{y}}_{t+1} &\propto \exp\left(W^o(W_{\text{emb}}\mathbf{y}_t + W^h\mathbf{h}_t + W^z\mathbf{z}^{(t)})\right) && \text{(next-word distribution)}.
\end{aligned}
$$

The modified LSTM takes the context vector as an extra input to every gate. The attention module f_{att} is a small MLP. The cell is repeated N times, once per generated word.



One step of attention-based captioning: the attention module turns $(\mathbf{x}, \mathbf{h}_{t-1})$ into weights $\alpha^{(t)}$, and the weighted feature $\mathbf{z}^{(t)}$ enters the LSTM.

The weights $\alpha^{(t)}_{i,j}$ are positive and sum to 1, so $\mathbf{z}^{(t)}$ is a weighted average of the location features.

"No attention" corresponds to uniform attention, every weight equal to $1/L$, which reduces to the naive model with the average-pooled feature.

Soft versus hard attention. The rule above is **soft attention**: a deterministic weighted average, differentiable in all parameters and trained by backpropagation. **Hard attention** instead samples one location $s_t \sim \text{Categorical}(\alpha^{(t)})$ and sets $\mathbf{z}^{(t)} = \mathbf{x}_{s_t}$. In the lecture's visualizations, soft attention spreads over the bird and the water, while hard attention picks one spot per word ("bird", "water"). The sampling step of hard attention has no gradient with respect to α, so hard attention is trained with score-function (REINFORCE-type) gradient estimators rather than plain backpropagation.

2 Spatial and Channel Attention

Attention weights can be placed on different axes of a feature map $U \in \mathbb{R}^{H \times W \times C}$.

- **Spatial attention** weights the $H \times W$ locations (Section 1).
- **Channel attention** weights the C channels.

Definition 10.2: squeeze-and-excitation block (SENet)

Given $U = [\mathbf{u}_1, \ldots, \mathbf{u}_C]$ with $\mathbf{u}_c \in \mathbb{R}^{H \times W}$:

$$
\begin{aligned}
s_c &= \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j) && \text{(squeeze: global average pooling)},\\
\mathbf{e} &= \sigma\big(W_2\ \mathrm{ReLU}(W_1 \mathbf{s})\big) && \text{(excitation)},\\
\tilde{\mathbf{u}}_c &= e_c\, \mathbf{u}_c && \text{(scale)},
\end{aligned}
$$

with $W_1 \in \mathbb{R}^{(C/r) \times C}$ and $W_2 \in \mathbb{R}^{C \times (C/r)}$. In words: global pooling, then a channel-wise weight, then multiplication with the original map.

Each $e_c \in (0, 1)$ comes from a sigmoid, not a softmax, so here the attention weights need not sum to 1. The block adds $2C^2/r$ weights (reduction ratio r, e.g. 16) and is inserted into residual blocks (SE-ResNet) or mobile blocks (ANTNets).

3 Inductive Bias

Motivation. We now have MLPs, CNNs, RNNs and attention. Why different models for different data? Each architecture builds in assumptions about its data, and attention is the one with the fewest.

So far we have seen four families:

- MLP: linear layers and nonlinear functions.
- CNN: convolution, pooling, nonlinearities and an MLP head.
- RNN: MLPs for the recurrent update, plus gating functions.
- Attention and Transformers: dot-product attention plus MLPs.

One can also mix data and models: a CNN for sequences (one-dimensional convolution), an RNN for images (scanning pixels in order), an MLP for images (flattened pixels).

Definition 10.3: inductive bias

The **inductive bias** of a learner is the set of assumptions it uses to predict outputs for inputs it has not observed.

Most architectural biases concern **locality** and **invariance**.

- Convolution exploits the 2D grid structure and translation equivariance (Chapter 7).
- An RNN exploits a 1D chain and a first-order Markov assumption: $\mathbf{h}_t$ depends on the past only through $\mathbf{h}_{t-1}$.

These assumptions make models efficient, but they often do not transfer across data modalities. The goal is one model for images, point clouds, audio and language. The key requirement is to make few assumptions about the input structure.

4 The General Attention Layer

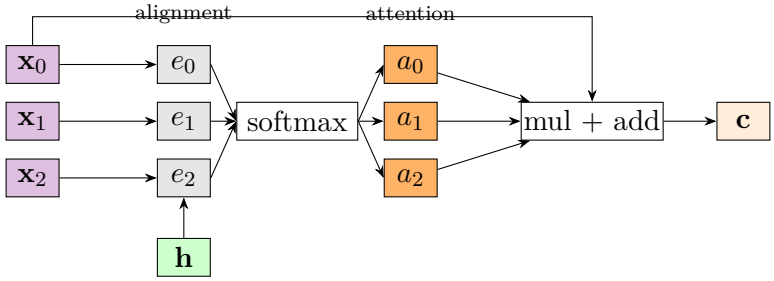
4.1 One query

Definition 10.4: attention with one query

Inputs: vectors $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (stacked as $\mathbf{x} \in \mathbb{R}^{N \times D}$) and a query $\mathbf{h} \in \mathbb{R}^D$.

$$\begin{aligned} \text{alignment:} \quad & e_i = f_{\text{att}}(\mathbf{h}, \mathbf{x}_i) = \frac{\mathbf{h} \cdot \mathbf{x}_i}{\sqrt{D}}, \\ \text{attention:} \quad & \mathbf{a} = \operatorname{softmax}(\mathbf{e}), \\ \text{output:} \quad & \mathbf{c} = \sum_{i=1}^{N} a_i \mathbf{x}_i \in \mathbb{R}^D \quad \text{(context vector)}. \end{aligned}$$

A feature map of size $H \times W \times D$ is stretched into $N = HW$ vectors. The operation does not care about the order of the features: reordering $\mathbf{x}_1, \dots, \mathbf{x}_N$ leaves $\mathbf{c}$ unchanged, since $\mathbf{c}$ is a sum. The slides call this **permutation invariance**.



The general attention layer with one query $\mathbf{h}$*: scores, softmax, then a weighted sum of the inputs.*

4.2 Why divide by $\sqrt{D}$

The lecture replaced a general f_{att} by a *scaled* dot product. The reasoning: in higher dimension the dot product has more terms, so the logits have larger variance. Large logits give a peaked, low-entropy softmax that puts almost all weight on one input. Dividing by $\sqrt{D}$ removes the growth.

In the lecture's computation, if the entries of $\mathbf{h}$ and $\mathbf{x}$ are independent with mean 0 and variance 1, then $z = \mathbf{h} \cdot \mathbf{x} = \sum_{m=1}^{D} h_m x_m$ has $\mathbb{E}z = 0$ and $\operatorname{Var} z = \sum_m \operatorname{Var}(h_m x_m) = D$. So the logits

have standard deviation $\sqrt{D}$, and dividing by $\sqrt{D}$ restores unit variance for every D. For large D without scaling, the largest logit is typically far above the others, the softmax is nearly one-hot, and almost no gradient passes through the attention weights.

4.3 Many queries, keys and values

Definition 10.5: multiple queries

With queries $\mathbf{q}_1, \dots, \mathbf{q}_M \in \mathbb{R}^D$ (stacked as $\mathbf{q} \in \mathbb{R}^{M \times D}$), each query produces its own context vector:

$$e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{x}_i}{\sqrt{D}}, \qquad \mathbf{a}_{\cdot,j} = \operatorname{softmax}(\mathbf{e}_{\cdot,j}), \qquad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{x}_i.$$

Here the inputs $\mathbf{x}$ play two roles, in the alignment and in the output. Adding a separate fully connected layer before each role makes the layer more expressive, and lets the input and output dimensions change.

Definition 10.6: cross-attention layer

With learnable $W_k \in \mathbb{R}^{D \times D_k}$ and $W_v \in \mathbb{R}^{D \times D_v}$ and queries $\mathbf{q} \in \mathbb{R}^{M \times D_k}$:

$$\mathbf{k} = \mathbf{x}W_k, \quad \mathbf{v} = \mathbf{x}W_v, \quad e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{k}_i}{\sqrt{D_k}}, \quad \mathbf{a}_{\cdot,j} = \operatorname{softmax}(\mathbf{e}_{\cdot,j}), \quad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{v}_i \in \mathbb{R}^{D_v}.$$

Definition 10.7: self-attention layer

If the queries are also computed from the inputs, $\mathbf{q} = \mathbf{x}W_q$ with $W_q \in \mathbb{R}^{D \times D_k}$, the layer maps N inputs to N outputs and is called **self-attention**. In matrix form, with rows as tokens,

$$\mathrm{SA}(\mathbf{x}) = \mathrm{Attn}(\mathbf{x}W_q, \mathbf{x}W_k, \mathbf{x}W_v) = \operatorname{softmax}\Big(\frac{\mathbf{x}W_q(\mathbf{x}W_k)^\top}{\sqrt{D_k}}\Big)\mathbf{x}W_v,$$

where the softmax acts on each row. This is the **scaled dot-product attention** of the Transformer, $\mathrm{Attn}(Q, K, V) = \operatorname{softmax}(QK^\top/\sqrt{d_k})V$.

The matrix $A = \operatorname{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n \times n}$ is row-stochastic, and row i of AV is a weighted average of all value vectors. So each output is a contextual encoding of the input whose receptive field is the whole sequence. This is the sense in which attention is **feature aggregation**.

The four steps in the lecture. With $\mathbf{x} \in \mathbb{R}^{n \times d}$ and projection matrices $W_q, W_k \in \mathbb{R}^{d \times d_k}$, $W_v \in \mathbb{R}^{d \times d_v}$, set $Q = \mathbf{x}W_q$ (the **query**, "the question"), $K = \mathbf{x}W_k$ (the **key**, "the index of the information") and $V = \mathbf{x}W_v$ (the **value**, "the information itself"). Then:

(i) $A = QK^\top \in \mathbb{R}^{n \times n}$, where $A_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ measures how much token i takes the information of token j into account;

(ii) $A' = QK^\top/\sqrt{d_k}$ (scaling): if the entries of $\mathbf{q}, \mathbf{k}$ are independent with mean 0 and variance 1, then $z = \sum_{m=1}^{d_k} q_m k_m$ has $\mathbb{E}z = 0$ and $\operatorname{Var} z = \sum_m \operatorname{Var}(q_m k_m) = d_k$, so dividing by $\sqrt{d_k}$ restores unit variance;

(iii) $\alpha_{ij} = \operatorname{softmax}(A')_{ij}$, row by row;

(iv) $\text{output}_i = \sum_j \alpha_{ij} \mathbf{v}_j$.

Cross versus self. In cross-attention the queries come from elsewhere (e.g. a decoder) and the keys and values from the input X; in self-attention Q, K and V all come from X. Attention-based captioning (Section 1) is cross-attention: the query is the state of the sentence being generated, and the keys and values are the image features. In the single-query layer of Definition 10.4 the keys and values coincide (both are $\mathbf{x}_i$).

5 Permutation Symmetry

Let P be a permutation matrix, so $P\mathbf{x}$ lists the same tokens in a different order. Cross-attention with fixed queries is **permutation invariant**: replacing $\mathbf{x}$ by $P\mathbf{x}$ reorders the keys and the values in the same way, so the weighted sums are unchanged. Self-attention is **permutation equivariant**: $\mathrm{SA}(P\mathbf{x}) = P\,\mathrm{SA}(\mathbf{x})$. Each score $\mathbf{q}_i \cdot \mathbf{k}_j$ depends only on which two tokens are compared, not on their positions, so reordering the inputs reorders the outputs in the same way. So self-attention alone treats a sentence as a set of words: "dog bites man" and "man bites dog" give the same outputs up to reordering. Word order has to be supplied separately.

6 Positional Encoding

Motivation. Self-attention is permutation equivariant, so it cannot see positions. We give each token a vector that encodes its position and attach it to the token's embedding.

Use a function $\mathrm{pos} : \mathbb{N} \to \mathbb{R}^d$ that turns position j into a d-dimensional vector $\mathbf{p}_j = \mathrm{pos}(j)$, and attach it to the input $\mathbf{x}_j$. The slides concatenate $\mathbf{p}_j$ to $\mathbf{x}_j$; the Transformer adds it, $\mathbf{x}_j + \mathbf{p}_j$. Two options for pos:

1. **Learn a lookup table**: a parameter vector $\mathrm{pos}(t)$ for each $t \in [0, T)$, so $T \times d$ parameters.
2. **Design a fixed function** with the desired properties.

The intuition for the fixed design is a binary counter. Writing $0, 1, 2, \ldots, 15$ in binary, the lowest bit alternates with period 2, the next with period 4, and so on. Each bit is a periodic function of the position, at a different frequency.

Definition 10.8: sinusoidal positional encoding

For even d and $k = 0, 1, \ldots, \frac{d}{2} - 1$, let $\omega_k = 1/10000^{2k/d}$ and

$$\mathrm{PE}(t)_{2k} = \sin(\omega_k t), \qquad \mathrm{PE}(t)_{2k+1} = \cos(\omega_k t).$$

Equivalently, $\mathrm{PE}(t) = \big(\sin\omega_0 t, \cos\omega_0 t,\ \ldots,\ \sin\omega_{d/2-1} t, \cos\omega_{d/2-1} t\big)$.

Any periodic functions can be used, or the encoding can be learned. Unlike a lookup table, the sinusoidal encoding is defined for every position t. The frequencies decrease geometrically, from $\omega_0 = 1$ down to about $1/10000$, like the bits of a binary counter: the first coordinates change quickly with the position and the last ones slowly.

Example 10.9: $d = 4$ (lecture example)

With $d = 4$ there are two frequencies, $\omega_0 = 1$ and $\omega_1 = 1/10000^{2/4} = 0.01$, so $\mathrm{PE}(t) =$

$(\sin t, \cos t, \sin 0.01t, \cos 0.01t)$ and

$$\mathrm{PE}(0) = (0, 1, 0, 1), \qquad \mathrm{PE}(1) \approx (0.841, 0.540, 0.010, 1.000),$$
$$\mathrm{PE}(2) \approx (0.909, -0.416, 0.020, 1.000).$$

The first pair changes quickly with t and the second slowly, like the low and high bits of a binary counter. With the encoding added, $\mathbf{z}_j = \mathbf{x}_j + \mathbf{p}_j$, a token keeps its embedding but shifts by its position: for $\mathbf{x}_{\text{dog}} = (1, 0, 2, 0)$,

$$\mathbf{x}_{\text{dog}} + \mathrm{PE}(0) = (1, 1, 2, 1), \qquad \mathbf{x}_{\text{dog}} + \mathrm{PE}(2) \approx (1.909, -0.416, 2.020, 1.000),$$

so "dog" at position 0 and at position 2 are different inputs, and "dog bites man" is no longer a reordering of "man bites dog".

7 Masked Self-Attention

When generating text, the model must not see future words. The lecture's example: training on "man plays guitar", the Transformer reads the whole shifted sentence at once and predicts every next token in parallel.

position	input	target
0	<SOS>	man
1	man	plays
2	plays	guitar
3	guitar	<EOS>

At position 1 the target is "plays"; if position 1 could attend to position 2, whose input is "plays", it would copy the answer. So a query at position j may only attend to positions $0, \ldots, j$. In self-attention, set the alignment scores from future positions to $-\infty$ by hand. Since $e^{-\infty} = 0$, those positions get zero weight.

Definition 10.10: masked (causal) self-attention

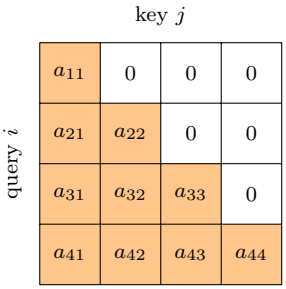
With $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$ as in Definition 10.7, query i uses the scores

$$e_{i,j} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_k}, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad \mathbf{y}_i = \sum_j a_{i,j} \mathbf{v}_j, \quad \mathbf{a}_{i,\cdot} = \mathrm{softmax}(\mathbf{e}_{i,\cdot}).$$

The slides index the same matrix the other way round, with the key index first: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D_k}$ for $i \le j$ and $-\infty$ for $i > j$, so for three tokens

$$E = \begin{pmatrix} e_{0,0} & e_{0,1} & e_{0,2} \\ -\infty & e_{1,1} & e_{1,2} \\ -\infty & -\infty & e_{2,2} \end{pmatrix}$$

(column j holds the scores of query j), the transpose of the lower-triangular pattern below. Every row of the attention matrix still sums to 1, and the output of position i uses only the tokens $1, \ldots, i$.



A causal attention matrix for four tokens: each query sees only itself and earlier tokens.

8 Multi-Head Attention

Definition 10.11: multi-head attention

With h heads, each with its own projections $W_q^{(r)}, W_k^{(r)} \in \mathbb{R}^{d\times d_k}$ and $W_v^{(r)} \in \mathbb{R}^{d\times d_v}$,

$$\text{head}_r = \text{Attn}\left(QW_q^{(r)}, KW_k^{(r)}, VW_v^{(r)}\right), \qquad \text{MHA}(Q,K,V) = [\text{head}_1, \ldots, \text{head}_h]\, W_o,$$

with $W_o \in \mathbb{R}^{hd_v\times d}$. Usually $d_k = d_v = d/h$.

Each head runs scaled dot-product attention (MatMul, Scale, optional Mask, SoftMax, MatMul) on its own linear projections, and the results are concatenated and mixed by a final linear layer. Different heads can attend to different relations, for example one to the previous word and one to the subject of the sentence.

9 The Transformer Encoder

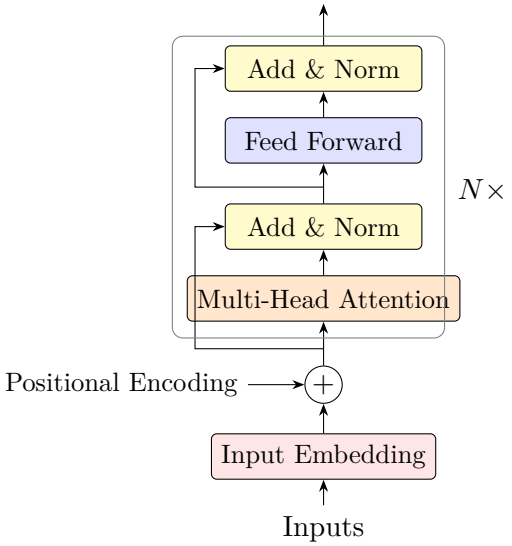
Motivation. The Transformer (Vaswani et al., 2017) stacks self-attention and position-wise MLPs, each with a skip connection and layer normalization.

Definition 10.12: Transformer encoder

1. The input is tokenized, $\mathbf{x} \in \mathbb{R}^{n\times d}$ after each token is encoded into a continuous embedding.
2. A positional encoding is added: $X_0 = \mathbf{x} + \text{PE}$.
3. Each of N layers applies multi-head self-attention with a skip connection, then a feed-forward network with a skip connection:

$$Z = \text{LN}\left(X + \text{MHA}(X,X,X)\right), \qquad X' = \text{LN}\left(Z + \text{FFN}(Z)\right),$$

   where $\text{FFN}(\mathbf{z}) = W_2\,\text{ReLU}(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$ is applied to each token separately.
4. The output of one layer is the input of the next.



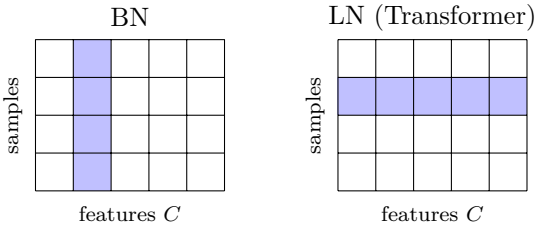
One encoder layer (inside the box), repeated N times; the side arrows are the skip connections.

Without PE every layer is still permutation equivariant, so positions enter only through PE. The cost of a layer is dominated by $QK^\top \in \mathbb{R}^{n\times n}$: $O(n^2 d)$ time and $O(n^2)$ memory in the sequence length n.

10 Layer Normalization

Batch normalization (Chapter 6) normalizes each channel using statistics over the mini-batch. Layer normalization normalizes each sample by itself.

- **BN**: for each channel C, average over the batch N and the positions H, W.
- **LN in CNNs**: for each sample, average over C, H, W.
- **LN in Transformers**: for each sample and each token, average over the C features only.



The shaded cells share one mean and variance: one feature over all samples (BN) versus all features of one sample (LN).

Definition 10.13: layer normalization

For $\mathbf{x} \in \mathbb{R}^d$, let $\mu = \frac{1}{d}\sum_k x_k$ and $\sigma^2 = \frac{1}{d}\sum_k (x_k - \mu)^2$. Then

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu \mathbf{1}}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta},$$

with learnable $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ and a small $\epsilon > 0$.

11 Why Transformers

The original Transformer was developed for machine translation. It turned out to scale surprisingly well with large datasets, and most large-scale models since (BERT, GPT-2, Megatron-LM, T5, Turing-NLG, with up to 17 billion parameters by early 2020) are built on it. Its weak inductive bias (only PE carries order) is a cost on small data and an advantage once data is plentiful.